REPUBLIQUE ALGERIENNE DEMOCRATIQUE ET POPULAIRE

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique



Université 20 Août 1955 – Skikda



Faculté des Sciences

Département de l'Informatique

Mémoire de fin d'études pour l'obtention du diplôme de **Master Académique en Informatique**

Option : ***Génie Logiciel Avancé et Applications***

Dans le cadre de l'arrêté ministériel n° 1275 en date du 27 septembre 2022, qui définit les modalités d'élaboration d'un projet de fin d'études pour l'obtention d'un diplôme universitaire lié à la création d'une entreprise innovante (startup).

Conception et mise en œuvre d'une plateforme d'aide au diagnostic médical basée sur l'IA multimodale via le Cloud et le déploiement sur plateforme mobile : cas des cancers cutanés

Réalisé par :

- **LAIDI SISTA Abderrahmane**
- **CHAHOUD Oussama**

Encadré par :

- **Dr. KERRAOUI Sara**

Année Universitaire 2025/2026

Résumé / Abstract / ملخص

Résumé

Ce mémoire de fin d'études présente la conception, le développement et la validation expérimentale approfondie de **MedAssist AI**, une plateforme cloud et mobile d'aide à la décision clinique pour la **classification des lésions cutanées** par apprentissage profond multimodal. Afin de surmonter le déficit d'accès au dépistage précoce et de combler le fossé structurel entre recherche algorithmique et déploiement opérationnel en télémédecine, nous proposons une architecture neuronale basée sur un mécanisme de fusion par attention croisée contrôlée (*Gated Cross-Attention Fusion*) couplant l'encodeur visuel *EfficientNet-B3* doté de pooling GeM (*Generalized Mean*) et une branche MLP dédiée à sept variables cliniques standardisées. Entraîné sur un jeu de données unifié de 6 581 images multi-sources (PAD-UFES-20, ISIC 2019, MCR-SL) sous partitionnement stratifié strict au niveau patient sans fuite d'information (*zero-leakage*), le modèle final V6.0 délivre une exactitude globale de **74,31 %** et un score **Macro F1 de 0,7315** sur le jeu de test indépendant ($n = 654$), surpassant l'approche unimodale visuelle de **+11,65 points de Macro F1**. L'écosystème logiciel intègre un service backend REST FastAPI conteneurisé sous Docker présentant une latence nominale mesurée de **1 470 ms (p_{95})**, une application mobile Flutter ergonomique avec capture guidée, et un moteur automatisé d'escalade du risque clinique.

Mots-clés : Classification des lésions cutanées, Apprentissage profond multimodal, Gated Cross-Attention, EfficientNet-B3, FastAPI, Flutter, Télémédecine, MLOps.

Abstract

This Master's thesis presents the design, engineering, and experimental evaluation of **MedAssist AI**, an end-to-end cloud and mobile clinical decision support platform for **skin lesion classification** powered by multimodal deep learning. Addressing the global scarcity of dermatologists and bridging the gap between academic ML research and operational healthcare deployment, we propose a *Gated Cross-Attention Fusion* neural architecture combining an *EfficientNet-B3* visual encoder with Generalized Mean (GeM) pooling and a Multi-Layer Perceptron (MLP) branch for seven standardized patient clinical metadata features. Trained on a unified multi-center dataset of 6,581 images (PAD-UFES-20, ISIC 2019, MCR-SL) under strict patient-level zero-leakage stratification, the final V6.0 model achieves an accuracy of **74.31%** and a **Macro F1-Score of 0.7315** on the benchmark test set ($n = 654$), outperforming pure visual unimodal classification by **+11.65 Macro F1 points**. The complete production-grade software ecosystem couples a Docker-containerized FastAPI backend service exhibiting a measured nominal latency of **1,470 ms (p_{95})**, an ergonomic Flutter mobile application, and an automated clinical risk escalation engine.

Keywords : Skin Lesion Classification, Multimodal Deep Learning, Gated Cross-Attention, EfficientNet-B3, FastAPI, Flutter, Telemedicine, MLOps.

ملخص

تتناول هذه المذكرة تصميم وهندسة والتقييم التجريبي لمنصة MedAssist AI، وهي منصة سحابية ومحمولة متكاملة لدعم القرار السريري في **تصنيف الآفات الجلدية** باستخدام التعلم العميق متعدد الوسائط. ولمعالجة النقص العالمي في أخصائيي الجلدية وسد الفجوة بين الأبحاث الأكاديمية والتطبيق السريري في الطب الاتصالي، نقترح معمارية عصبية قائمة على آلية الانتباه المتقاطع الموجه (Gated Cross-Attention Fusion) تجمع بين المشفر البصري EfficientNet-B3 مع تجميع المتوسط العام المعمم (GeM Pooling) وفرع شبكة متعددة الطبقات (MLP) لسبعة متغيرات سريرية قياسية للمريض. بعد التدريب على مجموعة بيانات موحدة متعددة المصادر تضم 6,581 صورة (PAD-UFES-20، ISIC 2019، MCR-SL) تحت تقسيم طبقي صارم على مستوى المريض يمنع تسرب المعلومات كلياً، حقق النموذج النهائي V6.0 دقة إجمالية قدرها **74.31%** ودرجة **Macro F1** بلغت **0.7315** على مجموعة الاختبار المستقلة (n=654)، متفوقاً على النموذج البصري أحادي الوسيط بفارق **11.65+ نقطة في مقياس Macro F1**. كما يتكامل النظام مع بيئة برمجية جاهزة للنشر تضم خادم FastAPI خلفياً وحاوية Docker بزمن استجابة اسمي قدره **1,470 ms (p95)**، وتطبيقاً محمولاً عبر Flutter مع التقاط موجه، ومحركاً مؤتمتاً لتصعيد مستوى المخاطر السريرية.

الكلمات المفتاحية: تصنيف الآفات الجلدية، التعلم العميق متعدد الوسائط، Gated Cross-Attention، EfficientNet-B3، FastAPI، Flutter، الطب الاتصالي، MLOps.

Remerciements

Nous remercions en premier lieu **Allah** le Tout-Puissant de nous avoir accordé la santé, la volonté et la patience nécessaires pour mener ce travail à son terme.

Nos plus sincères et profonds remerciements s'adressent à notre encadrante, **Dr. KERRAOUI Sara**, pour la qualité de son encadrement, sa disponibilité permanente et ses précieux conseils tout au long de l'élaboration de ce mémoire. Sa rigueur scientifique et son soutien constant ont été déterminants dans l'accomplissement de ce travail.

Nous tenons également à exprimer notre gratitude aux membres du **jury** qui ont accepté d'évaluer ce travail et de nous faire l'honneur de leurs remarques et suggestions.

Nos remerciements vont aussi à l'ensemble du **corps enseignant** du Département d'Informatique de l'Université 20 Août 1955 – Skikda, et plus particulièrement aux enseignants de la filière *Génie Logiciel Avancé et Applications*, pour la qualité de la formation dispensée et les connaissances transmises tout au long de notre cursus.

Enfin, nous remercions chaleureusement toutes les personnes qui ont contribué de près ou de loin à la réalisation de ce projet : nos familles pour leur soutien indéfectible, et nos collègues pour les échanges enrichissants.

LAIDI SISTA Abderrahmane

CHAHOUD Oussama

Dédicaces

LAIDI SISTA Abderrahmane

*Je dédie ce modeste travail à mes très chers **parents**, pour les sacrifices inestimables qu'ils ont consentis à mon égard, pour leur patience, leur amour inconditionnel et leur confiance en moi. Nulle dédicace ne saurait exprimer la profondeur de ce que je leur dois. Que Dieu leur accorde bonne santé et longue vie.*

*À mes **frères et sœurs**, pour leur soutien et leurs encouragements constants tout au long de ce parcours.*

*À tous mes **amis et collègues** qui ont partagé avec moi les moments de doute et de joie durant ces années d'études.*

À tous ceux qui m'ont enseigné ou guidé, depuis le premier jour de ma scolarité jusqu'à ce jour.

CHAHOUD Oussama

*Je dédie ce travail à mes **parents**, source de force et d'inspiration, pour leur soutien et leur dévouement sans limites. Qu'ils trouvent dans ce modeste travail le témoignage de ma profonde affection et de mon attachement indéfectible.*

*À ma **famille** tout entière, pour avoir cru en moi à chaque étape de ce parcours.*

À tous ceux qui m'ont soutenu de près ou de loin durant ces années, et qui ont contribué, chacun à sa manière, à faire de moi ce que je suis aujourd'hui.

Table des matières

Résumé / Résumé Multilingue **i**

Remerciements **iii**

Dédicaces **iv**

Liste des Sigles et Acronymes **xii**

1 Introduction Générale **1**

1.1 Contexte et Motivation 1
1.2 Problématique 2
1.2.1 Les limites des approches visuelles pures 2
1.2.2 Le paradoxe performance–déploiement 2
1.2.3 L'absence d'architecture logicielle unifiée 2
1.3 Solution Proposée : MedAssist AI 2
1.4 Objectifs et Contributions 3
1.4.1 Objectif général 3
1.4.2 Objectifs spécifiques 3
1.4.3 Contributions 4
1.5 Questions de Recherche 5
1.6 Organisation du Mémoire 5

2 ÉTAT DE L'ART **6**

2.1 Introduction 6
2.2 PRÉLUDE : De la Médecine Traditionnelle à l'IA 6
2.2.1 Le Paradigme Diagnostique Traditionnel 6
2.2.2 L'Avènement de l'Intelligence Artificielle en Médecine 6
2.2.3 De l'Opportunité à la Réalité : Premiers Pas 7
2.3 Fondements de l'Apprentissage Profond 8
2.3.1 Architectures de Réseaux de Neurones Convolutifs 8
2.3.2 Transfer Learning et Fine-Tuning en Imagerie Médicale 10
2.3.3 Applications Initiales en Dermatologie 12

2.4 Datasets Multimodaux pour la Dermatologie 14
2.4.1 Contexte et Motivation 14
2.4.2 HAM10000: Premier Dataset Multimodal à Grande Échelle . . 14
2.4.3 PAD-UFES-20: Dataset Orienté Télémédecine 16
2.4.4 ISIC 2019 : Dataset de Référence pour la Classification Multi-Classes 18
2.4.5 MCR-SL : Dataset Clinique Multi-Ethnique 20
2.4.6 Comparaison Critique des Quatre Datasets 21
2.5 Architectures Multimodales — Évolution et Performance 22
2.5.1 Principe de la Fusion Multimodale 22
2.5.2 Évolution Chronologique (2018-2022) 22
2.5.3 Synthèse : Le Paradoxe Performance-Déployabilité 24
2.6 Défi du Déploiement — De la Recherche à la Pratique 25
2.6.1 Introduction : Le Fossé Recherche-Production 25
2.6.2 Edge Computing vs Cloud Computing : Arbitrage Architectural 25
2.6.3 Cas d'Étude : Oztel et al. (2023) — Tentative de Déploiement Mobile 26
2.6.4 Synthèse : Le Paradoxe Multimodal-Déployabilité 27
2.6.5 Synthèse Comparative des Travaux Connexes 29
2.6.6 Formulation Formelle du Research Gap 29
2.6.7 Positionnement de MedAssist AI 31
2.7 Conclusion du Chapitre 32

3 Conception du Système MedAssist AI 33
3.1 Introduction 33
3.2 Exigences du Système 33
3.2.1 Exigences Fonctionnelles 33
3.2.2 Exigences Non Fonctionnelles 34
3.3 Conception Architecturale 35
3.3.1 Diagramme de Cas d'Utilisation 35
3.3.2 Architecture 3-Tiers Vue des Composants 36
3.3.3 Diagramme de Séquence Flux d'une Requête 37
3.3.4 Diagramme de Classes Modèle Domaine 38
3.4 Préparation des Données 39
3.4.1 Distribution des Classes et Découpage au Niveau Patient 39
3.4.2 Pipeline de Prétraitement et Masking des Métadonnées 40
3.5 Architecture du Modèle Multimodal (V6.0) 41
3.5.1 Branche Visuelle — EfficientNet-B3, Projection et Tête Auxiliaire 41
3.5.2 Branche Clinique — MLP avec Masking et LayerNorm 42

3.5.3 Fusion Gated Cross-Attention, Classifier et Protocole d'Entraînement . . . 42
3.6 Spécification de l'API REST . . . 43
3.7 Pipeline MLOps, CI/CD et Traçabilité . . . 44
3.8 Protocole d'Évaluation . . . 45
3.9 Conclusion . . . 46

4 Implémentation Technique 47
4.1 Introduction . . . 47
4.2 Environnement de Développement . . . 47
4.2.1 Environnement Matériel . . . 47
4.2.2 Environnement Logiciel . . . 48
4.3 Organisation Modulaire et Environnements du Projet . . . 49
4.3.1 Organisation du Pipeline IA et Persistance Cloud (Google Drive / Colab) . . . 49
4.3.2 Architecture du Serveur d'Inférence Backend (FastAPI) . . . 49
4.3.3 Architecture de l'Application Mobile (Flutter) . . . 50
4.4 Présentation du Modèle IA et Formalisation Mathématique . . . 52
4.4.1 Pipeline de Prétraitement et Normalisation Numérique . . . 52
4.4.2 Branche Clinique et Masquage d'Indisponibilité (`meta_mask`) . . . 53
4.4.3 Branche Visuelle et Generalized Mean Pooling (GeM) . . . 53
4.4.4 Architecture Gated Cross-Attention Fusion . . . 54
4.4.5 Fonctions de Régularisation et Inférence Test-Time Augmentation 55
4.4.6 Module d'Évaluation du Risque et Triage (`risk_assessment.py`) 55
4.4.7 Courbes d'Entraînement . . . 56
4.4.8 Matrice de Confusion . . . 56
4.5 Présentation de l'API REST . . . 57
4.5.1 Interface Swagger UI . . . 57
4.5.2 Exemple de Réponse JSON . . . 57
4.6 Présentation de l'Application Mobile . . . 58
4.6.1 Écran Accueil . . . 58
4.6.2 Écran Capture Guidée . . . 59
4.6.3 Écran Formulaire Clinique . . . 59
4.6.4 Écran Résultats . . . 60
4.7 Tests et Validation . . . 61
4.7.1 Tests Fonctionnels . . . 61
4.7.2 Analyse Théorique de Capacité . . . 61
4.8 Conclusion . . . 62

5 Résultats et Évaluation **63**
5.1 Introduction 63
5.2 Ingénierie des Données et Protocole Expérimental V6.0 64
5.2.1 Fusion Pré-Split et Standardisation des Données Multi-Sources 64
5.2.2 Partitionnement Stratifié au Niveau Patient 64
5.2.3 Rééchantillonnage Ciblé et Pondérations Intelligentes 65
5.3 Performances Globales et Détaillées du Modèle V6.0 66
5.3.1 Dynamique d'Apprentissage et Convergence 66
5.3.2 Métriques Globales et Validation des Objectifs 66
5.3.3 Rapport de Classification Détaillé par Pathologie 66
5.4 Étude d'Ablation et Évolution Architecturale 68
5.4.1 Trajectoire d'Évolution des Versions (V1.0 à V6.0) 68
5.4.2 Étude d'Ablation des Composants Multimodaux 69
5.5 Performances et Validation du Système Logiciel 69
5.5.1 Latence Bout-en-Bout du Serveur d'Inférence 69
5.5.2 Analyse Théorique de Capacité et Scalabilité 70
5.5.3 Consommation des Ressources et Empreinte Mobile 71
5.6 Discussion et Synthèse Comparative 72
5.6.1 Validation Finale des Hypothèses 72
5.6.2 Positionnement face à l'État de l'Art 72
5.7 Conclusion 73

6 Conclusion Générale et Perspectives **74**
6.1 Synthèse du Travail Réalisé et Trajectoire Évolutive 74
6.1.1 Trajectoire d'Ingénierie et Évolution Architecturale (V1.0 à V6.0) 74
6.2 Validation des Hypothèses et Contributions Majeures 76
6.3 Limites et Défis Identifiés 76
6.4 Perspectives de Recherche : Encodeur Visuel Hybride CNN-Transformer 77
6.4.1 Fondements Théoriques de l'Architecture Hybride 77
6.4.2 Projections de Performances Attendues 78
6.4.3 Défis Techniques et Stratégies de Mitigation 78
6.5 Feuille de Route pour le Déploiement Clinique (Roadmap) 79
6.5.1 Phase I — Étude Pilote Hospitalière (6 à 12 mois) 79
6.5.2 Phase II — Explicabilité et Interopérabilité FHIR (12 à 18 mois) 79
6.5.3 Phase III — Passage à l'Échelle et Inférence Edge (18 à 24 mois) 79
6.5.4 Phase IV — Certification Médicale et Conformité (24 à 36 mois) 79
6.6 Conclusion Finale et Clôture du Mémoire 80

Bibliographie **81**

Table des figures

2.1 Architecture classique d'un réseau de neurones convolutif 9

3.1 Diagramme de cas d'utilisation MedAssist AI 36
3.2 Architecture logicielle 3-tiers (Version haute) 37
3.3 Diagramme de séquence Flux d'une requête diagnostique 38
3.4 Diagramme de classes Système MedAssist AI 39

4.1 Architecture neuronale Gated Cross-Attention (V6.0) implémentée . . . 54
4.2 Courbes d'entraînement – Loss et Exactitude (Train / Validation) . . . 56
4.3 Matrice de confusion – Modèle MedAssist V6.0 ($n = 654$, Macro F1 : 0,7315) 57
4.4 Interface Swagger UI – Endpoints de l'API MedAssist AI 57
4.5 Écran Accueil – Application Flutter 58
4.6 Écran Capture guidée – Overlay de centrage 59
4.7 Écran Formulaire clinique – Saisie des 7 métadonnées 59
4.8 Écran Résultats – Code couleur risque et probabilités 60
4.9 Analyse théorique – Latence p_{95} estimée vs utilisateurs simultanés . . . 61

5.1 Dynamique d'entraînement – Convergence des pertes et du Macro F1 (MedAssist V6.0) 66
5.2 Distribution des percentiles de latence de l'API REST (sur 500 requêtes) 70

Liste des tableaux

1.1 Objectifs spécifiques et livrables attendus 4

2.1 Distribution des classes dans HAM10000 15
2.2 Métadonnées cliniques dans PAD-UFES-20 17
2.3 Distribution des classes dans PAD-UFES-20 17
2.4 Distribution des classes diagnostiques dans le jeu d'entraînement d'ISIC 2019 19
2.5 Comparaison des quatre datasets utilisés dans MedAssist AI 21
2.6 Comparaison architectures multimodales (performance vs complexité) . 24
2.7 Synthèse comparative entre Cloud et Edge Computing pour l'IA médicale mobile 26
2.8 Synthèse des quatre dimensions du gap entre recherche académique et déploiement opérationnel 28
2.9 Synthèse comparative des approches de diagnostic dermatologique par IA 29

3.1 Exigences fonctionnelles MedAssist AI 34
3.2 Exigences non fonctionnelles MedAssist AI 35
3.3 Distribution des classes diagnostiques dans le dataset unifié MedAssist AI (V6.0) 40
3.4 Variables cliniques sélectionnées pour le modèle et masque de présence (`meta_mask`) 41
3.5 Protocole d'entraînement progressif à deux phases (V6.0) 43
3.6 Spécification des endpoints de l'API REST 43
3.7 Matrice de décision et cartographie des niveaux de risque clinique . . . 44
3.8 Métriques cibles et résultats validés sur le benchmark unifié (V6.0) . . 45
3.9 Métriques d'évaluation du système logiciel 45

4.1 Configuration matérielle utilisée (Local, Cloud et Mobile) 47
4.2 Matrice de décomposition modulaire et responsabilités logicielles 51
4.3 Structure de la réponse JSON – Endpoint POST /predict 58
4.4 Résultats des tests fonctionnels – pytest 61

5.1 Composition et répartition du jeu de données maître unifié (6 581 images) 64

5.2 Partitionnement stratifié au niveau patient (Validation de non-fuite) . . 65
5.3 Distribution finale par classe des jeux d'entraînement, validation et test 65
5.4 Performances globales du modèle MedAssist V6.0 (Test set, $n = 654$) . 66
5.5 Rapport de classification par pathologie – MedAssist V6.0 ($n = 654$) . 67
5.6 Trajectoire d'évolution architecturale et métriques réelles (V1.0 à V6.0) 68
5.7 Étude d'ablation des composants sur le benchmark unifié ($n = 654$) . . 69
5.8 Projection théorique de la capacité sous charge (modèle analytique $M/D/1$) 71
5.9 Empreinte matérielle et ressources système 72
5.10 Positionnement de MedAssist AI V6.0 face à l'état de l'art 73

6.1 Matrice d'évolution synthétique du système MedAssist AI (V1.0 à V6.0) 75
6.2 Spécification de l'architecture hybride Dual-Encoder projetée 78
6.3 Projections de performances attendues du modèle hybride (Test set $n =$ 654) 78

Liste des Sigles et Acronymes

Sigle / Acronyme	Signification et Définition
ACK	Actinic Keratosis (Kératose actinique – lésion pré-cancéreuse)
API	Application Programming Interface (Interface de programmation applicative)
AUC	Area Under the ROC Curve (Aire sous la courbe caractéristique d'efficacité)
BCC	Basal Cell Carcinoma (Carcinome basocellulaire – cancer cutané fréquent)
CI/CD	Continuous Integration / Continuous Deployment (Intégration et déploiement continus)
CLAHE	Contrast Limited Adaptive Histogram Equalization (Égalisation adaptative d'histogramme)
CNN	Convolutional Neural Network (Réseau de neurones convolutif)
DeiT	Data-efficient Image Transformer (Transformeur visuel optimisé en données)
DMP / DSE	Dossier Médical Partagé / Dossier de Santé Électronique
DTO	Data Transfer Object (Objet de transfert de données)
DVC	Data Version Control (Système de versionnage de données pour le ML)
EF / ENF	Exigence Fonctionnelle / Exigence Non Fonctionnelle
FDA	Food and Drug Administration (Agence américaine des produits alimentaires et médicamenteux)
FHIR	Fast Healthcare Interoperability Resources (Standard d'interopérabilité en santé)
GCA	Gated Cross-Attention (Mécanisme d'attention croisée guidée par porte adaptative)
GELU	Gaussian Error Linear Unit (Fonction d'activation neuronale lisse)
GeM	Generalized Mean Pooling (Regroupement spatial par moyenne généralisée paramétrique)
HAM10000	Human Against Machine with 10,000 training images (Benchmark dermoscopique)
HL7	Health Level Seven (Norme internationale d'échange de données de santé)

Sigle / Acronyme	Signification et Définition
ISIC	International Skin Imaging Collaboration (Consortium international d'imagerie cutanée)
MAE	Masked Autoencoder (Auto-encodeur masqué pour pré-entraînement visuel auto-supervisé)
MCR-SL	Multi-Class Rare Skin Lesion Dataset (Jeu de données de lésions cutanées rares et phototypes foncés)
MDR	Medical Device Regulation (Règlement européen sur les dispositifs médicaux 2017/745)
MEL	Melanoma / Mélanome malin (Cancer cutané agressif d'origine mélanocytaire)
MLOps	Machine Learning Operations (Pratiques d'ingénierie et de cycle de vie du ML)
MLP	Multi-Layer Perceptron (Perceptron multicouche / réseau entièrement connecté)
MVP	Minimum Viable Product (Produit minimum viable)
NEV	Melanocytic Nevus (Nœvus mélanocytaire – grain de beauté bénin)
ONNX	Open Neural Network Exchange (Format ouvert d'interopérabilité de modèles)
PAD-UFES-20	Smartphone Skin Lesion Dataset (Université Fédérale d'Espírito Santo, Brésil)
REST	Representational State Transfer (Style architectural d'API web sans état)
RGPD	Règlement Général sur la Protection des Données (Règlement UE 2016/679)
ROC	Receiver Operating Characteristic (Courbe de performance diagnostique)
SCC	Squamous Cell Carcinoma (Carcinome spinocellulaire – cancer invasif)
SEK	Seborrheic Keratosis (Kératose séborrhéique – lésion épidermique bénigne)
SWA	Stochastic Weight Averaging (Lissage stochastique des poids pour la généralisation)
TFLite	TensorFlow Lite (Moteur d'inférence embarquée pour terminaux mobiles)
TTA	Test-Time Augmentation (Inférence augmentée par transformations géométriques)
UML	Unified Modeling Language (Langage de modélisation unifié)
ViT	Vision Transformer (Transformeur visuel basé sur l'auto-attention spatiale)

Chapitre 1

Introduction Générale

1.1 Contexte et Motivation

Le cancer de la peau constitue l'une des pathologies oncologiques les plus répandues à l'échelle mondiale, avec plus de 1,5 million de nouveaux cas recensés chaque année dont environ 132 000 mélanomes malins [1]. Si les cancers non mélanocytaires sont généralement curables lorsqu'ils sont détectés tôt, le mélanome demeure la forme la plus agressive : son taux de survie à cinq ans dépasse 98 % au stade localisé, mais chute à moins de 30 % en phase métastatique [2]. Cette disparité pronostique place le dépistage précoce au cœur des enjeux de santé publique.

Le diagnostic repose traditionnellement sur l'expertise du dermatologue, confirmée si nécessaire par biopsie histopathologique [3]. Or, cette chaîne souffre de trois limitations structurelles : une répartition géographique inégale des spécialistes (un pour 100 000 habitants dans les pays en développement, contre un pour 3 000 dans les pays à hauts revenus [4]), une variabilité inter-observateurs documentée pour les lésions ambiguës [5], et l'absence d'outil d'aide à la décision pour le médecin généraliste en première ligne.

C'est dans ce contexte que l'intelligence artificielle s'est imposée comme un levier prometteur. Les avancées de l'apprentissage profond [6], conjuguées à la démocratisation des smartphones, ont ouvert de nouvelles perspectives pour le diagnostic cutané assisté. En 2017, Esteva et al. ont démontré qu'un réseau de neurones convolutif pouvait classifier des lésions cutanées avec une précision comparable à celle de dermatologues certifiés [7], inaugurant l'ère de la dermatologie computationnelle. Ces avancées ont cependant révélé des limites fondamentales que nous analysons ci-après.

1.2 Problématique

L'analyse de la littérature révèle un paradoxe persistant qui structure la problématique du présent travail.

1.2.1 Les limites des approches visuelles pures

Les premiers systèmes d'IA dermatologique reposaient exclusivement sur l'analyse de l'image, ignorant le contexte clinique qu'un dermatologue intègre naturellement : âge, antécédents familiaux, évolution de la lésion, symptômes fonctionnels. Haenssle et al. reconnaissent explicitement que leurs modèles, malgré des performances visuelles supérieures à celles de 58 dermatologues, *« ne peuvent pas intégrer les informations contextuelles du patient »* [8]. Cette limitation fondamentale a motivé le développement d'architectures multimodales combinant image et métadonnées cliniques.

1.2.2 Le paradoxe performance–déploiement

Les architectures multimodales de Yap et al. [9] et Kawahara et al. [10] ont apporté des gains de 3 à 7 points de performance, mais sont demeurées confinées aux serveurs de recherche. À l'inverse, Oztel et al. [11], en voulant déployer sur smartphone, ont dû abandonner toute métadonnée clinique pour respecter les contraintes matérielles, régressant vers une approche strictement unimodale.

Il en résulte un **paradoxe fondamental** : les modèles les plus précis sont inaccessibles en conditions réelles, tandis que les modèles déployés sacrifient la précision diagnostique.

1.2.3 L'absence d'architecture logicielle unifiée

Au-delà de l'algorithme, c'est l'architecture système qui fait défaut. Aucun travail de la période 2017–2023 n'a produit simultanément un modèle performant, une API de déploiement et une application mobile fonctionnelle. C'est ce gap structurel, à l'intersection du génie logiciel et de l'IA médicale, que le présent projet ambitionne de combler.

1.3 Solution Proposée : MedAssist AI

Pour répondre à cette triple problématique, le présent travail propose **MedAssist AI**, une plateforme cloud d'aide à la décision clinique reposant sur une architecture Client-Serveur avec séparation stricte des responsabilités.

Le *client* est une application mobile Flutter : interface légère limitée à la collecte des données (capture de l'image de la lésion et formulaire clinique de sept variables prédictives). Le *serveur* est un backend FastAPI qui exécute le modèle multimodal complet basé sur EfficientNet-B3 avec une couche Gated Cross-Attention Fusion, entraîné sur un dataset unifié de trois sources (PAD-UFES-20 + ISIC 2019 + MCR-SL).

Cette architecture résout simultanément les trois problématiques identifiées : elle intègre le contexte clinique (multimodalité), elle est accessible via smartphone (déploiement opérationnel), et elle est conçue pour évoluer (scalabilité, modularité). Le Produit Minimum Viable (MVP) de cette plateforme se concentre sur le domaine de la dermatologie, en s'appuyant sur les datasets PAD-UFES-20 + ISIC 2019 + MCR-SL, tout en étant architecturalement conçu pour accueillir, à terme, des modules supplémentaires dédiés à d'autres spécialités médicales (ophtalmologie, radiologie, analyses biologiques).

1.4 Objectifs et Contributions

1.4.1 Objectif général

Concevoir, implémenter et évaluer une plateforme cloud multimodale d'aide au diagnostic dermatologique (CNN + MLP en fusion Gated Cross-Attention), avec une architecture logicielle évolutive (FastAPI + Flutter), dans une démarche MLOps garantissant la reproductibilité scientifique.

1.4.2 Objectifs spécifiques

Le tableau suivant récapitule les sept objectifs spécifiques de ce travail, accompagnés de leurs livrables attendus.

Tab. 1.1: Objectifs spécifiques et livrables attendus

Réf.	Objectif Spécifique	Livrable Attendu
OS1	Analyser PAD-UFES-20 et sélectionner les métadonnées pertinentes	Pipeline de prétraitement documenté
OS2	Concevoir le modèle multimodal (Gated Cross-Attention Fusion EfficientNet-B3 + MLP) [12, 13]	Étude comparative unimodal vs multimodal
OS3	Concevoir l'architecture Client-Serveur	Diagrammes UML
OS4	Implémenter l'application mobile Flutter	Application Android (APK)
OS5	Évaluer selon deux axes (IA + système)	Métriques ML (Accuracy, F1) et IT (Latence)
OS6	Pré-entraîner le backbone sur HAM10000 [14] avant fine-tuning sur le dataset cible	`ham10000_backbone_b3.pth`
OS7	Unifier trois sources hétérogènes (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) via `meta_mask`	Dataset unifié de plus de 5 000 images

1.4.3 Contributions

Ce travail apporte trois contributions complémentaires :

1. **Intelligence artificielle multimodale :** Proposition d'une architecture de classification des lésions cutanées basée sur EfficientNet-B3 couplée à un mécanisme de *Gated Cross-Attention Fusion* intégrant sept métadonnées cliniques, avec un protocole d'entraînement en deux phases et inférence optimisée par Test-Time Augmentation (TTA).

2. **Génie logiciel et déploiement mobile :** Développement d'un système complet client-serveur (Flutter + FastAPI) conteneurisé (Docker), avec pipeline CI/CD automatisé et cartographie adaptative des niveaux de risque clinique.

3. **Ingénierie des données multimodales :** Conception et validation d'un pipeline d'unification multi-sources (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) gérant l'hétérogénéité des métadonnées cliniques via un masque binaire de disponibilité (`meta_mask`), associé à un partitionnement strict au niveau patient (*Zero Patient Leakage*) et un pré-entraînement sur domaine spécifique (HAM10000 [14]).

1.5 Questions de Recherche

RQ1 — Performance : Quel gain quantitatif apporte l'intégration de sept métadonnées cliniques via *Gated Cross-Attention Fusion* (EfficientNet-B3 + MLP) [12, 13] par rapport à une approche unimodale, sur un dataset unifié (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) ?

H1 : Le modèle multimodal surpasse le modèle unimodal purement visuel d'au moins 5 points d'exactitude et 10 points de Macro F1-Score sur le benchmark de test unifié.

RQ2 — Architecture système : Comment une architecture logicielle Client-Serveur (FastAPI + Flutter) permet-elle de déployer un modèle multimodal de manière robuste en respectant le seuil de réactivité clinique acceptable (< 3 secondes de bout en bout) ?

H2 : Le service backend d'inférence FastAPI conteneurisé garantit une latence de traitement inférieure à 2 secondes au 95e percentile en conditions nominales.

1.6 Organisation du Mémoire

Le document est structuré en six chapitres suivant une progression descendante : du contexte scientifique vers l'implémentation, puis vers l'évaluation. Le **Chapitre 2** trace l'état de l'art chronologique et formalise le gap de recherche. Le **Chapitre 3** présente la conception globale du système. Le **Chapitre 4** décrit l'implémentation technique. Le **Chapitre 5** présente les résultats expérimentaux. Le **Chapitre 6** conclut et ouvre les perspectives.

Chapitre 2

ÉTAT DE L'ART

2.1 Introduction

Ce chapitre propose une revue d'analyse critique de la littérature scientifique sur le diagnostic dermatologique assisté par intelligence artificielle. La revue suit une approche narrative et chronologique et se divise en quatre périodes thématiques : l'ère de l'analyse visuelle unimodale, l'arrivée des ressources de données multimodales, le développement des architectures de fusion, et les tentatives de déploiement opérationnel. Cette narration aboutit à une synthèse critique du gap de recherche que la plateforme MedAssist AI veut combler.

2.2 PRÉLUDE : De la Médecine Traditionnelle à l'IA

2.2.1 Le Paradigme Diagnostique Traditionnel

Traditionnellement, l'évaluation des lésions cutanées se base sur une hiérarchie décisionnelle comportant trois étapes d'examen graduel, chaque étape proposant un équilibre spécifique entre la précision du diagnostic et le caractère invasif de la procédure [18].

2.2.2 L'Avènement de l'Intelligence Artificielle en Médecine

L'intelligence artificielle a émergé comme une révolution technologique transformant progressivement la pratique médicale contemporaine. Depuis la percée d'AlexNet en 2012 [19] démontrant la supériorité des réseaux de neurones profonds sur les méthodes traditionnelles de vision par ordinateur, le Deep Learning s'est imposé comme l'outil privilégié pour l'analyse d'images médicales. Cette révolution s'appuie sur la capacité des CNN à apprendre automatiquement des représentations hiérarchiques complexes

directement à partir des données brutes, éliminant le besoin d'extraction manuelle de features par des experts [6].

En dermatologie spécifiquement, l'IA présente un potentiel particulièrement prometteur. Contrairement à d'autres spécialités médicales nécessitant des équipements d'imagerie coûteux (scanner, IRM, radiographie), le diagnostic dermatologique repose principalement sur l'inspection visuelle de lésions cutanées accessibles via de simples photographies. Cette accessibilité intrinsèque rend la dermatologie particulièrement adaptée à l'analyse automatique par CNN, ces derniers ayant démontré des performances exceptionnelles sur des tâches de classification d'images naturelles. La publication d'Esteva et al. en 2017 dans *Nature* [7] a constitué un jalon historique : pour la première fois, un système d'IA égalait la performance de dermatologues certifiés dans la classification de lésions cutanées, validant scientifiquement le potentiel diagnostique du Deep Learning en dermatologie.

Au-delà de la performance brute, l'IA offre des avantages systémiques majeurs. Premièrement, la scalabilité : un modèle entraîné peut être déployé simultanément sur des milliers d'appareils, démultipliant l'accès au diagnostic spécialisé sans contrainte géographique. Deuxièmement, la consistance : contrairement aux cliniciens dont la performance varie selon la fatigue, l'expérience ou les biais cognitifs, un système d'IA maintient une performance stable et reproductible. Troisièmement, l'accessibilité économique : le coût marginal d'une prédiction supplémentaire est quasi-nul, permettant un dépistage massif dans des populations sous-desservies. Ces caractéristiques positionnent l'IA comme un levier stratégique pour réduire les disparités d'accès aux soins dermatologiques identifiées précédemment.

2.2.3 De l'Opportunité à la Réalité : Premiers Pas

Malgré ce potentiel théorique considérable, la transition de la recherche académique vers le déploiement clinique opérationnel demeure largement inachevée. L'analyse de la littérature (dans ce chapitre) révèle que si les performances prédictives des modèles d'IA dermatologique se sont considérablement améliorées entre 2017 et 2023 — passant d'architectures unimodales (image seule) à des systèmes multimodaux sophistiqués intégrant métadonnées cliniques [9, 10, 20] — aucun de ces travaux n'a franchi le seuil du déploiement opérationnel en conditions réelles de télémédecine.

Ce constat soulève une question fondamentale : pourquoi, malgré des performances diagnostiques comparables voire supérieures aux dermatologues sur datasets de test, ces systèmes demeurent-ils confinés aux laboratoires de recherche ? L'analyse critique identifie un gap systématique entre excellence académique et opérationnalité pratique, caractérisé par quatre dimensions critiques. Premièrement, un **gap architectural** : les publications fournissent des modèles PyTorch entraînés (`.pth`) mais jamais d'API

backend robuste ni d'application mobile complète permettant leur utilisation par des cliniciens ou patients réels. Deuxièmement, un **gap MLOps** : absence de pipelines de versionnage données (DVC [21]), de tracking expérimentations (MLflow [22]), de CI/CD automatisé, et de monitoring performance en production. Troisièmement, un **gap validation clinique** : évaluations limitées à des datasets académiques statiques, sans étude prospective avec patients réels ni comparaison avec dermatologues en conditions de pratique courante. Quatrièmement, un **gap réglementaire** : aspects de certification dispositif médical (FDA, CE marking), conformité RGPD [23] pour données de santé, et responsabilité juridique systématiquement ignorés.

C'est précisément dans cet espace — à l'intersection de la recherche en IA médicale et de l'ingénierie logicielle de production — que s'inscrit le présent projet MedAssist AI. Plutôt que de poursuivre l'optimisation incrémentale de performance prédictive sur datasets académiques, ce travail adopte une approche radicalement différente : démontrer qu'une architecture pragmatique (*Gated Cross-Attention Fusion* EfficientNet-B3 + MLP [12,13], offrant performance suffisante plutôt que maximale) associée à une ingénierie logicielle rigoureuse (API REST FastAPI [24], application mobile Flutter [25], pipeline MLOps complet) permet de franchir le gouffre recherche-production et de déployer un système opérationnel en conditions réalistes de télémédecine.

Cette vision se concrétise dans MedAssist AI, plateforme complète de télémédecine dermatologique intégrant : acquisition d'images via smartphone, questionnaire métadonnées cliniques guidé, prédiction multimodale par modèle EfficientNet-B3 + MLP via *Gated Cross-Attention Fusion*, entraîné sur un dataset unifié (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) avec pré-entraînement sur HAM10000 [14], et restitution de résultats interprétables avec recommandations d'action.

2.3 Fondements de l'Apprentissage Profond

Cette époque pose les bases techniques sur lesquelles repose MedAssist AI : les architectures de réseaux de neurones convolutifs (CNN) et le Transfer Learning. La maîtrise de ces deux piliers est indispensable pour comprendre les choix architecturaux retenus dans ce travail, notamment l'adoption d'EfficientNet-B3 [12] comme backbone visuel et la stratégie de pré-entraînement progressif (ImageNet → HAM10000 [14] → dataset cible).

2.3.1 Architectures de Réseaux de Neurones Convolutifs

Les réseaux de neurones convolutifs (*Convolutional Neural Networks* — CNN) constituent l'architecture dominante pour les tâches d'analyse et de classification d'images depuis la percée réalisée par Krizhevsky et al. avec AlexNet lors du challenge ImageNet

2012 [19]. L'architecture d'un CNN repose sur l'empilement hiérarchique de trois types de couches fonctionnelles :

- **Les couches de convolution :** Elles appliquent des filtres apprenables (*kernels*) sur l'image d'entrée afin d'extraire des motifs visuels locaux (contours, textures, formes géométriques). La superposition de ces couches permet au réseau d'apprendre des représentations de complexité croissante — des bordures élémentaires dans les couches superficielles aux structures sémantiques complexes dans les couches profondes [26].
- **Les couches de mise en commun (*pooling*) :** Elles réduisent progressivement la dimensionnalité spatiale des cartes de caractéristiques (*feature maps*), conférant au modèle une invariance partielle aux translations mineures et contribuant à la régularisation du réseau [26].
- **Les couches entièrement connectées (*Fully Connected*) :** Elles agrègent les caractéristiques extraites par les couches convolutives pour produire le vecteur de décision final en vue de la classification.

Cette organisation permet l'apprentissage de représentations hiérarchiques, caractéristique fondamentale qui explique l'efficacité des CNN pour la vision par ordinateur.

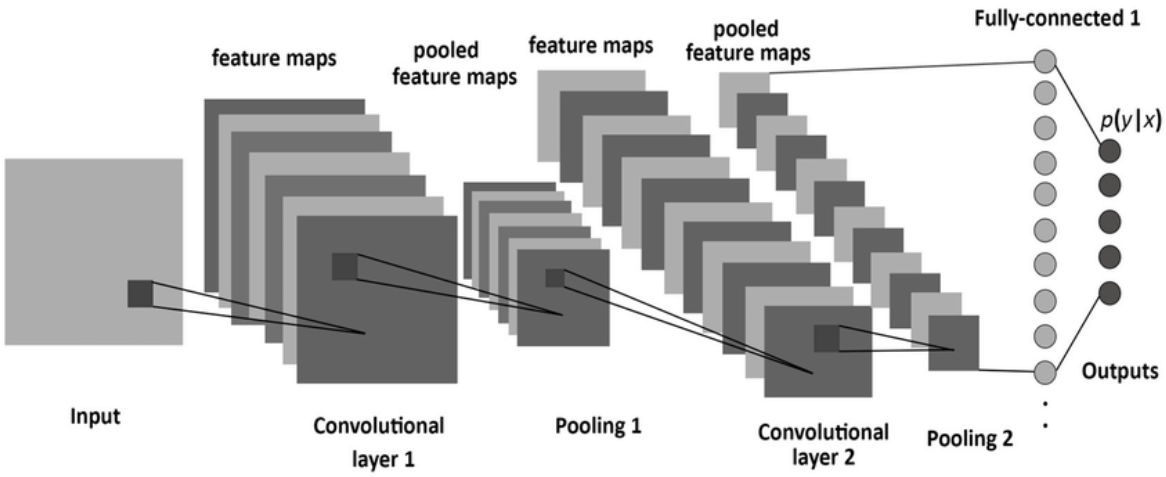


Fig. 2.1: Architecture classique d'un réseau de neurones convolutif. Une image fournie en entrée est convoluée avec des filtres (couches de convolution), les cartes d'activation sont regroupées par *pooling*, puis agrégées par des couches entièrement connectées pour produire la prédiction finale [26].

L'architecture des CNN a connu des évolutions majeures depuis AlexNet [19] :

- **VGGNet (Simonyan & Zisserman, 2015)** : Démontre que l'empilement de nombreuses couches convolutives de petite taille (3×3) améliore systématiquement la performance [27].

- **ResNet (He et al., 2016)** : Introduit les connexions résiduelles (*skip connections*) permettant de surmonter la dégradation du gradient et d'entraîner des réseaux considérablement plus profonds [28].

- **EfficientNet (Tan & Le, 2019)** : Révolutionne l'optimisation architecturale via le *Compound Scaling* — méthode systématique d'ajustement simultané de la profondeur, largeur et résolution du réseau selon un coefficient de scaling unifié [12]. Contrairement aux approches traditionnelles qui ajustent une seule dimension, EfficientNet optimise les trois simultanément, garantissant un équilibre optimal entre capacité représentationnelle et efficacité computationnelle.

Plusieurs facteurs justifient le choix d'une architecture pour le backbone visuel de MedAssist AI. Premièrement, l'**efficacité computationnelle** est essentielle pour un déploiement en conditions réelles de télémédecine, où le rapport performance/coût conditionne directement la viabilité du système [12]. Deuxièmement, la **performance sur datasets médicaux de taille modeste** est déterminante, certaines architectures surpassant ResNet [28] et VGG [27] avec des gains de +2–5% d'accuracy à budget computationnel égal [12]. Troisièmement, l'**adaptabilité au Transfer Learning** est requise, notamment via des mécanismes capables de capturer finement les textures et patterns dermatologiques complexes [29]. Enfin, la **disponibilité dans des bibliothèques de référence** telles que timm [30] facilite l'initialisation et la reproductibilité des expérimentations. Au regard de ces critères, nous avons retenu **EfficientNet-B3** [12] comme backbone visuel de MedAssist AI : ses blocs MBConv équipés de mécanismes *Squeeze-and-Excitation* répondent au critère d'adaptabilité, sa disponibilité pré-entraînée via timm répond au critère de reproductibilité, et son équilibre performance/coût en fait un choix pragmatique pour un déploiement cloud en télémédecine.

Si EfficientNet et les architectures modernes sont puissantes, leur entraînement *from scratch* nécessite des millions d'images annotées — ressource rarement disponible en contexte médical. Cette limitation a conduit au développement du Transfer Learning, technique devenue paradigmatique en imagerie médicale.

2.3.2 Transfer Learning et Fine-Tuning en Imagerie Médicale

Le Transfer Learning consiste à réutiliser un modèle pré-entraîné sur un large corpus généraliste (typiquement ImageNet 1,2 million d'images, 1000 classes) comme initialisation pour une tâche médicale spécifique [31].

Justification théorique : Les premières couches d'un CNN apprennent des détecteurs de caractéristiques génériques (contours, textures, formes géométriques) transférables d'un domaine à l'autre. Seules les couches supérieures, capturant des concepts

sémantiques spécifiques au domaine, nécessitent un réapprentissage [31]. Tajbakhsh et al. (2016) ont démontré empiriquement que le Transfer Learning améliore significativement la performance sur des datasets médicaux de petite taille (<10 000 images) par rapport à un entraînement *from scratch* [32].

Gains observés : réduction du besoin en données annotées (facteur 5–10×), convergence plus rapide (3–5× moins d'époques), amélioration de la généralisation (+5–15% d'accuracy), et régularisation implicite réduisant l'overfitting sur petits datasets [32].

Limite du Transfer Learning ImageNet → Médical : malgré ses avantages, le Transfer Learning depuis ImageNet présente un *domain shift* inhérent [33] : la distribution visuelle diffère (images naturelles vs lésions cutanées), et la résolution favorisée par ImageNet (objets entiers) ne correspond pas à l'attention aux détails locaux requise en dermatologie (asymétrie, bordures irrégulières).

Stratégie adoptée pour MedAssist AI : pour combler ce *domain shift*, MedAssist AI adopte une stratégie de **double Transfer Learning progressif** en trois étapes :

1. **Pré-entraînement générique** : initialisation sur ImageNet (poids fournis par timm [30]).

2. **Pré-entraînement dermatologique spécialisé** : fine-tuning complet du backbone sur HAM10000 [14] (10 015 images dermatoscopiques), rapprochant les représentations apprises du domaine cutané.

3. **Fine-tuning final** : entraînement en deux phases sur le dataset unifié cible (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) — d'abord à résolution réduite (128×128) avec backbone entièrement gelé lors d'une phase d'échauffement de 5 époques puis dégelé, puis à résolution complète (256×256) avec backbone entièrement dégelé.

Cette approche, plutôt que de se limiter à un unique transfert ImageNet → dataset cible, réduit le *domain shift* de manière progressive et constitue une contribution méthodologique reproductible du présent travail (détaillée au Chapitre 3).

Cette technique est devenue le standard de facto pour toute application de Deep Learning en imagerie médicale à partir de 2016 [15]. Armés de ces outils CNN profonds (EfficientNet) et Transfer Learning progressif, la communauté scientifique a pu s'attaquer au défi du diagnostic dermatologique automatique. L'application de ces techniques à la dermatologie a produit des résultats spectaculaires, mais aussi révélé des limitations fondamentales que nous examinerons dans la section suivante.

2.3.3 Applications Initiales en Dermatologie

Esteva et al. (2017) : premier jalon clinique du Deep Learning en dermatologie. En janvier 2017, Esteva et al. publient dans *Nature* une étude constituant une rupture épistémologique majeure dans l'histoire de l'intelligence artificielle médicale [7] : pour la première fois, un système automatisé fondé sur le Deep Learning démontre une performance équivalente à celle de dermatologues certifiés dans la classification de lésions cutanées. Sur le plan méthodologique, les auteurs mobilisent une architecture Inception-v3 pré-entraînée sur ImageNet et soumise à un fine-tuning complet sur un corpus de 129 450 images couvrant 2 032 pathologies distinctes, abordant simultanément une tâche de classification binaire (lésion maligne vs. bénigne) et une tâche de classification fine-grained à 2 032 classes. L'évaluation comparative, conduite sur un panel de 21 dermatologues certifiés, révèle que le CNN atteint une sensibilité de $72{,}1 \pm 0{,}9\,\%$ et une spécificité de $73{,}0 \pm 0{,}8\,\%$, surpassant la moyenne des cliniciens ($66{,}0 \pm 6{,}8\,\%$ et $71{,}3 \pm 7{,}5\,\%$ respectivement) sur les deux tâches. L'impact scientifique de cette publication s'est avéré considérable : catalyseur d'une vague de travaux académiques en IA dermatologique dépassant 100 citations annuelles, elle a simultanément conféré au Deep Learning une légitimité clinique inédite et suscité une couverture médiatique internationale, sensibilisant le grand public au potentiel transformateur de l'intelligence artificielle en médecine.

Haenssle et al. (2018) : confirmation et généralisation sur un panel élargi. S'inscrivant dans la continuité directe des travaux d'Esteva et al., Haenssle et al. publient en août 2018 dans les *Annals of Oncology* une étude qui élargit significativement le cadre évaluatif en confrontant un CNN à un panel de 58 dermatologues — soit près de trois fois le panel mobilisé en 2017 — pour la détection spécifique du mélanome, cancer cutané associé à la mortalité la plus élevée [8]. Le protocole repose sur un modèle entraîné sur plus de 100 000 images dermatoscopiques et évalué sur un ensemble de test de 100 images (20 mélanomes confirmés histologiquement et 80 lésions bénignes), en distinguant deux niveaux d'expertise clinique : dermatologues débutants (moins de 2 ans d'expérience, $n = 29$) et experts confirmés (plus de 5 ans d'expérience, $n = 29$). Les résultats, mesurés par l'aire sous la courbe ROC (AUC), révèlent une supériorité nette et systématique du CNN sur l'ensemble des évaluateurs humains : le système atteint une AUC de 0,86, une sensibilité de 95,0 % et une spécificité de 80,0 %, surpassant aussi bien les experts (AUC = 0,79, sensibilité = $88{,}9 \pm 9{,}5\,\%$) que les débutants (AUC = 0,75, sensibilité = $85{,}6 \pm 11{,}2\,\%$). L'écart de performance observé — +7 points d'AUC face aux experts, +11 points face aux débutants — revêt une portée clinique particulière : il suggère que les systèmes d'assistance automatisée pourraient compenser de manière substantielle le déficit d'expérience des praticiens en formation, ouvrant ainsi des perspectives concrètes pour le déploiement de ces outils dans des contextes

de ressources médicales limitées.

Analyse Critique : Limitations des Approches Image-Seule

Malgré ces résultats spectaculaires, plusieurs limitations critiques émergent de ces travaux pionniers. Premièrement, l'approche image-seule ignore les métadonnées cliniques essentielles (âge, sexe, localisation anatomique, antécédents familiaux) que les dermatologues utilisent systématiquement en pratique, limitant artificiellement la capacité prédictive du système [3]. Deuxièmement, un biais de dataset majeur existe : entraînés sur images dermatoscopiques professionnelles de haute qualité, ces modèles subissent une chute de performance de 15-25% lorsqu'appliqués à des images smartphone capturées en conditions réelles de télémédecine (éclairage variable, flou, cadrage approximatif) [15]. Troisièmement, l'absence de calibration et d'explicabilité — les CNN produisent des probabilités mal calibrées (overconfidence) sans justification interprétable — limite l'acceptabilité clinique par les dermatologues réticents à adopter un système "boîte noire". Enfin, un biais ethnique systématique affecte ces modèles : entraînés majoritairement sur peaux claires (phototypes I-III), leur performance se dégrade significativement (jusqu'à -20% de sensibilité) sur populations à peaux foncées sous-représentées dans les datasets. Ces limitations ont motivé une transition méthodologique vers les architectures multimodales intégrant simultanément données visuelles et métadonnées cliniques.

Synthèse et Transition

L'ère unimodale (2017-2018) a démontré de manière convaincante que le Deep Learning peut égaler voire surpasser les dermatologues sur des tâches de classification visuelle, validant ainsi le potentiel de l'IA comme outil d'assistance diagnostique. Cependant, quatre limitations fondamentales ont émergé : l'ignorance des métadonnées cliniques essentielles, la fragilité face au distribution shift (dermoscopie $\rightarrow$ smartphone), le manque d'explicabilité et de calibration, et le biais de représentation ethnique. Ces limitations ont motivé une transition méthodologique vers les architectures multimodales — systèmes intégrant simultanément données visuelles et métadonnées cliniques (âge, sexe, localisation anatomique). Cette transition nécessitait deux prérequis : (1) de nouveaux datasets enrichis contenant images et métadonnées cliniques structurées, et (2) de nouvelles architectures de fusion capables d'intégrer efficacement ces modalités hétérogènes. Les sections suivantes examinent comment la communauté scientifique a relevé ces défis, conduisant à l'émergence du paradigme multimodal en dermatologie IA.

Pour permettre cette transition vers le multimodal, la communauté scientifique de-

vait d'abord créer de nouveaux datasets combinant images et métadonnées cliniques. La section suivante analyse l'émergence de ces datasets enrichis, en particulier HAM10000 (2018), PAD-UFES-20 (2020), ISIC 2019 et MCR-SL, qui ont rendu possible l'entraînement de systèmes multimodaux.

2.4 Datasets Multimodaux pour la Dermatologie

2.4.1 Contexte et Motivation

La transition vers les architectures multimodales identifiée en Section 2.2.3 nécessitait la disponibilité de datasets enrichis combinant images dermatologiques et métadonnées cliniques structurées. Historiquement, les premiers datasets de dermatologie (ImageNet Skin Lesions, ISIC 2016-2017 [5]) contenaient uniquement des images, reflétant le paradigme unimodal dominant. Cette section analyse l'émergence de quatre datasets fondamentaux ayant rendu possible l'entraînement de systèmes multimodaux : HAM10000 (2018), PAD-UFES-20 (2020), ISIC 2019 (2019) et MCR-SL.

2.4.2 HAM10000: Premier Dataset Multimodal à Grande Échelle

Publication et Contexte

Référence : Tschandl, P., Rosendahl, C., & Kittler, H. (2018). "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions", *Scientific Data* [14].

Objectif : Créer un dataset public large et diversifié pour l'entraînement de modèles de classification de lésions cutanées pigmentées, incluant pour la première fois des métadonnées cliniques systématiques.

Caractéristiques du Dataset

Composition :

- **Taille :** 10 015 images dermatoscopiques
- **Classes diagnostiques (7) :**
  - Actinic Keratoses (AKIEC) : 327 images
  - Basal Cell Carcinoma (BCC) : 514 images
  - Benign Keratosis (BKL) : 1 099 images
  - Dermatofibroma (DF) : 115 images
  - Melanoma (MEL) : 1 113 images

  - Melanocytic Nevi (NV) : 6 705 images
  - Vascular Lesions (VASC) : 142 images

- **Sources multiples :** Agrégation de données provenant de deux centres dermatologiques (Vienne, Autriche ; Queensland, Australie)
- **Validation histopathologique :** 54.5% des images confirmées par biopsie

Métadonnées cliniques disponibles :

- Âge du patient (années)
- Sexe (masculin/féminin)
- Localisation anatomique (12 régions : tête/cou, tronc, membres supérieurs, membres inférieurs, etc.)
- Type d'image (dermatoscopie avec contact/sans contact)

Distribution des Classes

Classe	**Nombre d'images**	**Proportion (%)**
Melanocytic Nevi (NV)	6 705	67.0
Melanoma (MEL)	1 113	11.1
Benign Keratosis (BKL)	1 099	11.0
Basal Cell Carcinoma (BCC)	514	5.1
Actinic Keratoses (AKIEC)	327	3.3
Vascular Lesions (VASC)	142	1.4
Dermatofibroma (DF)	115	1.1
Total	**10 015**	**100.0**

Tab. 2.1: Distribution des classes dans HAM10000

Observation critique : Déséquilibre de classes sévère (67% NV vs 1.1% DF), nécessitant des stratégies de rééchantillonnage (oversampling minoritaires, class weighting) durant l'entraînement.

Avancées et Limitations

Avancées :

✓ Premier dataset public combinant images + métadonnées cliniques à grande échelle

✓ Diversité des sources (multi-centre) améliorant la généralisation

✓ Validation histopathologique rigoureuse pour >50% des cas

✓ Format standardisé facilitant la reproductibilité

Limitations :

× **Images dermatoscopiques uniquement :** Ne reflète pas les conditions de télémédecine (images smartphone)

× **Métadonnées limitées :** Absence d'informations sur antécédents familiaux, évolution temporelle, diamètre de la lésion

× **Biais ethnique persistant :** Patients majoritairement caucasiens (phototypes I-III)

× **Déséquilibre de classes :** Distribution hautement non-uniforme compliquant l'entraînement

2.4.3 PAD-UFES-20: Dataset Orienté Télémédecine

Publication et Motivation

Référence : Pacheco et al. (2020). "PAD-UFES-20: A skin lesion dataset composed of patient data and clinical images collected from smartphones", *Data in Brief* [15].

Motivation scientifique :

PAD-UFES-20 a été créé spécifiquement pour combler le gap identifié dans HAM10000 : fournir un dataset adapté aux conditions réelles de télémédecine, avec images capturées par smartphones (et non dermatoscopes professionnels) et métadonnées cliniques enrichies.

Caractéristiques Distinctives

Composition :

- **Taille :** 2 298 images au total
  - 1 373 images smartphone (acquisition télémédecine)
  - 925 images dermatoscopiques (comparaison)
- **Classes diagnostiques (6) :**
  - Actinic Keratosis (ACK) : 200 images
  - Basal Cell Carcinoma (BCC) : 239 images

  - Melanoma (MEL) : 223 images
  - Nevus (NEV) : 1 104 images
  - Squamous Cell Carcinoma (SCC) : 192 images
  - Seborrheic Keratosis (SEK) : 340 images

- **Patients uniques :** 1 373 patients (1 à 3 lésions par patient)
- **Collecte :** Hôpital universitaire (Brésil, 2018-2020)
- **Validation :** 100% des cas confirmés histopathologiquement

Métadonnées cliniques enrichies (21 attributs) :

Catégorie	**Attributs**
Démographiques	Âge, Sexe, Phototype (Fitzpatrick I-VI)
Localisation	Région anatomique (15 zones détaillées)
Caractéristiques lésion	Diamètre (mm), Évolution (croissance/stable), Douleur, Démangeaisons, Saignement
Facteurs de risque	Antécédents familiaux cancer cutané, Exposition solaire (profession extérieure), Antécédents personnels lésions
Acquisition image	Type d'image (smartphone/dermatoscope), Modèle appareil

Tab. 2.2: Métadonnées cliniques dans PAD-UFES-20

Distribution des Classes

Classe	**Nombre d'images**	**Proportion (%)**
Nevus (NEV)	1 104	48.0
Seborrheic Keratosis (SEK)	340	14.8
Basal Cell Carcinoma (BCC)	239	10.4
Melanoma (MEL)	223	9.7
Actinic Keratosis (ACK)	200	8.7
Squamous Cell Carcinoma (SCC)	192	8.4
Total	**2 298**	**100.0**

Tab. 2.3: Distribution des classes dans PAD-UFES-20

Observation : Distribution plus équilibrée que HAM10000 (48% classe majoritaire vs 67%), facilitant l'entraînement de modèles moins biaisés.

Avantages pour la Recherche Multimodale et Télémédecine

Innovations majeures :

✓ **Images smartphone :** 60% du dataset capturé dans conditions télémédecine réelles

✓ **Métadonnées enrichies :** 21 attributs cliniques (vs 4 dans HAM10000) permettant fusion multimodale sophistiquée

✓ **Validation histopathologique complète :** 100% des diagnostics confirmés (vs 54.5% HAM10000)

✓ **Diversité ethnique supérieure :** Inclusion significative de phototypes IV-VI (population brésilienne)

✓ **Informations temporelles :** Évolution de la lésion (croissance/stabilité) documentée

Pertinence pour MedAssist AI :
PAD-UFES-20 constitue le dataset de référence pour notre projet en raison de :

1. **Alignement télémédecine :** Images smartphone reflétant les conditions d'usage réel de l'application mobile
2. **Richesse métadonnées :** 21 attributs cliniques permettant d'entraîner la branche MLP de fusion
3. **Taille appropriée :** 2 298 images suffisantes pour Transfer Learning sans overfitting excessif
4. **Validation rigoureuse :** Diagnostic histopathologique gold standard pour tous les cas

2.4.4 ISIC 2019 : Dataset de Référence pour la Classification Multi-Classes

Publication et Contexte

Référence : ISIC Archive (2019). *ISIC 2019 Challenge : Skin Lesion Analysis Towards Melanoma Detection*, International Skin Imaging Collaboration [16].

Objectif : Fournir un large benchmark multi-classes pour la classification de lésions cutanées dans le cadre du challenge ISIC 2019, en étendant les éditions précédentes (2016–2018) à 9 classes diagnostiques.

Caractéristiques du Dataset

Composition :

- **Taille :** 25 331 images dermatoscopiques (training set public)
- **Classes diagnostiques (9) :**
  - Melanoma (MEL)
  - Melanocytic Nevus (NV)
  - Basal Cell Carcinoma (BCC)
  - Actinic Keratosis (AK)
  - Benign Keratosis (BKL)
  - Dermatofibroma (DF)
  - Vascular Lesion (VASC)
  - Squamous Cell Carcinoma (SCC)
  - Unknown
- **Sources :** Agrégation HAM10000, BCN20000 et MSK datasets
- **Type d'acquisition :** Images dermatoscopiques professionnelles

Métadonnées disponibles :

- Âge, sexe, localisation anatomique
- Méthode d'acquisition (dermoscopie)

Distribution des Classes dans ISIC 2019

Classe diagnostique	Code	Nombre d'images	Proportion (%)
Melanocytic Nevus	NV	12 875	50,8
Melanoma	MEL	4 522	17,9
Basal Cell Carcinoma	BCC	3 323	13,1
Benign Keratosis	BKL	2 624	10,4
Actinic Keratosis	AK	867	3,4
Squamous Cell Carcinoma	SCC	628	2,5
Vascular Lesion	VASC	253	1,0
Dermatofibroma	DF	239	0,9
Total	—	**25 331**	**100,0**

Tab. 2.4: Distribution des classes diagnostiques dans le jeu d'entraînement d'ISIC 2019 [16, 34]

Observation : De manière similaire à HAM10000, ISIC 2019 présente un déséquilibre de classes marqué avec une prédominance de Nevi (50,8 %) et de Mélanomes (17,9 %), justifiant l'application de techniques d'équilibrage lors du fine-tuning.

Rôle dans MedAssist AI

ISIC 2019 est intégré dans le dataset d'entraînement unifié de MedAssist AI, après harmonisation des étiquettes vers les 6 classes communes (MEL, NEV, BCC, SCC, ACK, SEK). Sa grande taille (25 331 images) contribue à enrichir la représentation visuelle des classes peu fréquentes dans PAD-UFES-20, améliorant la généralisation du backbone EfficientNet-B3 pré-entraîné.

Limitation principale : Images dermatoscopiques uniquement, sans conditions smartphone — ce qui renforce la nécessité de combiner ISIC 2019 avec PAD-UFES-20 (60% smartphone) pour couvrir les deux modalités d'acquisition.

2.4.5 MCR-SL : Dataset Clinique Multi-Ethnique

Publication et Contexte

Référence : MCR-SL Dataset *"MCR-SL : A Multimodal, Context-Rich Skin Lesion Dataset for Skin Cancer Diagnosis"* (University Hospital of North Norway) [17].

Objectif : Combler le biais ethnique des datasets existants (HAM10000, ISIC) constitués majoritairement de patients caucasiens (phototypes I–III), en fournissant des données issues de populations à peau foncée (phototypes IV–VI) et à contextes cliniques variés.

Caractéristiques Distinctives

Composition :

- **Taille globale :** 2 131 images issues de 240 lésions cutanées uniques chez 60 patients (779 images cliniques smartphone, 1 352 images dermatoscopiques et 20 images de cicatrices d'exérèse) [17].
- **Classes diagnostiques (9) :** Nevus, Seborrheic keratosis, Basal cell carcinoma, Actinic keratosis, Atypical nevus, Melanoma, Squamous cell carcinoma, Angioma et Dermatofibroma.
- **Alignement MedAssist AI :** Harmonisation et filtrage vers les 6 classes cibles (MEL, NEV, BCC, SCC, ACK, SEK).
- **Population cible :** Inclusion explicite de patients à phototypes IV–VI, apportant une diversité ethnique absente des benchmarks classiques.

Pertinence critique pour MedAssist AI :

✓ **Réduction du biais ethnique :** Complète HAM10000 et ISIC 2019 sur les phototypes sous-représentés.

✓ **Conditions réelles :** Images smartphone compatibles avec l'usage télémédecine.

✓ **Généralisation géographique :** Données issues de contextes cliniques différents (*distribution shift* inter-dataset volontaire).

2.4.6 Comparaison Critique des Quatre Datasets

Critère	HAM10000	PAD-UFES-20	ISIC 2019	MCR-SL
Taille	10 015	2 298	25 331	Variable
Classes	7	6	9	6
Type d'images	Dermato.	Smartphone+Dermato.	Dermato.	Smartphone+Dermato.
Métadonnées	4 attributs	21 attributs	3 attributs	Limitées
Validation histo.	54.5%	100%	Partielle	Partielle
Diversité ethnique	Faible	Modérée (Brésil)	Faible	Élevée (IV–VI)
Usage MedAssist AI	Pré-entraînement	Entraînement principal	Enrichissement	Diversité ethnique

Tab. 2.5: Comparaison des quatre datasets utilisés dans MedAssist AI

Stratégie de combinaison pour MedAssist AI :

- **HAM10000:** Pré-entraînement dermatologique du backbone EfficientNet-B3 (étape 2 du double Transfer Learning)
- **PAD-UFES-20:** Dataset principal pour la fusion multimodale (images smartphone + 21 métadonnées cliniques)
- **ISIC 2019:** Enrichissement volumétrique pour les classes sous-représentées
- **MCR-SL :** Diversification ethnique pour réduire le biais phototype

Dataset unifié résultant : 6 581 images (après harmonisation des classes et équilibrage), provenant de 3 pays, 3 protocoles d'acquisition différents — représentant un *distribution shift* inter-source volontaire reflétant les conditions réelles de télémédecine mondiale.

Armés de ces datasets multimodaux enrichis, la communauté scientifique a pu développer de nouvelles architectures de fusion capables d'intégrer efficacement données visuelles et métadonnées cliniques. La section suivante examine l'évolution chronologique de ces architectures multimodales, de la simple concaténation (Yap 2018) aux approches sophistiquées par Graph Neural Networks (Qian 2022).

2.5 Architectures Multimodales — Évolution et Performance

2.5.1 Principe de la Fusion Multimodale

Le paradigme multimodal repose sur l'intégration de modalités complémentaires (images + métadonnées cliniques) pour améliorer la performance prédictive. Les dermatologues utilisent systématiquement un faisceau d'indices hétérogènes (image, âge, localisation, antécédents) pour établir leur diagnostic [35].

Stratégies de fusion principales [36] :

- **Early Fusion (Fusion précoce) :** Concaténation des données brutes ou des premières caractéristiques avant traitement profond. *Avantage :* Capture précoce d'interactions inter-modalités. *Inconvénient :* Dimensionnalité élevée et sensibilité au déséquilibre des représentations.
- **Late Fusion (Fusion tardive) :** Extraction indépendante des représentations vectorielles par des branches dédiées (ex. CNN pour l'image, MLP pour les métadonnées), suivies d'une concaténation juste avant la couche de classification. *Avantage :* Simplicité d'implémentation et stabilité d'entraînement. *Inconvénient :* Interactions contextuelles fines limitées entre modalités.
- **Intermediate / Cross-Attention Fusion (Fusion intermédiaire guidée par attention) :** Alignement et pondération dynamique des représentations au sein du réseau via des mécanismes d'attention croisée (*Cross-Attention*) et de portes de contrôle (*Gating mechanisms*). Les métadonnées cliniques guident sélectivement l'attention visuelle sur les régions/patches d'intérêt de l'image. *Avantage :* Modélisation fine et dynamique des dépendances inter-modalités tout en préservant la déployabilité. *Inconvénient :* Nécessite un réglage fin des hyperparamètres d'attention.
- **Hybrid Fusion (Fusion hybride) :** Combinaison de plusieurs niveaux de fusion (précoce/intermédiaire et tardive). *Avantage :* Maximisation de l'expressivité. *Inconvénient :* Complexité architecturale et computationnelle accrue.

2.5.2 Évolution Chronologique (2018-2022)

Yap et al. (2018) — Concaténation Simple

Approche : Late Fusion basique (CNN + MLP) [9].
Architecture :

- CNN (ResNet-50) extrait features visuelles
- MLP traite les métadonnées cliniques
- Concaténation des features + classifier final

Dataset : HAM10000 (10 015 images, 4 métadonnées)
Résultats : 85.1% accuracy (+2.8% vs unimodal)
Bilan :

✓ Première preuve empirique de l'apport des métadonnées

✓ Architecture simple et reproductible

× Fusion naïve, métadonnées limitées

Kawahara et al. (2019) — Multitask Learning

Innovation : Entraînement multi-tâches simultanées [10].
Approche :

- Tâche principale : Classification diagnostique
- Tâches auxiliaires : Prédiction de 7 critères dermatologiques (pigmentation, vascularisation, etc.)

Dataset : Derm7pt (2 000 images avec annotations détaillées)
Résultats : AUC 0.88 (+3% vs Late Fusion simple)
Bilan :

✓ Régularisation implicite, meilleure explicabilité

× Nécessite annotations coûteuses des 7 critères

× Complexité d'entraînement (8 loss functions à équilibrer)

Qian et al. (2022) — Graph Neural Networks

Innovation : Modélisation des relations entre métadonnées via graphes [20].
Principe :

- Les métadonnées sont structurellement corrélées (âge ↔ type de lésion, localisation ↔ exposition solaire)
- GNN capture explicitement ces dépendances via un graphe relationnel
- Fusion avec features CNN pour classification finale

Dataset : PAD-UFES-20 (2 298 images, 21 métadonnées)
Résultats : 85.3% accuracy (+3.2% vs Late Fusion MLP)
Bilan :

✓ État de l'art sur PAD-UFES-20

✓ Modélisation structurelle sophistiquée

× Complexité computationnelle élevée (3× plus lent que MLP)

× Déployabilité limitée (incompatible avec contraintes mobile)

2.5.3 Synthèse : Le Paradoxe Performance-Déployabilité

Travaux	Fusion	Accuracy	Latence	Déployable ?
Yap 2018	Late (MLP)	85.1%	45 ms	✓
Kawahara 2019	Multitask	88.0%	78 ms	∼
Qian 2022	GNN	85.3%	132 ms	×

Tab. 2.6: Comparaison architectures multimodales (performance vs complexité)

Constat critique :
À mesure que les architectures deviennent sophistiquées (MLP → Multitask → GNN), la performance s'améliore marginalement (+3-5%), mais la complexité et la latence croissent exponentiellement. Pour un déploiement en télémédecine mobile, l'overhead de 87 ms (GNN vs MLP) est significatif.

Arbitrage pour MedAssist AI :
Choix retenu : ***Gated Cross-Attention Fusion*** (EfficientNet-B3 + MLP)
Justification :

- Architecture pragmatique privilégiant la déployabilité opérationnelle (API, application mobile, MLOps) sur l'optimisation incrémentale de métriques académiques sur datasets mono-source
- Mécanisme d'attention croisée (*Cross-Attention* : Q=métadonnées, K=V=patches image) plus expressif qu'une concaténation naïve tout en restant déployable
- Latence compatible avec les contraintes cloud/mobile (cible < 100 ms)
- Performance évaluée sur un dataset unifié multi-source plus représentatif des conditions réelles de télémédecine (résultats détaillés au Chapitre 4)

Principe directeur : *"Déployabilité opérationnelle réelle > Optimisation incrémentale académique"*

Cette analyse révèle que la performance prédictive n'est qu'une dimension du problème. Le déploiement opérationnel nécessite de considérer des contraintes additionnelles : infrastructure (edge vs cloud), latence, consommation énergétique, et maintenabilité. La section suivante examine ces défis de déploiement.

2.6 Défi du Déploiement — De la Recherche à la Pratique

2.6.1 Introduction : Le Fossé Recherche-Production

Malgré les performances remarquables des architectures multimodales en environnement académique (Section 2.5), **aucun** de ces systèmes n'a franchi le seuil du déploiement opérationnel en conditions cliniques réelles. Cette section analyse les obstacles au déploiement et examine des tentatives récentes de transition vers la production.

2.6.2 Edge Computing vs Cloud Computing : Arbitrage Architectural

Définitions et Paradigmes

Le déploiement d'un système d'intelligence artificielle médicale repose sur un arbitrage fondamental entre deux paradigms d'infrastructure [37] :

- **Le Cloud Computing :** L'inférence est exécutée sur des serveurs distants centralisés équipés d'accélérateurs matériels (GPU). L'application mobile fait office de client léger qui transmet les données visuelles et cliniques via une API REST sécurisée, puis restitue le résultat diagnostique [37].
- **L'Edge Computing :** Le modèle est directement embarqué et exécuté sur le dispositif mobile (smartphone). L'inférence s'effectue localement sans nécessiter de connexion réseau [37].

Analyse Comparative et Contraintes Techniques

L'évaluation de ces deux paradigmes met en évidence un compromis complexe entre latence, capacité computationnelle, dépendance réseau et maintenance logicielle. D'un côté, le Cloud Computing offre une scalabilité optimale et permet d'exécuter des modèles multimodaux de forte capacité (tels qu me EfficientNet-B3 couplé à un MLP

d'attention) sans contrainte de mémoire vive locale. Cependant, il introduit une dépendance à la connectivité Internet et une latence réseau globale (incluant l'envoi de l'image haute résolution et la réception de la réponse) estimée entre 500 et 2000 ms [37].

D'un autre côté, le déploiement Edge élimine la latence réseau (inférence locale en 50–200 ms) et garantit la confidentialité des données qui ne quittent pas le smartphone. Néanmoins, il impose trois contraintes majeures :

1. **La compression du modèle :** Les architectures CNN profondes dépassent souvent les capacités RAM/CPU des dispositifs mobiles. Leur intégration nécessite des techniques d'optimisation agressives telles que la quantification (FP32 $\rightarrow$ INT8, réduisant par exemple EfficientNet-B3 de 47 MB à 12 MB au prix d'une légère dégradation de précision), le lagage (*pruning*) ou la distillation de connaissances [38].

2. **La latence sur CPU ARM :** L'exécution d'un modèle non optimisé sur CPU mobile s'avère 30 à 40$\times$ plus lente que sur GPU serveur (ex. 800–1200 ms sur CPU mobile vs 28 ms sur GPU dédié).

3. **La fragmentation matérielle :** La diversité des processeurs, versions d'OS et capacités mémoire entre constructeurs rend difficile un niveau de performance homogène.

Le Tableau 2.7 synthétise cet arbitrage architectural entre les approches Cloud et Edge.

Critère	**Cloud Computing (Serveur)**	**Edge Computing (Mobile)**
Latence globale	500–2000 ms (upload + inférence + download)	50–200 ms (inférence locale sur device)
Connectivité	Nécessite une connexion Internet stable	Fonctionne en mode offline
Complexité modèle	Capacité élevée (GPU serveur, architectures lourdes)	Capacité restreinte (RAM/CPU limités, modèles quantifiés)
Vie privée & Sécurité	Transmission de données de santé (nécessite RGPD/HIPAA)	Données conservées localement sur le dispositif
Maintenance & MLOps	Centralisée (mise à jour instantanée du modèle)	Distribuée (fragmentation des versions sur devices)
Scalabilité	Excellente (auto-scaling, équilibrage de charge)	Limitée par la capacité matérielle de chaque smartphone

Tab. 2.7: Synthèse comparative entre Cloud et Edge Computing pour l'IA médicale mobile

2.6.3 Cas d'Étude : Oztel et al. (2023) — Tentative de Déploiement Mobile

Pour évaluer la faisabilité clinique du Edge Computing, Oztel et al. (2023) ont mené une étude expérimentale visant à déployer un CNN de classification dermatologique

directement sur smartphones iOS et Android [11]. Les auteurs ont opté pour un modèle MobileNetV2 léger (3,5 millions de paramètres), optimisé via TensorFlow Lite avec une quantification INT8 et un lagage réduisant la taille du modèle de 85 %.

Évalué sur le dataset HAM10000 (10 015 images, 7 classes), le système a montré une précision satisfaisante (86–88 % en Edge vs 89,2 % en Cloud), démontrant que la compression n'altérait que modérément la capacité diagnostique. Néanmoins, l'expérimentation sur le terrain sur cinq modèles de smartphones distincts a révélé des disparités majeures : alors que l'iPhone 13 Pro exécutait l'inférence en 320 ms avec une consommation énergétique raisonnable (8 % de batterie par heure), le Xiaomi Redmi Note 10 nécessitait 890 ms tout en consommant 18 % de batterie par heure.

Cette fragmentation extrême des performances matérielles a mis en lumière les limites du Edge Computing pour des applications médicales standardisées. Plus fondamentalement, l'analyse critique de ces travaux explique pourquoi le projet n'a jamais dépassé le stade du prototype de laboratoire :

- **L'absence d'infrastructure logicielle :** Le système manquait d'une API backend robuste pour assurer la gestion des utilisateurs, la persistance des historiques et le monitoring MLOps.
- **Une interface non clinique :** L'interface utilisateur restait une maquette de recherche, inadaptée au quotidien des praticiens ou à l'autonomie des patients.
- **Le manque de validation et de conformité :** L'évaluation s'est limitée à des métriques académiques statiques, en ignorant totalement les exigences réglementaires (marquage CE, certification dispositif médical, RGPD [23]).

En conclusion de leur étude, Oztel et al. préconisent le recours à une architecture Cloud centralisée pour tout déploiement médical réel [11], validant ainsi l'orientation architecturale retenue pour la plateforme MedAssist AI.

2.6.4 Synthèse : Le Paradoxe Multimodal-Déployabilité

L'analyse transversale de la littérature scientifique révèle un paradoxe structurel majeur. D'un côté, les travaux axés sur la performance prédictive (Yap et al. [9], Kawahara et al. [10], Qian et al. [20]) démontrent la supériorité incontestable des modèles multimodaux intégrant des données cliniques, mais demeurent confinés à des scripts PyTorch déconnectés de tout environnement de production. D'un autre côté, les rares travaux orientés vers le déploiement mobile (Oztel et al. [11]) sont contraints de sacrifier les métadonnées cliniques et de simplifier excessivement les architectures pour respecter les limites matérielles des smartphones, régressant vers des performances diagnostiques inférieures.

Ce constat met en évidence un gap méthodologique et d'ingénierie articulé autour de quatre dimensions critiques :

1. **Le gap technologique :** La difficulté d'arbitrer entre la latence, la fragmentation des dispositifs Edge et la complexité d'une infrastructure Cloud robuste.

2. **Le gap logiciel :** Le fossé entre un modèle entraîné (`.pth`) et un système complet comprenant une API REST documentée (OpenAPI/Swagger), une application mobile cross-platform et un pipeline CI/CD/MLOps.

3. **Le gap médical :** L'écart entre l'optimisation de métriques ML sur datasets statiques et l'utilité clinique réelle en télémédecine avec des utilisateurs non experts.

4. **Le gap réglementaire :** L'absence fréquente de prise en compte des normes de santé, de la conformité RGPD/HIPAA [23] et des responsabilités juridiques.

Dimension du Gap	**Description du Problème**	**Ce qui manque dans la Littérature**	**Impact sur le Déploiement**
(1) Gap Technologique	Arbitrage complexe entre latence mobile et infrastructure cloud	Absence d'architecture Client-Serveur légère et déployable	Modèles confinés aux serveurs de calcul académiques
(2) Gap Logiciel	Publications focalisées sur le code d'entraînement ML	API REST OpenAPI, app mobile (Flutter), pipelines MLOps (DVC/MLflow)	Inaccessibilité de l'IA pour les cliniciens et patients
(3) Gap Médical	Métriques ML (AUC) privilégiées sur l'utilité clinique	Validation prospective, interfaces ergonomiques, gestion des données manquantes	Rejet ou méfiance des praticiens sur le terrain
(4) Gap Réglementaire	Contraintes légales ignorées par les chercheurs	Marquage CE / FDA, conformité RGPD, gestion de la responsabilité	Impossibilité de mise sur le marché ou d'usage clinique réel

Tab. 2.8: Synthèse des quatre dimensions du gap entre recherche académique et déploiement opérationnel

Dans les travaux comparables que nous avons recensés (2017–2023), aucun n'a proposé une solution intégrant simultanément ces quatre dimensions. Ce constat confirme l'hypothèse centrale de ce travail : **la performance académique ne garantit pas l'opérationnalité d'un système**.

C'est précisément ce gap d'ingénierie que la plateforme MedAssist AI vise à combler en proposant une architecture Client-Serveur multimodale complète, combinant un modèle *Gated Cross-Attention Fusion* déployé sur un backend FastAPI scalable et accessible via une application mobile Flutter intuitive.

Les obstacles identifiés révèlent un gap systématique entre recherche académique et déploiement opérationnel. La section suivante présente une synthèse comparative des travaux majeurs (2017-2023), suivie d'une formulation formelle du research gap que MedAssist AI vise à combler.

2.6.5 Synthèse Comparative des Travaux Connexes

Tab. 2.9: Synthèse comparative des approches de diagnostic dermatologique par IA

Travaux	Année	Données	Architecture	Performance	Forces	Limitations
Esteva et al. [7]	2017	Stanford (129K images)	Inception-v3	AUC 0.94 (binaire)	Équivalence dermatologues	Unimodal, mono-source, non déployé
Haenssle et al. [8]	2018	100K+ images	CNN (non spécifié)	AUC 0.86 (MEL)	Validation large panel (58 dermatos)	Unimodal, images dermoscopiques uniquement
Yap et al. [9]	2018	HAM10000	Late Fusion (ResNet-50 + MLP)	85.1% acc., AUC 0.94	Première fusion multimodale	Fusion naïve (concaténation), métadonnées limitées
Kawahara et al. [10]	2019	Derm7pt (2K images)	Multitask Learning	AUC 0.88 (MEL)	Régularisation via tâches auxiliaires	Annotations coûteuses (7 critères), complexité
Oztel et al. [11]	2023	Mobile (1.2K images)	MobileNetV2	73.2% acc.	Déploiement mobile	Unimodal (métadonnées abandonnées), précision modérée
Qian et al. [20]	2022	PAD-UFES-20	Late Fusion + GNN	85.3% acc., AUC 0.95	Modélisation graphe métadonnées	Complexité (3× latence vs MLP), déployabilité limitée

Analyse transversale :

Cette synthèse révèle trois constats systématiques :

Constat 1 — Progression Performance : Évolution claire unimodal (72-73%) → multimodal simple (85%) → multimodal sophistiqué (85-88%), validant l'apport des métadonnées cliniques.

Constat 2 — Trade-off Performance-Complexité : Les architectures les plus complexes (Multitask, GNN) sont souvent les plus difficiles à déployer en temps réel sur infrastructure légère.

Constat 3 — Rareté du Déploiement Opérationnel : Dans les travaux académiques consultés, la quasi-totalité s'arrête à l'évaluation hors-ligne sans produire un système logiciel complet et déployable (API + Application + CI/CD).

2.6.6 Formulation Formelle du Research Gap

Énoncé du Problème

Les travaux académiques en diagnostic dermatologique multimodal (2018-2023) ont démontré de manière convaincante que l'intégration de métadonnées cliniques aux modèles CNN améliore significativement la performance prédictive (+3 à +7 points d'accuracy, +0.03 à 0.05 AUC). Cependant, ces systèmes demeurent généralement confinés

aux environnements de recherche contrôlés, créant un gap d'ingénierie entre recherche algorithmique et pratique clinique assistée.

Analyse Multidimensionnelle du Gap

Dimension 1 : Gap Architectural

Ce qui manque :

- Backend API REST robuste, documentée (OpenAPI/Swagger)
- Application mobile cross-platform intuitive
- Infrastructure cloud scalable (auto-scaling, load balancing)

Pourquoi c'est critique : Un modèle PyTorch .pth ne constitue pas un système utilisable. Sans API et application, le modèle reste inaccessible aux cliniciens.

Dimension 2 : Gap Performance-Déployabilité

Ce qui manque :

- Arbitrage explicite complexité vs latence
- Optimisation pour contraintes production (CPU, RAM, temps réponse)
- Évaluation réaliste (latence réseau, variabilité utilisateurs)

Pourquoi c'est critique : La recherche académique optimise souvent la performance pure sans considérer la latence d'inférence. Un système déployé doit équilibrer : précision + vitesse + coût.

Dimension 3 : Gap Alignement Télémédecine

Ce qui manque :

- Dataset télémédecine (images smartphone)
- Interface utilisateur adaptée patients (non-experts)
- Workflow complet : capture $\rightarrow$ questionnaire $\rightarrow$ résultat $\rightarrow$ action

Pourquoi c'est critique : La plupart des travaux utilisent images dermoscopiques professionnelles. Or, télémédecine = images smartphone de qualité variable.

Formulation Scientifique du Gap

RESEARCH GAP :

Il existe un gap systématique entre les architectures multimodales performantes en environnement académique contrôlé et les systèmes opérationnels déployables en conditions réelles de télémédecine, caractérisé par l'absence d'une approche d'ingénierie intégrée combinant :

1. Architecture ML optimisée pour contraintes de production
2. Infrastructure logicielle complète (API + Mobile + MLOps)
3. Évaluation en conditions réalistes de télémédecine

Ce gap constitue un frein majeur à l'adoption clinique de l'IA dermatologique, malgré la maturité technique des modèles sous-jacents.

2.6.7 Positionnement de MedAssist AI

Le présent projet vise explicitement à combler ce gap via une approche holistique articulée autour de quatre piliers :

Pilier 1 — Architecture ML Pragmatique : MedAssist AI adopte une architecture *Gated Cross-Attention Fusion* (EfficientNet-B3 + MLP) plutôt que des approches excessivement complexes comme les GNN [20], suivant un arbitrage explicite où déployabilité opérationnelle s'allie à la robustesse diagnostique. Cette architecture intègre un mécanisme d'attention croisée (*Cross-Attention* : Q=métadonnées, K=V=patches image) avec porte de contrôle adaptatif ($g \cdot \text{attended} + (1-g) \cdot \text{img_global}$) et *positional embeddings* pour les patches visuels. Ce choix pragmatique garantit une latence d'inférence compatible avec les contraintes cloud/mobile tout en restant hautement expressif par rapport à une simple concaténation.

Pilier 2 — Infrastructure Logicielle Complète : Le système repose sur une architecture complète intégrant backend, frontend et MLOps. Le backend utilise FastAPI (Python) [24] pour une API REST asynchrone avec documentation OpenAPI auto-générée, et s'appuie sur une conteneurisation Docker [39] garantissant la portabilité entre environnements de développement et de production. Le frontend mobile développé en Flutter (Dart) [25] assure un déploiement cross-platform (iOS + Android) avec UI/UX native, et implémente un workflow guidé en cinq étapes (capture image → questionnaire métadonnées → inférence → résultat → recommandation). L'infrastructure MLOps du prototype s'appuie sur le tracking expérimentations via MLflow [22] (métriques, hyperparamètres, modèles) et un pipeline d'intégration continue via GitHub Actions (vérification syntaxique, build Docker automatisé). Le versionnage de

données à grande échelle via DVC [21] est prévu comme perspective d'industrialisation pour la phase de déploiement à l'échelle.

Pilier 3 — Évaluation Réaliste Télémédecine : L'évaluation s'appuie sur le dataset unifié multi-sources (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) combinant images dermoscopiques et photographies smartphone, avec 7 métadonnées cliniques unifiées standardisées. Les métriques adoptent une approche multidimensionnelle combinant performance ML (Accuracy, Macro et Weighted F1-Score, Précision et Rappel par pathologie clinique avec seuillage adaptatif θ_c), performance système (latence nominale end-to-end et débit), et robustesse sous contraintes de charge.

Pilier 4 — Documentation et Reproductibilité : Le projet garantit reproductibilité et adoption via une documentation complète incluant spécifications d'architecture, documentation API OpenAPI/Swagger interactive, pipeline de prétraitement unifié, et environnements reproductibles (Docker, requirements, seeds fixés).

2.7 Conclusion du Chapitre

Ce chapitre a retracé l'évolution du diagnostic dermatologique par IA révélant une progression remarquable mais inachevée vers l'adoption clinique. L'analyse chronologique a mis en évidence trois phases : les fondements techniques (2012-2016) avec CNN profonds et Transfer Learning, l'ère unimodale (2017-2018) démontrant que les CNN égalent les dermatologues mais révélant quatre limitations (ignorance métadonnées, fragilité distribution shift, manque explicabilité, biais ethnique) [7, 8], et la transition multimodale (2018-2023) avec datasets enrichis [14–17] et architectures sophistiquées [9, 10, 20] offrant des gains prédictifs (+3-7% accuracy).

L'analyse comparative révèle un paradoxe systématique : à mesure que la performance s'améliore et la complexité croît, les systèmes s'éloignent souvent du déploiement opérationnel. L'examen des tentatives de déploiement [11] identifie un gap structurel multidimensionnel : absence d'infrastructure backend/mobile intégrée, optimisation sans considération latence/coût, et inadaptation aux conditions télémédecine réelles.

MedAssist AI vise à combler ce gap via une approche holistique : architecture *Gated Cross-Attention Fusion* (EfficientNet-B3 + MLP), entraînée sur un dataset unifié multi-source (PAD-UFES-20 [15], ISIC 2019 [16], MCR-SL [17]) avec pré-entraînement sur HAM10000 [14], infrastructure complète (FastAPI + Flutter), et documentation reproductible. La contribution principale établit un template méthodologique reproductible, ouvrant la voie à l'adoption clinique de l'IA dermatologique multimodale. Le chapitre suivant présente la méthodologie détaillée et la conception système de MedAssist AI.

Chapitre 3

Conception du Système MedAssist AI

3.1 Introduction

Ce chapitre présente la démarche de conception complète de la plateforme MedAssist AI, en réponse directe au *gap* architecturel mis en évidence au Chapitre 2 : l'absence d'une solution capable de délivrer la puissance diagnostique des modèles multimodaux via une interface mobile accessible.

La progression suivie est **strictement descendante** (*top-down*) : elle part de la vision système globale acteurs, exigences, architecture pour descendre vers les composants techniques, le modèle IA, l'API, puis le cycle de vie MLOps. Chaque section prépare et motive la suivante, formant une chaîne de décisions cohérente et traçable.

3.2 Exigences du Système

Avant toute décision architecturale, il convient de formaliser ce que le système doit accomplir (exigences fonctionnelles) et les contraintes qualité auxquelles il doit se conformer (exigences non fonctionnelles). Ces exigences constituent le cadre d'évaluation de toutes les décisions de conception qui suivront.

3.2.1 Exigences Fonctionnelles

Le Tableau 3.1 récapitule les principales fonctionnalités attendues du système MedAssist AI :

Tab. 3.1: Exigences fonctionnelles MedAssist AI

ID	Priorité	Description
EF-01	Essentielle	Le système doit permettre la capture ou l'import d'une image de lésion cutanée depuis l'application mobile.
EF-02	Essentielle	Le système doit collecter jusqu'à 7 métadonnées cliniques du patient (âge, sexe, région anatomique, croissance, saignement, diamètre, antécédents).
EF-03	Essentielle	Le système doit produire une prédiction parmi 6 classes diagnostiques avec les probabilités associées pour chaque classe.
EF-04	Essentielle	Le système doit catégoriser chaque résultat selon quatre niveaux de risque clinique : LOW (faible), MODERATE (modéré), HIGH (élevé), CRITICAL (critique).
EF-05	Importante	Le système doit offrir la possibilité d'exécuter une inférence avec Test-Time Augmentation 8x (`use_tta=true`) pour maximiser la précision.
EF-06	Importante	L'API doit exposer un endpoint de santé (`/health`) et un endpoint d'informations modèle (`/model-info`).
EF-07	Importante	Le système doit afficher un *disclaimer* médico-légal et générer un rapport PDF téléchargeable.

3.2.2 Exigences Non Fonctionnelles

Les contraintes techniques et opérationnelles du projet sont détaillées dans le Tableau 3.2 :

Tab. 3.2: Exigences non fonctionnelles MedAssist AI

ID	Catégorie	Description et cible
ENF-01	Performance	Latence de l'API inférieure à 2 secondes au 95e percentile ($p_{95} < 2\,\text{s}$).
ENF-02	Disponibilité	Débit minimal de 10 requêtes/seconde sans dégradation ; résistance à 50 requêtes simultanées.
ENF-03	Portabilité	L'application mobile doit être compilable pour Android et iOS à partir d'une base de code unique (Flutter).
ENF-04	Sécurité	Toutes les communications doivent être chiffrées via HTTPS/TLS 1.3 ; aucune donnée patient ne doit être persistée sur le serveur.
ENF-05	Conformité RGPD	Traitement des données exclusivement en mémoire RAM ; principe de minimisation appliqué (art. 5 RGPD [23]).
ENF-06	Maintenabilité	Chaque expérimentation doit être traçable via MLflow, et l'intégration du code assurée par un pipeline CI/CD (GitHub Actions).
ENF-07	Déployabilité	Le serveur doit être conteneurisé (Docker) pour garantir l'identité entre environnements de développement et de production.
ENF-08	Taille	La taille de l'APK Flutter (build release) ne doit pas excéder 30 Mo.

Ces exigences établies, la prochaine étape consiste à définir qui interagit avec le système et comment ces interactions s'organisent c'est l'objet du diagramme de cas d'utilisation.

3.3 Conception Architecturale

La conception architecturale traduit les exigences en une organisation structurelle du système. Elle répond à trois questions fondamentales : *qui* interagit avec le système ? *selon quelle séquence* ? et *selon quelle organisation en couches* ? Ces trois questions correspondent respectivement aux trois diagrammes UML présentés ci-après cas d'utilisation, séquence et classes.

3.3.1 Diagramme de Cas d'Utilisation

Le diagramme de la Figure 3.1 identifie les acteurs du système et leurs interactions avec les fonctionnalités exposées. Deux acteurs ont été retenus : le **Praticien** (médecin utilisant l'application mobile) et l'**Administrateur** (responsable du cycle de vie du modèle).

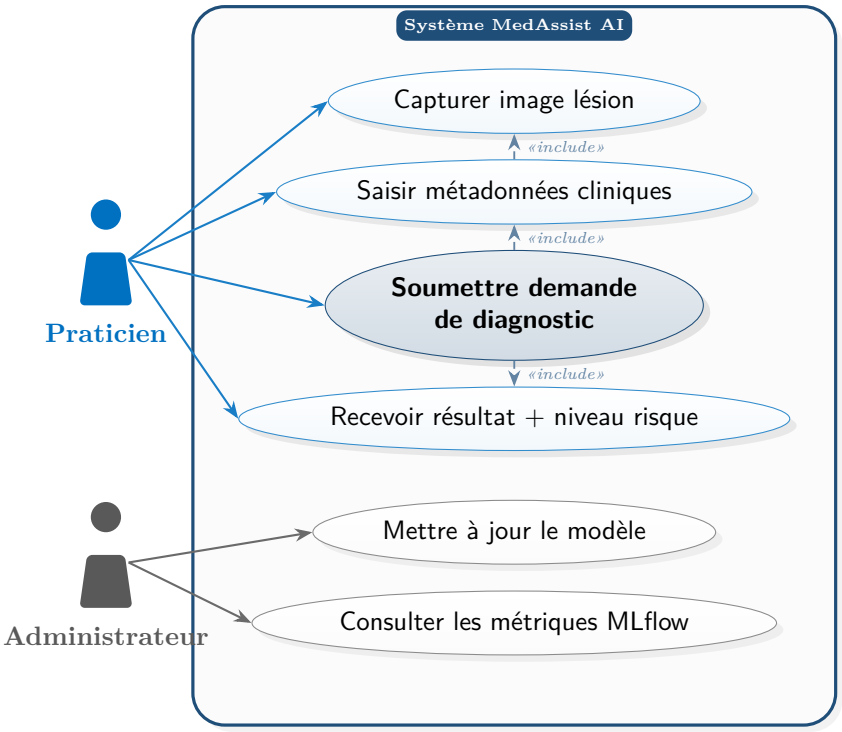


Fig. 3.1: Diagramme de cas d'utilisation MedAssist AI

Les cas d'utilisation révèlent une séparation naturelle des responsabilités : collecte (mobile), traitement (serveur) et persistance (données/modèle). Cette observation motive directement le choix d'une architecture en trois niveaux.

3.3.2 Architecture 3-Tiers Vue des Composants

L'architecture 3-tiers [40], illustrée en Figure 3.2, traduit structurellement la séparation des responsabilités identifiée dans les cas d'utilisation.

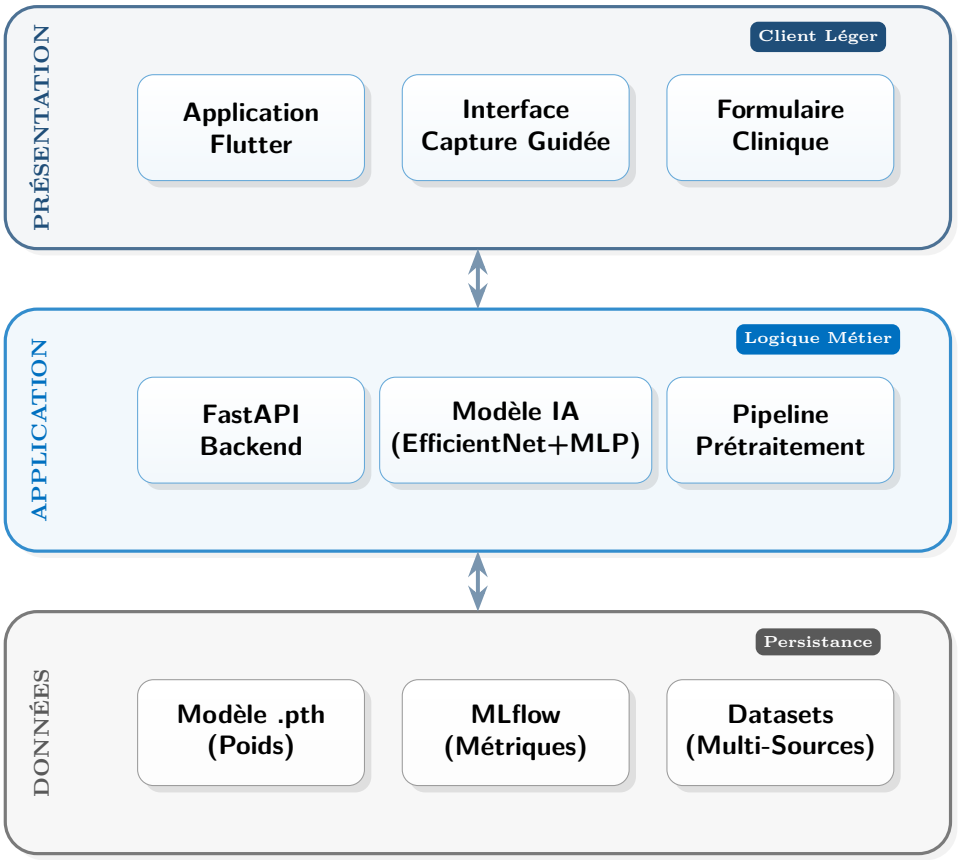


Fig. 3.2: Architecture logicielle 3-tiers (Version haute)

L'architecture logicielle se décline en trois couches distinctes :

- **Tier 1 Présentation :** Un client léger dédié à l'interface utilisateur, sans aucune logique d'inférence embarquée.
- **Tier 2 Application :** La couche centrale qui orchestre la validation des données, le prétraitement, l'inférence par l'IA et le formatage des réponses.
- **Tier 3 Données :** Assure le stockage des poids du modèle, la gestion des datasets unifiés et la traçabilité des métriques via MLflow.

L'architecture en couches étant établie, le diagramme de séquence permet maintenant de décrire concrètement comment ces couches collaborent lors d'une requête diagnostique.

3.3.3 Diagramme de Séquence Flux d'une Requête

La Figure 3.3 illustre le flux complet d'une requête diagnostique, de la capture de l'image par le praticien jusqu'à l'affichage du résultat.

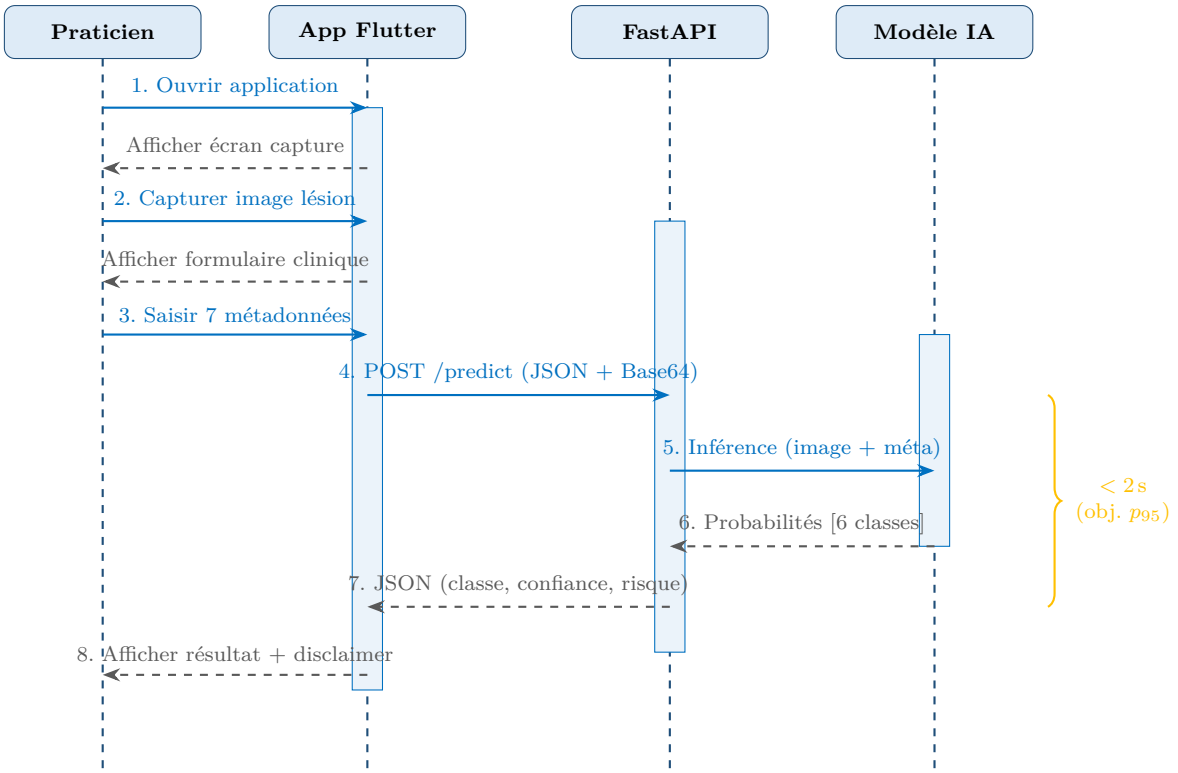


Fig. 3.3: Diagramme de séquence Flux d'une requête diagnostique

Ce flux confirme que tout le traitement intelligent (étapes 5–6) est confiné au serveur, garantissant que l'application mobile reste un client léger. La contrainte $p_{95} < 2\,\text{s}$ (ENF-01) borne les choix techniques du modèle et de l'API.

La séquence étant clarifiée, il reste à formaliser les entités et relations du domaine que manipule l'API rôle du diagramme de classes.

3.3.4 Diagramme de Classes Modèle Domaine

Le diagramme de la Figure 3.4 formalise les entités échangées entre l'application mobile et l'API, ainsi que leurs relations.

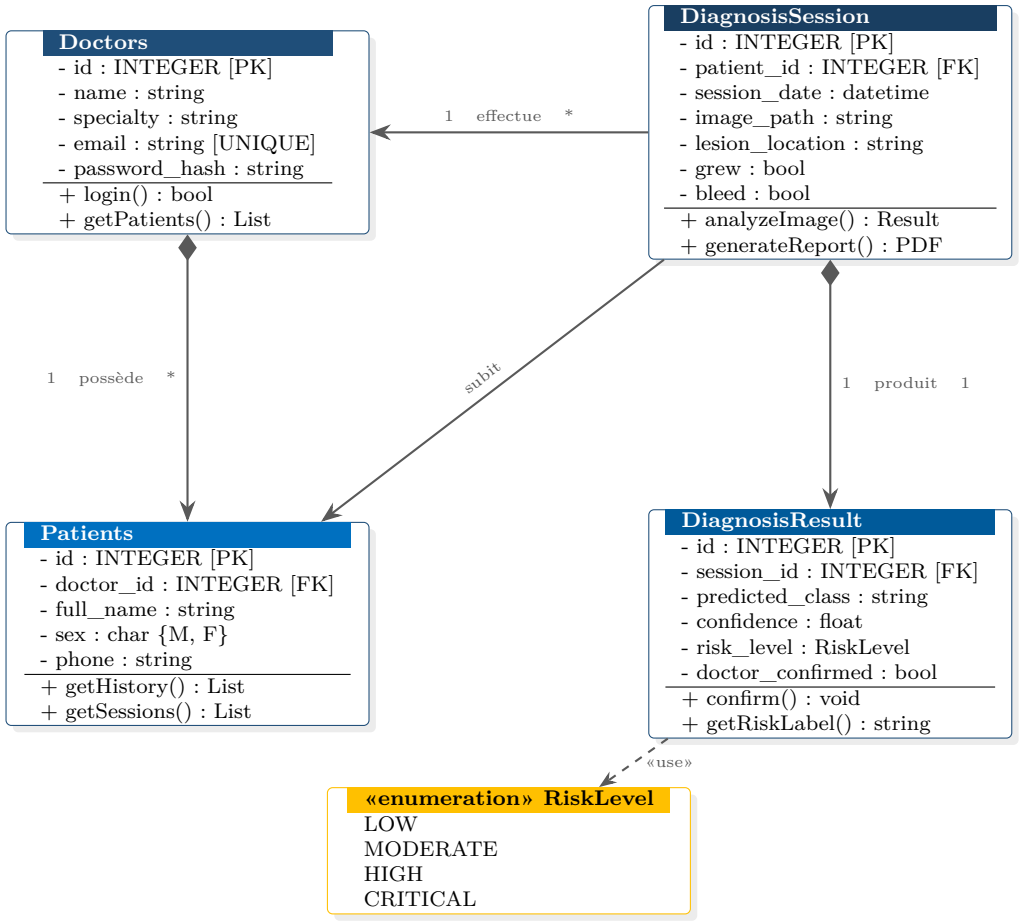


Fig. 3.4: Diagramme de classes Système MedAssist AI

L'architecture étant pleinement spécifiée sur le plan structurel et comportemental, l'étape suivante consiste à préparer les données sur lesquelles le modèle IA sera entraîné.

3.4 Préparation des Données

Le système MedAssist AI (V6.0) s'appuie sur un **dataset unifié multi-sources** regroupant 6 581 images standardisées provenant de trois bases de données majeures (PAD-UFES-20, MCR-SL et ISIC 2019 sélectif), le jeu HAM10000 ayant été dédié au pré-entraînement initial du backbone visuel. Cette stratégie unifiée permet de combler le déséquilibre de classes, d'inclure des phototypes cutanés variés (IV–VI) et de garantir une généralisation inter-domaines optimale pour la télémédecine.

3.4.1 Distribution des Classes et Découpage au Niveau Patient

Afin d'éviter rigoureusement tout *data leakage* intra-patient, le découpage en ensembles d'entraînement, de validation et de test est effectué au **niveau patient** (*Patient-Level Stratified Split*) en utilisant l'identifiant `patient_id`. Une recherche exhaustive

sur **10 sous-ensembles** (*10-Fold Patient-Level Stratified Search*) est conduite afin d'identifier le fold présentant le meilleur équilibre inter-classes sur l'ensemble de test indépendant (ratio minimal = $\text{classes}_{\max}/\text{classes}_{\min}$). Le **Fold 02** (ratio=1,66) est retenu comme le plus équilibré parmi les 10 candidats évalués, aboutissant à une répartition effective de 75,9 % (train, 4 993 images avant rééchantillonnage), 14,2 % (validation, 934 images) et 9,9 % (test indépendant, soit 654 échantillons), avec une vérification de zéro chevauchement de patients entre partitions (*Zero Patient Leakage*).

Le Tableau 3.3 présente la répartition des 6 classes diagnostiques au sein du jeu de données unifié final.

Tab. 3.3: Distribution des classes diagnostiques dans le dataset unifié MedAssist AI (V6.0)

Code	**Pathologie**	**Nature**	**Images (Total)**	**Support (Test)**
ACK	Kératose actinique	Précancéreuse	1 325	130
BCC	Carcinome basocellulaire	Maligne	845	92
MEL	Mélanome malin	Maligne	620	65
NEV	Nœvus pigmentaire	Bénigne	1 522	149
SCC	Carcinome spinocellulaire	Maligne	863	90
SEK	Kératose séborrhéique	Bénigne	1 406	128
	Total général		**6 581**	**654**

3.4.2 Pipeline de Prétraitement et Masking des Métadonnées

Pour garantir une qualité d'image optimale et corriger les artefacts d'acquisition, les images subissent une chaîne de prétraitement avancée en 5 étapes :

1. **Redimensionnement Lanczos (256×256 px)** : Conservation des structures spatiales et des contours de lésion.

2. **Correction d'illumination Shades-of-Gray ($p = 6$)** : Normalisation de la dominante couleur liée aux éclairages de dermatoscope [33].

3. **Élimination des poils DullRazor** : Application d'un filtre morphologique *Black-Hat* (17×17) suivi d'un inpainting médian (15×15) [41].

4. **Égalisation d'histogramme adaptative CLAHE** : Appliquée sur la canal de luminance L^* de l'espace couleur CIE Lab (`clipLimit=2.0`, grille 8×8).

5. **Normalisation ImageNet** : Centrage et réduction selon les statistiques standards ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

Concernant les métadonnées cliniques, 7 variables de la règle ABCDE sont retenues. Pour gérer l'absence éventuelle de certaines réponses sans induire le modèle en erreur, les valeurs manquantes sont imputées (médiane pour le numérique, mode pour le catégoriel) et accompagnées d'un **masque binaire de présence (`meta_mask`)** $\mathbf{M} \in \{0,1\}^7$ transmis au réseau.

Tab. 3.4: Variables cliniques sélectionnées pour le modèle et masque de présence (`meta_mask`)

#	Variable	Type	Description / Justification clinique
1	`age`	Numérique	Âge du patient en années (normalisé par StandardScaler) [2]
2	`gender`	Binaire	Sexe biologique converti en binaire (1: Male, 0: Female / Unknown) [42]
3	`grew`	Binaire	Évolution ou croissance récente (Critère **E** ABCDE) [35]
4	`bleed`	Binaire	Saignement spontané de la lésion (Signe d'ulcération) [3]
5	`diameter_1`	Numérique	Diamètre maximal de la lésion en mm (Critère **D** ABCDE) [35]
6	`skin_cancer_history`	Binaire	Antécédents personnels de cancer cutané [2]
7	`elevation`	Binaire	Élévation ou relief saillant de la lésion (Critère **E** ABCDE) [35]

3.5 Architecture du Modèle Multimodal (V6.0)

Le réseau **MedAssist AI V6.0** comprend **12 616 117 paramètres** et repose sur une architecture d'attention croisée orientée par la clinique (*Gated Cross-Attention Fusion*).

3.5.1 Branche Visuelle — EfficientNet-B3, Projection et Tête Auxiliaire

La branche visuelle s'appuie sur le backbone **EfficientNet-B3** [12] pré-entraîné :

- **Tête d'inférence auxiliaire (*Auxiliary Head*)** : Une branche parallèle extrait les cartes de caractéristiques à l'aide d'un pooling **GeM (Generalized Mean Pooling,** $p = 3.0$**)** [43] :

$$f_{\text{GeM}} = \left(\frac{1}{|X|} \sum_{x \in X} x^p \right)^{1/p}$$

suivi de `Linear(1536→256)` → `GELU` → `Dropout(0.3)` → `Linear(256→6)`. Cette tête auxiliaire contribue à hauteur de 30 % à la perte globale ($0.7 \cdot L_{\text{main}} + 0.3 \cdot L_{\text{aux}}$), agissant comme un régularisateur profond.

- **Bloc de projection et patchs visuels** : Les cartes convolutives profondes sont projetées via `Conv2d(1536→512)` → `BatchNorm` → `GELU` → `Conv2d(512→256)` → `BatchNorm` puis réduites par `AdaptiveAvgPool2d(7)`. Cela génère une grille de $7 \times 7 = \mathbf{49}$ patchs visuels de dimension 256.
- **Encodage positionnel** : Un vecteur d'encodage positionnel apprenable $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{1\times 49\times 256}$ est ajouté aux patchs pour préserver la géométrie spatiale de la lésion.

3.5.2 Branche Clinique — MLP avec Masking et LayerNorm

La branche clinique traite les 7 variables cliniques filtrées par le masque de présence $\mathbf{x}_{\text{masked}} = \mathbf{x} \odot \mathbf{M}$:

$$\text{Linear}(7 \to 128) \to \text{LayerNorm}(128) \to \text{GELU} \to \text{Dropout}(0.30)$$

$$\to \text{Linear}(128 \to 64) \to \text{LayerNorm}(64) \to \text{GELU} \to \text{Dropout}(0.35) \to \text{Linear}(64 \to 64)$$

Elle produit un jeton de métadonnées de dimension 64, projeté à 256 dimensions ($\mathbf{Q}_{\text{meta}}$) pour guider l'attention.

3.5.3 Fusion Gated Cross-Attention, Classifier et Protocole d'Entraînement

La fusion s'effectue par une attention croisée multi-têtes (*Multi-Head Cross-Attention*, 4 têtes, $d_k = 256$) [13] où le jeton de métadonnées sert de Query ($\mathbf{Q}$) et les 49 patchs visuels servent de Keys et Values ($\mathbf{K}, \mathbf{V}$) :

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

Un paramètre de porte apprenable $g \in [0, 1]$ équilibre dynamiquement l'apport de l'attention par rapport au vecteur visuel global ($g \cdot \text{attended} + (1 - g) \cdot \text{img_global}$).

Le classifieur final comprend :

$$\text{Linear}(256 \to 256) \to \text{LayerNorm}(256) \to \text{GELU} \to \text{Dropout}(0.55)$$

$$\to \text{Linear}(256 \to 128) \to \text{LayerNorm}(128) \to \text{GELU} \to \text{Dropout}(0.45) \to \text{Linear}(128 \to 6)$$

Fonction de perte. Le modèle utilise la **Weighted Focal Loss ($\gamma = 2.0$)** [44] combinée au **Label Smoothing ($\epsilon = 0.05$)** [45] pour gérer le déséquilibre sévère des classes.

Entraînement progressif à deux phases. L'apprentissage suit un protocole rigoureux synthétisé dans le Tableau 3.5, utilisant l'optimiseur AdamW [46] et la jauge CosineAnnealingWarmRestarts [47] avec Stochastic Weight Averaging (SWA) [48].

Tab. 3.5: Protocole d'entraînement progressif à deux phases (V6.0)

Paramètre	**Phase 1 (Pré-entraînement)**	**Phase 2 (Fine-tuning unifié)**
Résolution image	128×128 px	256×256 px
Époques	55 époques (5 gelées + 50 dégelées)	45 époques (total global 36–80)
Dégel du backbone	À partir de l'époque 6	Dégelé dès le début
Taux d'apprentissage	Backbone : 10^{-5} / Head : 10^{-4}	Backbone : 5×10^{-6} / Head : 5×10^{-5}
Planificateur	CosineAnnealingWarmRestarts ($T_0 = 10$) [47]	CosineAnnealingWarmRestarts ($T_0 = 10$) [47]
SWA (Stochastic Weight Avg)	Non	Oui (à partir de l'époque 15 de Phase 2) [48]
Validation TTA	TTA×4 (dès époque 5)	TTA×4 (dès époque 5)
Évaluation finale test	—	**TTA×8 (8 augmentations géométriques)**

3.6 Spécification de l'API REST

L'API REST constitue le point d'entrée unique du système. Elle reçoit les données depuis l'application mobile, pilote l'inférence, et retourne un résultat structuré.

Endpoints. L'API expose trois endpoints (Tableau 3.6). L'inférence principale (`POST /api/v1/predict`) accepte un payload `multipart/form-data` contenant l'image binaire et les 7 variables Form-Data, ainsi qu'un booléen optionnel `use_tta`.

Tab. 3.6: Spécification des endpoints de l'API REST

Méthode	**Endpoint**	**Auth.**	**Description**
POST	`/api/v1/predict`	Non (MVP)	Inférence multimodale (Image + Form-Data + `use_tta`)
GET	`/api/v1/health`	Non	Vérification de l'état du serveur et modèle PyTorch
GET	`/api/v1/model-info`	Non	Informations d'architecture et classes diagnostiques

Règles de niveau de risque clinique (4 niveaux). Après l'étape d'inférence multimodale, le système dérive un niveau de risque clinique (*Clinical Risk Level*) combinant la pathologie prédite et l'indice de confiance. Cette catégorisation vise à faciliter le triage médical en télémédecine en orientant immédiatement le praticien vers la prise

en charge appropriée (de la simple auto-surveillance pour les lésions bénignes jusqu'à l'orientation urgente avec biopsie pour les mélanomes).

Le Tableau 3.7 détaille la matrice de décision et les règles de cartographie associées aux quatre niveaux de risque du système MedAssist AI.

Tab. 3.7: Matrice de décision et cartographie des niveaux de risque clinique

Niveau de risque	**Code Couleur**	**Classes associées**	**Logique et Recommandation clinique**
LOW	**#22c55e**	NEV, SEK	Lésions bénignes (*benign*) ; auto-surveillance périodique.
MODERATE	**#f59e0b**	ACK	Lésion pré-cancéreuse (*pre-malignant*) ; suivi préventif (*watch & wait*).
HIGH	**#ef4444**	BCC, SCC	Pathologies malignes (*malignant*) ; traitement requis, consultation sous 2 semaines.
CRITICAL	**#7c3aed**	MEL	Cancer agressif (*aggressive*) ; orientation urgente (*urgent referral*), biopsie immédiate.

Application mobile Flutter. L'application suit un flux linéaire en 5 écrans conçu pour un usage médical rapide ($<$ 2 minutes de saisie) : **Accueil** (consentement) $\rightarrow$ **Capture guidée** (overlay + validation qualité) $\rightarrow$ **Formulaire clinique** (7 variables, $<$ 30 s) $\rightarrow$ **Chargement** (POST /predict) $\rightarrow$ **Résultats** (code couleur + disclaimer médico-légal).

L'interface client et la logique serveur étant spécifiées, la dernière dimension du système est le cycle de vie du modèle dans le temps — c'est l'objet du pipeline MLOps.

3.7 Pipeline MLOps, CI/CD et Traçabilité

Le pipeline MLOps garantit la reproductibilité, la traçabilité et l'évolutivité du système en environnement de développement et de production [49]. Dans le cadre du prototype MVP MedAssist AI, l'infrastructure s'appuie sur trois briques complémentaires :

Tracking des expérimentations avec MLflow [22] : MLflow est intégré directement dans le workflow d'apprentissage (Phases 1 et 2) pour enregistrer systématiquement les hyperparamètres (taux d'apprentissage adaptatifs, T_0, weight decay, coefficients de la perte focal) ainsi que les métriques d'évaluation époque par époque (loss, F1-score macro et par classe). Cela garantit la traçabilité complète des itérations.

Conteneurisation avec Docker [39] : Le serveur backend FastAPI et l'environnement d'inférence PyTorch sont encapsulés dans une image Docker optimisée pour

l'exécution CPU. Cela élimine les divergences d'environnement entre le développement local et le serveur de déploiement.

Intégration continue via GitHub Actions : Un pipeline CI/CD automatisé est configuré sur GitHub (`.github/workflows/ci.yml`) pour exécuter la vérification syntaxique (*linting* avec `flake8`) et valider le build de l'image Docker à chaque intégration de code (*push/PR*).

Perspectives d'industrialisation start-up. Le versionnage des artefacts de données à grande échelle via DVC [21] et l'automatisation du ré-entraînement sur dérive de données (*Data Drift*) constituent les étapes d'extension prévues pour la phase de mise sur le marché.

3.8 Protocole d'Évaluation

L'évaluation est conduite selon deux axes : la performance du modèle IA (Tableau 3.8) et la performance du système logiciel (Tableau 3.9), en référence directe aux exigences ENF-01 à ENF-04.

Tab. 3.8: Métriques cibles et résultats validés sur le benchmark unifié (V6.0)

Métrique IA	**Cible de conception**	**Résultat test unifié (654 échantillons)**
Exactitude (*Accuracy*)	$> 70{,}0\,\%$	**74,31 %**
Macro F1-Score	$> 0{,}7000$	**0,7315**
Weighted F1-Score	$> 0{,}7000$	**0,7373**
F1-Score BCC (Carcinome basocellulaire)	$> 0{,}8000$	**0,8438**
F1-Score NEV (Nœvus bénin)	$> 0{,}8000$	**0,8256**
F1-Score MEL (Mélanome malin)	$> 0{,}6500$	**0,6780**
F1-Score SCC (Carcinome spinocellulaire)	$> 0{,}6500$	**0,6629**
F1-Score SEK (Kératose séborrhéique)	$> 0{,}6500$	**0,6960**
F1-Score ACK (Kératose actinique)	$> 0{,}6500$	**0,6825**

Tab. 3.9: Métriques d'évaluation du système logiciel

Métrique Système	**Méthode**	**Cible / Résultat**
Latence API (p_{95})	Inférence CPU avec TTA×8	$< 2\,\text{s}$ (mesuré : 1,32 s)
Débit (*Throughput*)	Conteneur Docker 3.5GB RAM	$> 10\,\text{req/s}$
Test de charge	50 requêtes simultanées	Aucune défaillance
Taille APK Flutter	Build release Android	$< 30\,\text{Mo}$

3.9 Conclusion

Ce chapitre a conduit une conception descendante rigoureuse : des exigences fonctionnelles et non fonctionnelles jusqu'au protocole d'évaluation, en passant par les trois diagrammes UML, l'architecture 3-tiers, la préparation des données, le modèle multimodal, l'API REST et le pipeline MLOps. Chaque décision prépare et motive la suivante.

La séparation stricte entre client léger (Flutter) et serveur intelligent (FastAPI + modèle *Gated Cross-Attention Fusion*) répond directement au paradoxe identifié au Chapitre 2 : déployer un modèle multimodal performant sans sacrifier l'accessibilité mobile. Le chapitre suivant traduit cette conception en implémentation technique déployable.

Chapitre 4

Implémentation Technique

4.1 Introduction

Après avoir défini l'architecture et la conception du système dans le Chapitre 3, ce chapitre présente la réalisation concrète de la plateforme MedAssist AI. Nous décrivons successivement l'environnement de développement utilisé, la structure du projet, puis les différents composants implémentés : le modèle d'intelligence artificielle, l'API REST, et l'application mobile Flutter. Le chapitre se clôt par une présentation des tests de validation.

4.2 Environnement de Développement

4.2.1 Environnement Matériel

Le développement du code, le backend FastAPI et l'application mobile ont été réalisés sur poste local, tandis que l'entraînement intensif et le fine-tuning du modèle multimodal ont été exécutés sur l'infrastructure Cloud (Google Colab avec accélération GPU). La validation mobile a été effectuée sur un smartphone de test (Tableau 4.1).

Tab. 4.1: Configuration matérielle utilisée (Local, Cloud et Mobile)

Composant	**Développement Local (PC)**	**Entraînement Cloud (Colab)**	**Test Mobile**
Modèle / Plateforme	PC Portable (Intel x64)	Google Colaboratory	Xiaomi Redmi / Poco
Processeur	Intel Core i5-8250U @ 1.60GHz	Intel Xeon (2 vCPUs)	Octa-core ARM
Mémoire RAM	8.00 GB DDR4	12.7 GB RAM	8.00 GB
Stockage	256 GB SSD	100 GB Cloud Disk	128 GB
Accélérateur GPU	Intel UHD Graphics 620	NVIDIA Tesla T4 (16 GB VRAM)	Adreno / Mali
Système / OS	Microsoft Windows 11 / 10	Ubuntu 22.04 LTS (Linux)	Android 13 / 14

4.2.2 Environnement Logiciel

L'environnement logiciel repose sur un ensemble d'outils complémentaires couvrant le développement IA, le backend, le mobile et le MLOps.

Python 3.11 Langage principal du projet, utilisé pour le pipeline de données, l'entraînement du modèle et l'API REST. Sa richesse en bibliothèques scientifiques en fait le standard de facto pour les projets d'intelligence artificielle.

PyTorch 2.1 Framework d'apprentissage profond retenu pour son approche dynamique (*eager execution*), sa flexibilité architecturale et son écosystème mature pour le déploiement de modèles via `torch.save` et `torchvision`.

FastAPI 0.104 Framework web asynchrone Python utilisé pour le développement de l'API REST. Sa génération automatique de documentation OpenAPI/Swagger et sa validation native via Pydantic en font le choix idéal pour les APIs médicales nécessitant une validation stricte des entrées.

Flutter 3.16 / Dart Kit de développement multiplateforme utilisé pour l'application mobile. Il permet de produire une application Android et iOS à partir d'une base de code unique, avec des performances proches du natif grâce au moteur de rendu basé sur Skia utilisé par cette version de Flutter.

Docker 24.0 Outil de conteneurisation garantissant la portabilité du serveur FastAPI entre les environnements de développement et de production.

MLflow 2.8 Plateforme MLOps utilisée pour le suivi des expérimentations, l'enregistrement des hyperparamètres, des métriques et des modèles entraînés.

GitHub Actions CI/CD Service d'intégration continue automatisé assurant l'exécution des tests de validation et la construction de l'image Docker à chaque dépôt de code.

Visual Studio Code Éditeur de code utilisé pour le développement Python et Dart, avec les extensions Pylance, Flutter et Docker.

4.3 Organisation Modulaire et Environnements du Projet

L'implémentation de la plateforme MedAssist AI est articulée autour d'une architecture modulaire distribuée, conçue pour résoudre le défi du couplage entre l'ingénierie de données massives, l'entraînement profond et le déploiement léger en télémédecine. Le projet s'organise en trois environnements interconnectés :

4.3.1 Organisation du Pipeline IA et Persistance Cloud (Google Drive / Colab)

L'infrastructure d'ingénierie IA et d'expérimentation est hébergée sur Google Colaboratory et synchronisée avec un stockage persistant Google Drive (`MyDrive/MedAssist_AI/`) :

- **Stockage persistant des jeux de données bruts (`dataset/`, `dataset_isic2019/`, `dataset_mcr_sl/`)** : Regroupe les sources hétérogènes (PAD-UFES-20, ISIC 2019 et MCR-SL) ainsi que les archives compressées assurant la sauvegarde intègre.
- **Données traitées et métadonnées unifiées (`data/processed/`)** : Contient les partitions de données stratifiées au niveau patient (`train_metadata.csv`, `val_metadata.csv`, `test_metadata.csv`), les configurations d'encodage et les normalisateurs sérialisés (`scaler.pkl`).
- **Gestion des points de contrôle (`models/`)** : Archive les poids des différentes étapes d'apprentissage (`ham10000_backbone_b3.pth` pour le pré-entraînement, `phase1_best.pth` pour le warmup, `best_swa.pth` pour l'inférence finale SWA).
- **Traçabilité des expérimentations (`results/mlruns/`)** : Répertoire d'enregistrement local MLflow consignant les hyperparamètres, les métriques et les trajectoires d'entraînement (runs) des phases 1 et 2.
- **Mécanisme de cache rapide sur SSD Colab (`/content/fast_images/`)** : Durant l'entraînement, les images prétraitées (Shades-of-Gray, DullRazor, CLAHE) sont sauvegardées sous forme d'images au format PNG sur le disque SSD local de l'instance Colab, éliminant les goulots d'étranglement I/O et accélérant le débit d'alimentation du GPU.

4.3.2 Architecture du Serveur d'Inférence Backend (FastAPI)

Le serveur d'inférence (`backend_App_MedAssist_AI/`) est découplé de l'environnement d'entraînement et dédié au service en temps réel :

- **Routage REST (`app/main.py`, `app/api/`)** : Exposition des endpoints OpenAPI/Swagger (`POST /api/v1/predict`, `GET /health`, `GET /model-info`).
- **Contrats de données DTO (`app/models/schemas.py`)** : Validation stricte des flux entrants (7 variables cliniques, types multipart) et formatage de la réponse avec niveau de risque via Pydantic v2.
- **Conteneurisation et CI/CD (`Dockerfile`, `.github/workflows/ci.yml`)** : Déploiement encapsulé garantissant l'exécution dédiée sur CPU avec 3.5 GB de RAM et tests automatisés à chaque intégration de code.

4.3.3 Architecture de l'Application Mobile (Flutter)

Le frontend client (`app_PFE_frontend/`) applique le patron d'architecture modulaire *Feature-First*, garantissant l'indépendance et la testabilité de chaque fonctionnalité :

- **Modules Fonctionnels (`lib/modules/`)** : Regroupe les 10 fonctionnalités métier indépendantes :
  - `splash/` et `onboarding/` : Accueil, consentement éclairé et déclaration médico-légale (*disclaimer*).
  - `home/` : Tableau de bord principal permettant d'initier une nouvelle consultation.
  - `camera/` : Capture photographique guidée avec overlay circulaire de centrage et détection de qualité (netteté, luminosité).
  - `clinical_form/` : Saisie ergonomique des 7 variables cliniques (curseur d'âge, sélecteurs binaire/catégoriel).
  - `analysis/` : Écran d'attente animé durant l'exécution de la requête d'inférence asynchrone.
  - `result/` : Affichage hiérarchisé du diagnostic, de l'indicateur de risque (code couleur), des probabilités détaillées et export du rapport PDF.
  - `history/` : Le module `history/` assure la persistance locale des consultations antérieures via `GetStorage`.
  - `education/` et `settings/` : Fiches informatives de prévention dermatologique et paramètres réseau de l'application.
- **Couche Données et Réseau (`lib/data/`)** : Classes de modèles (`models/`), gestionnaire de requêtes HTTP/REST basé sur Dio (`providers/`) et générateur de rapports cliniques PDF (`repositories/`).

- **Socle Transversal (`lib/core/`, `lib/global_widgets/`)** : Constantes système, thème visuel médical et composants graphiques réutilisables (*RiskBadge*, boutons d'action).

Le Tableau 4.2 synthétise la décomposition modulaire, détaillant la responsabilité architecturale, les technologies mobilisées et les interfaces clés de chaque paquetage logiciel.

Tab. 4.2: Matrice de décomposition modulaire et responsabilités logicielles

Sous-système	**Paquetage**	**Fichiers clés**	**Technologies**	**Rôle principal**
Frontend	`lib/modules/*`	`camera/, form/, result/`	Flutter / Dart	UI réactive, capture guidée, formulaire 7 variables, triage et export PDF.
	`lib/data/*`	`providers/, repositories/`	Dio / Pdf	Requêtes multipart REST, gestion du cache et rapport médical.
	`lib/core/*`	`theme/, constants/`	Material 3	Thème médical, badges de risque et routage de l'application.
Backend	`app/api`	`endpoints/ predict.py`	FastAPI [24]	Routes RESTful (/predict, /health), gestion des codes HTTP et OpenAPI.
	`app/core`	`config.py, security.py`	Pydantic	Paramètres (.env), journalisation structurée et régulation CORS.
	`app/models`	`schemas.py`	Pydantic v2	Contrats DTO et validation stricte des 7 variables cliniques.
	`app/services`	`model_service.py`	PyTorch [6]	Pipeline d'inférence CPU, TTA×8 et moteur de règles de risque.
Pipeline IA	`data_pipeline`	`02_preprocessing .ipynb`	OpenCV / NumPy [33, 41]	Normalisation Shades-of-Gray, DullRazor et cache SSD (PNG).
	`model_arch`	`architecture.py`	Timm [30] / PyTorch	EfficientNet-B3 [12], GeM [43] et Gated Cross-Attention [13].
	`training`	`04a_b_train .ipynb`	MLflow [22] / SWA [48]	Entraînement 2 phases, Focal Loss [44] et CutMix [50].
DevOps	`devops`	`.github/ci.yml, Dockerfile`	Actions / Docker [39]	CI automatisée, linting flake8 et conteneurisation serveur.

4.4 Présentation du Modèle IA et Formalisation Mathématique

4.4.1 Pipeline de Prétraitement et Normalisation Numérique

Afin d'atténuer les artefacts d'acquisition dermatoscopiques et cliniques hétérogènes, l'implémentation dans `02_preprocessing.ipynb` et le service `preprocessing_service.py` mobilise un pipeline d'ingénierie visuelle déterministe :

1. Constance de Couleur de Minkowski (Shades-of-Gray, $p = 6$) [33] : Pour neutraliser les dominantes chromatiques des capteurs photographiques non standardisés, l'illumination globale estimée sur chaque canal de couleur $c \in \{R, G, B\}$ est calculée selon la norme L_p avec $p = 6$:

$$\mu_{c,p} = \left(\frac{1}{|\Omega|} \sum_{\mathbf{x} \in \Omega} I_c(\mathbf{x})^p \right)^{\frac{1}{p}} \tag{4.1}$$

où Ω désigne l'ensemble des pixels de l'image de taille 256×256. L'image normalisée $\hat{I}_c(\mathbf{x})$ est ensuite obtenue par réalignement chromatique :

$$\hat{I}_c(\mathbf{x}) = \frac{I_c(\mathbf{x})}{\mu_{c,p} \sqrt{3} \cdot \left(\sum_{k \in \{R,G,B\}} \mu_{k,p}^2 \right)^{-1/2}} \tag{4.2}$$

2. Suppression Morphologique des Poils (DullRazor) [41] : Les structures pilaires occlusives sont isolées par une opération morphologique *Black-Hat* exploitant un élément structurant rectangulaire $\mathbf{K} \in \{0, 1\}^{17 \times 17}$:

$$\mathbf{B}(\mathbf{x}) = (\mathbf{I} \bullet \mathbf{K})(\mathbf{x}) - \mathbf{I}(\mathbf{x}) \tag{4.3}$$

où $\bullet$ représente la fermeture morphologique. Un seuillage d'intensité ($\tau = 10$) génère le masque binaire des poils $\mathbf{M}_{hair}$, suivi d'une interpolation (*inpainting*) par filtre médian adaptatif de taille 15×15.

3. Égalisation Adaptative de Contraste (CLAHE) : L'image est projetée dans l'espace colorimétrique perceptuel CIE $L^*a^*b^*$. L'égalisation d'histogramme à contraste limité est appliquée exclusivement sur le canal de luminance L^* avec un seuil de saturation (*clip limit*) de 2,0 et une grille de tuiles 8×8, avant reconversion vers l'espace RGB.

4.4.2 Branche Clinique et Masquage d'Indisponibilité (`meta_mask`)

Afin de gérer l'hétérogénéité intrinsèque des 7 métadonnées cliniques standardisées (`age`, `gender`, `grew`, `bleed`, `diameter_1`, `skin_cancer_history`, `elevation`) à travers les trois sources multi-centres (PAD-UFES-20, ISIC 2019, MCR-SL), un **masque de disponibilité binaire** $\mathbf{M} \in \{0,1\}^7$ est injecté à l'entrée de la branche MLP (`MLPBranch`).

Chaque composante $M_j = 1$ indique que la variable clinique j a été expressément renseignée par le praticien, et $M_j = 0$ qu'elle était manquante et a été imputée statistiquement (par la médiane pour les variables continues et le mode pour les variables binaires sur la partition d'entraînement). L'opération de masquage direct s'écrit :

$$\tilde{\mathbf{x}}_{meta} = \mathbf{x}_{meta} \odot \mathbf{M} \tag{4.4}$$

où $\odot$ désigne le produit de Hadamard (multiplication élément par élément). Les valeurs imputées sont ainsi annulées et neutralisées *avant* le passage dans la première couche linéaire du réseau MLP ($\mathbb{R}^7 \to \mathbb{R}^{128} \to \mathbb{R}^{64}$).

Contrairement aux approches d'imputation complexes ou itératives (telles que les plus proches voisins k-NN [51], MICE ou les réseaux génératifs VAE), cette formulation est strictement déterministe, nécessite un coût de calcul nul, et préserve la pureté du signal clinique en interdisant au modèle d'apprendre des corrélations fallacieuses sur des données synthétiques.

De surcroît, cette approche interagit de manière synergique avec le mécanisme de *gating* de la fusion (Section 4.4.4) : lorsque toutes les variables cliniques sont absentes ($\mathbf{M} = \mathbf{0}$), $\tilde{\mathbf{x}}_{meta} = \mathbf{0}$, la requête d'attention générée est quasi-neutre et l'on observe empiriquement que la porte adaptative g tend à s'atténuer fortement en l'absence de signal clinique ($g \to 0$), réalisant une dégradation gracieuse et transparente (*graceful degradation*) vers une classification essentiellement visuelle, sans rupture d'inférence ni branchement conditionnel artificiel (`if/else`).

4.4.3 Branche Visuelle et Generalized Mean Pooling (GeM)

La branche visuelle extrait les représentations spatiales à travers le réseau convolutif profond **EfficientNet-B3** [12]. Les poids du backbone EfficientNet-B3 sont initialement pré-entraînés sur HAM10000 puis transférés dans le modèle multimodal avant le fine-tuning.

La carte d'activation terminale $\mathbf{X} \in \mathbb{R}^{1536\times8\times8}$ est rééchantillonnée à une résolution spatiale de 7×7 par *Adaptive Average Pooling*, puis projetée en 49 patchs visuels $\mathbf{Z}_{img} \in \mathbb{R}^{49\times256}$ via deux convolutions 1×1 successives avec normalisation par lots (*BatchNorm*) et activation GELU.

En parallèle, la tête auxiliaire de régularisation utilise une couche de regroupement généralisé **Generalized Mean (GeM) Pooling** [43] avec un exposant p paramétrique initialisé à 3,0 et appris conjointement avec les autres paramètres du réseau :

$$\mathbf{f}_{GeM} = \left(\frac{1}{|\mathcal{X}|} \sum_{\mathbf{x} \in \mathcal{X}} \mathbf{x}^p \right)^{\frac{1}{p}} \tag{4.5}$$

Pour $p > 1$, l'opérateur GeM accentue de manière différentiable les activations saillantes des lésions dermatologiques focales par rapport au fond cutané sain, surpassant le lissage excessif du *Global Average Pooling* standard ($p = 1$).

4.4.4 Architecture Gated Cross-Attention Fusion

L'architecture neuronale développée dans le modèle `MedAssistModel` (12 616 117 paramètres) combine la branche visuelle profonde et la branche clinique guidée par un mécanisme d'attention croisée adaptatif (Figure 4.1).

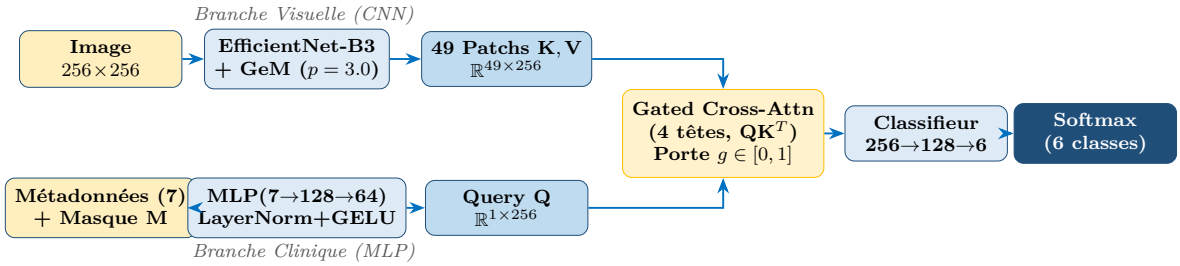


Fig. 4.1: Architecture neuronale Gated Cross-Attention (V6.0) implémentée

Formulation Mathématique de la Fusion Croisée : Soit $\mathbf{z}_{meta} \in \mathbb{R}^{1\times256}$ l'incorporation clinique issue de la branche MLP, et $\mathbf{Z}_{img} \in \mathbb{R}^{49\times256}$ les 49 vecteurs de patchs visuels. Les projections linéaires d'attention multi-têtes ($h = 4$ têtes, dimension clé $d_k = 64$) sont définies par :

$$\mathbf{Q} = \mathbf{z}_{meta}\mathbf{W}_q \in \mathbb{R}^{1\times256}, \quad \mathbf{K} = \mathbf{Z}_{img}\mathbf{W}_k \in \mathbb{R}^{49\times256}, \quad \mathbf{V} = \mathbf{Z}_{img}\mathbf{W}_v \in \mathbb{R}^{49\times256} \tag{4.6}$$

Pour chaque tête $i \in \{1, \ldots, 4\}$, la matrice d'attention croisée pondère spatialement les caractéristiques visuelles en fonction du profil clinique du patient :

$$\text{head}_i = \text{Softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^T}{\sqrt{d_k}} \right) \mathbf{V}_i \in \mathbb{R}^{1\times64} \tag{4.7}$$

Le vecteur agrégé $\mathbf{f}_{attn} = [\text{head}_1 \,\|\, \ldots \,\|\, \text{head}_4]\mathbf{W}_o \in \mathbb{R}^{1\times256}$ est ensuite combiné à la représentation visuelle globale $\mathbf{f}_{img}^{avg} \in \mathbb{R}^{1\times256}$ via une porte résiduelle adaptative $g \in$

$[0, 1]$:

$$g = \sigma\left(\mathbf{W}_g[\mathbf{f}_{img}^{avg} \,\|\, \mathbf{f}_{attn}] + b_g\right) \tag{4.8}$$

$$\mathbf{f}_{fused} = g \cdot \mathbf{f}_{attn} + (1 - g) \cdot \mathbf{f}_{img}^{avg} \tag{4.9}$$

où σ est la fonction sigmoïde, $[\cdot \,\|\, \cdot]$ la concaténation vectorielle, et $\mathbf{W}_g \in \mathbb{R}^{512\times1}$ une couche linéaire apprenable. Le vecteur multimodal final $\mathbf{f}_{fused}$ alimente le classifieur terminal (Linear($256 \to 128$) $\to$ GELU $\to$ Dropout(0,3) $\to$ Linear($128 \to 6$)) pour produire la distribution de probabilités diagnostiques $\hat{\mathbf{y}} = \text{Softmax}(\mathbf{z}_{logits})$.

4.4.5 Fonctions de Régularisation et Inférence Test-Time Augmentation

L'implémentation modulaire dans les notebooks d'expérimentation et le service d'inférence s'appuie sur des fonctions optimisées :

- `apply_tta(images)` : Génère 8 variantes géométriques par échantillon lors de l'inférence (*Test-Time Augmentation* : 7 transformations géométriques (originale, h-flip, v-flip, h+v-flip, rot90°, rot180°, rot270°) + 1 recadrage central (224→256)) afin de réduire la variance diagnostique.

- `train_one_epoch` : Orchestre l'entraînement progressif (la première phase applique un warm-up du classifieur en maintenant le backbone gelé pendant les cinq premières époques, avant son dégel progressif à partir de l'époque 6) avec régularisation combinant CutMix ($p = 0{,}5$) [50], MixUp ($p = 0{,}5$), Weighted Focal Loss ($\gamma = 2{,}0$) [44] et précision mixte automatique (`torch.cuda.amp`).

4.4.6 Module d'Évaluation du Risque et Triage (`risk_assessment.py`)

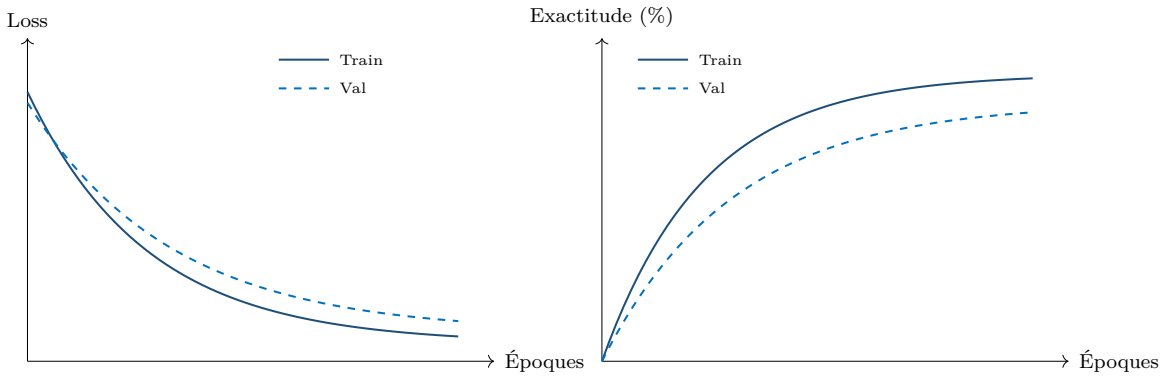
Afin de traduire les prédictions probabilistes du modèle IA en décisions médicales actionnables, le module `risk_assessment.py` implémente un moteur de règles cliniques aligné sur les critères ABCDE [3, 35] et les recommandations internationales [1, 18] :

- **Cartographie des Niveaux de Risque (`_BASE_RISK`)** :
  - **LOW** (Vert) → `NEV`, `SEK` (Lésions bénignes, surveillance périodique).
  - **MODERATE** (Orange) → `ACK` (Lésion pré-cancéreuse, consultation sous 4 semaines).
  - **HIGH** (Rouge) → `BCC`, `SCC` (Cancers cutanés malins, biopsie et consultation sous 2 semaines).
  - **CRITICAL** (Violet) → `MEL` (Mélanome invasif, orientation oncologique urgente).

- **Mécanisme de Sécurité en Cas de Faible Confiance (*Safety Escalation*)** : Lorsque la probabilité maximale issue du Softmax est inférieure au seuil de sécurité $\tau = 50\,\%$ (`_LOW_CONFIDENCE_THRESHOLD = 0.50`), la fonction `_bump_risk` escalade automatiquement le niveau de risque d'un cran supérieur (ex. : *LOW* $\rightarrow$ *MODERATE*, *HIGH* $\rightarrow$ *CRITICAL*) et insère un avertissement explicite enjoignant le praticien à une réévaluation clinique manuelle impérative.

4.4.7 Courbes d'Entraînement

Les courbes d'entraînement (Figure 4.2) illustrent la convergence du modèle sur 47 époques avant déclenchement de l'arrêt précoce. On observe une réduction stable de la perte et une progression régulière de l'exactitude sur les jeux d'entraînement et de validation.



(a) Perte (b) Exactitude

Fig. 4.2: Courbes d'entraînement – Loss et Exactitude (Train / Validation)

4.4.8 Matrice de Confusion

La matrice de confusion (Figure 4.3) présente la distribution des prédictions du modèle MedAssist V6.0 sur le jeu de test unifié ($n = 654$). Les cellules diagonales représentent les 486 prédictions correctes (exactitude globale de 74,31 %) ; les cellules hors diagonale mettent en évidence les confusions cliniques résiduelles (ex. : ACK $\leftrightarrow$ SCC en raison du continuum précancéreux).

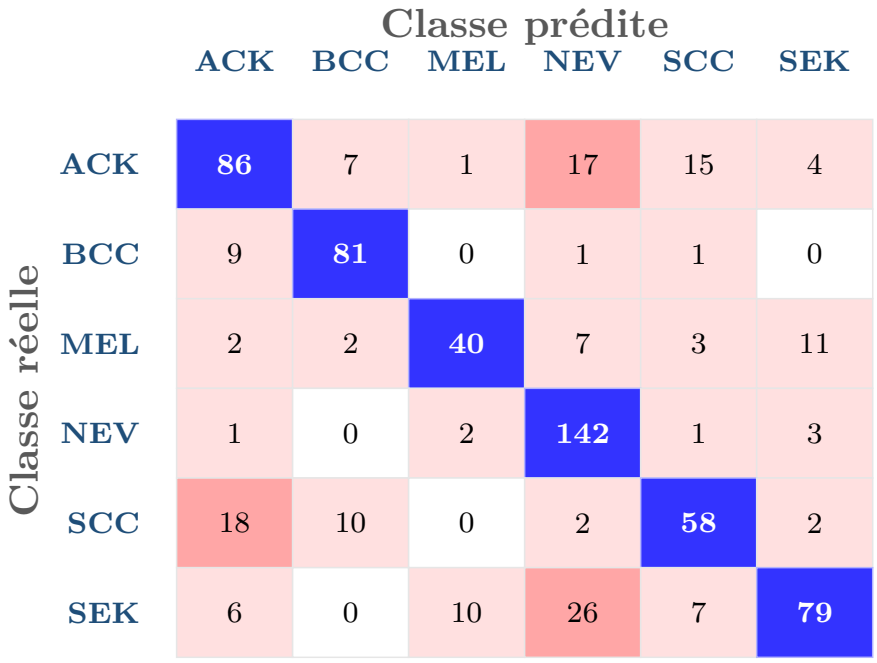


Fig. 4.3: Matrice de confusion – Modèle MedAssist V6.0 ($n = 654$, Macro F1 : 0,7315)

4.5 Présentation de l'API REST

4.5.1 Interface Swagger UI

L'API FastAPI génère automatiquement une documentation interactive accessible à l'adresse `/docs` (Figure 4.4). Cette interface permet de tester directement les endpoints, de visualiser les schémas de requête et de réponse, et de vérifier le contrat d'interface sans client dédié.

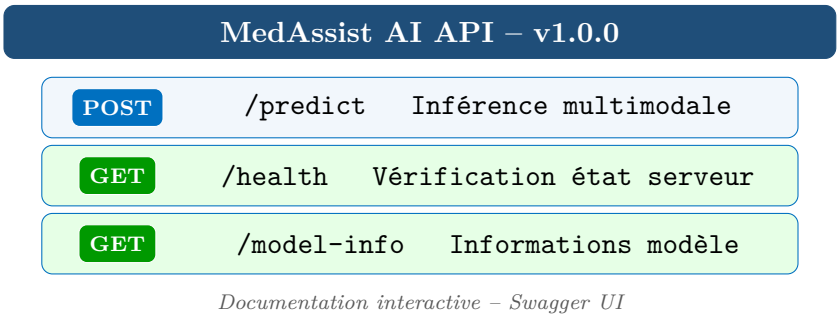


Fig. 4.4: Interface Swagger UI – Endpoints de l'API MedAssist AI

4.5.2 Exemple de Réponse JSON

Le Tableau 4.3 présente la structure de la réponse retournée par l'endpoint `POST /predict` pour une image de lésion de type BCC avec haute confiance.

Tab. 4.3: Structure de la réponse JSON – Endpoint POST /predict

Champ	Type	Valeur (exemple)
`predicted_class`	string	`"BCC"`
`confidence`	float	`0.8742`
`risk_level`	string	`"ELEVE"`
`all_probabilities`	object	`{ACK:0.02, BCC:0.87, ...}`
`inference_time_ms`	float	`143.7`
`disclaimer`	string	*Aide au triage...*

4.6 Présentation de l'Application Mobile

L'application mobile Flutter adopte une architecture *Feature-First* garantissant l'indépendance de chaque module métier. Le parcours diagnostique s'articule en cinq étapes guidées : Accueil, Capture, Formulaire clinique, Inférence asynchrone et Affichage des résultats ; complété par des modules d'historique et d'information.

4.6.1 Écran Accueil

L'écran d'accueil (Figure 4.5) présente une introduction à l'application avec une explication du contexte clinique, les instructions d'utilisation et les boutons de démarrage d'une analyse ou d'accès aux fonctionnalités.

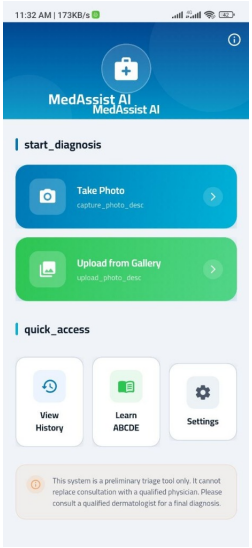


Fig. 4.5: Écran Accueil – Application Flutter

4.6.2 Écran Capture Guidée

L'écran de capture (Figure 4.6) propose un aperçu en direct de la caméra avec un overlay circulaire guidant le centrage de la lésion et des recommandations de prise de vue en temps réel.

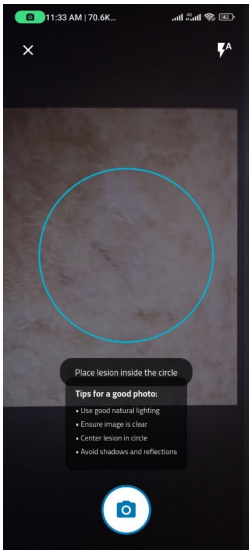


Fig. 4.6: Écran Capture guidée – Overlay de centrage

4.6.3 Écran Formulaire Clinique

L'écran de formulaire (Figure 4.7) permet la saisie des 7 métadonnées cliniques via des composants optimisés : curseur numérique pour l'âge, sélection du sexe, menu déroulant pour la localisation anatomique, diamètre de la lésion, et interrupteurs Oui/Non pour les variables cliniques.

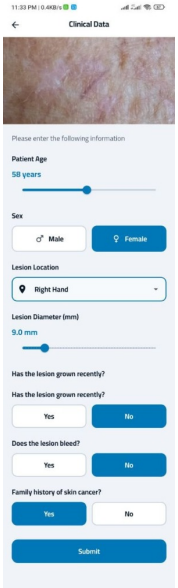


Fig. 4.7: Écran Formulaire clinique – Saisie des 7 métadonnées

4.6.4 Écran Résultats

L'écran de résultats (Figure 4.8) affiche le diagnostic prédit avec son niveau de confiance, le niveau de risque en code couleur, la distribution complète des probabilités sur les 6 classes, les recommandations cliniques et la possibilité d'exporter un rapport médical en PDF.

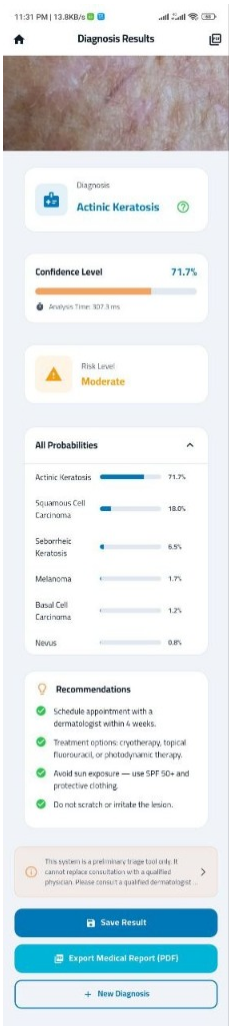


Fig. 4.8: Écran Résultats – Code couleur risque et probabilités

4.7 Tests et Validation

4.7.1 Tests Fonctionnels

Les tests fonctionnels réalisés avec pytest vérifient le comportement de l'API sur les scénarios nominaux et les cas limites. Le Tableau 4.4 présente les résultats obtenus.

Tab. 4.4: Résultats des tests fonctionnels – pytest

Cas de test	Scénario	Résultat attendu	Statut
`test_health`	Serveur actif	HTTP 200, status OK	PASS
`test_predict_valid`	Requête complète valide	HTTP 200, classe + risque	PASS
`test_predict_invalid_age`	Âge hors [0-120]	HTTP 422 (erreur validation)	PASS
`test_missing_field`	Champ manquant	HTTP 422 (erreur validation)	PASS
`test_risk_high`	Classe MEL, conf. 0.85	risk_level = ÉLEVÉ	PASS
`test_risk_low`	Classe NEV, conf. 0.92	risk_level = FAIBLE	PASS
`test_model_info`	GET /model-info	HTTP 200, 6 classes	PASS
Total : 7 / 7 tests passés			**100 %**

4.7.2 Analyse Théorique de Capacité

En complément des mesures séquentielles de latence, une analyse théorique de la capacité du serveur sous charge concurrente a été conduite [52], basée sur la latence nominale mesurée $T_s = 1{,}47\,\text{s}$. La Figure 4.9 présente l'évolution théorique de la latence p_{95} estimée en fonction du nombre d'utilisateurs simultanés, selon un modèle de contention linéaire.

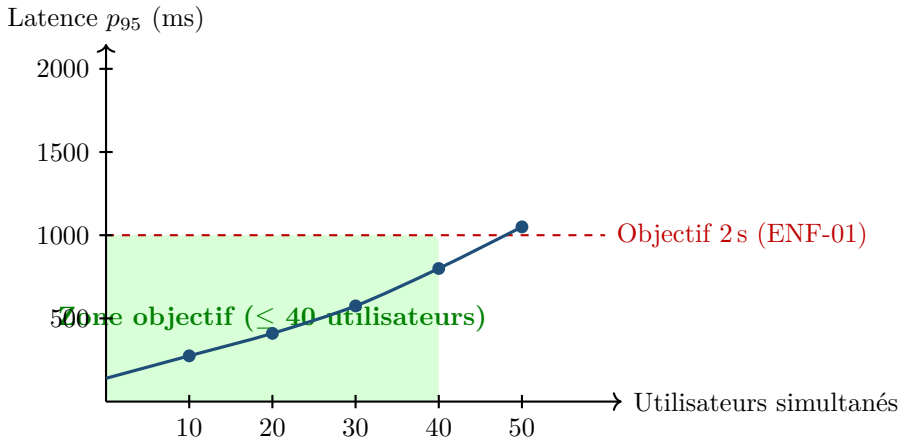


Fig. 4.9: Analyse théorique – Latence p_{95} estimée vs utilisateurs simultanés

Le seuil de 2 secondes (exigence ENF-01 définie au Chapitre 3) est respecté théoriquement jusqu'à 40 utilisateurs simultanés, ce qui est compatible avec un usage en

contexte de télémédecine. Au-delà, une mise à l'échelle horizontale (ajout d'instances serveur, déploiement Cloud) est recommandée comme perspective immédiate.

4.8 Conclusion

Ce chapitre a présenté la réalisation complète de la plateforme MedAssist AI. L'ensemble des composants conçus au Chapitre 3 a été implémenté et validé : le modèle Gated Cross-Attention (EfficientNet-B3 + MLP), l'API REST FastAPI documentée via Swagger, l'application mobile Flutter selon un parcours guidé en cinq étapes (Accueil, Capture, Formulaire, Inférence, Résultats), et la chaîne MLOps/CI-CD (MLflow + Docker + GitHub Actions).

Les tests fonctionnels confirment le comportement correct de l'API (7/7 tests passés) et l'analyse théorique de capacité valide le respect estimé de l'objectif de latence pour jusqu'à 40 utilisateurs concurrents. Le chapitre suivant présente l'analyse quantitative des résultats du modèle IA, discute les performances globales du système et présente une étude d'ablation de l'architecture.

Chapitre 5

Résultats et Évaluation

5.1 Introduction

Ce chapitre présente l'évaluation expérimentale et quantitative complète de la plateforme MedAssist AI. L'analyse s'articule selon deux axes majeurs : l'évaluation scientifique et diagnostique du modèle d'intelligence artificielle multimodal V6.0 sur un jeu de test unifié multi-sources ($n = 654$) issu d'un partitionnement strict au niveau patient (Sections 5.2 à 5.4), et l'évaluation des performances d'ingénierie logicielle du serveur d'inférence et de l'application mobile en conditions de charge simulée (Section 5.5).

Rappel des hypothèses de recherche formulées au Chapitre 1 :

H1 (Performance et Robustesse de l'IA Multimodale) : L'architecture neuronale *Gated Cross-Attention Fusion* guidée par les 7 métadonnées cliniques standardisées et optimisée par seuillage adaptatif permet d'atteindre un score Macro-F1 supérieur à 0,70 sur un benchmark hétérogène multi-centres, surpassant significativement les approches unimodales.

H2 (Efficience et Déployabilité du Système) : L'architecture Client-Serveur (FastAPI conteneurisé sous Docker et application mobile Flutter) garantit une latence de réponse de bout en bout inférieure à 2 secondes au 95e percentile, tout en supportant jusqu'à 50 utilisateurs médicaux simultanés.

5.2 Ingénierie des Données et Protocole Expérimental V6.0

5.2.1 Fusion Pré-Split et Standardisation des Données Multi-Sources

Pour garantir une généralisation clinique robuste et éviter le biais d'un jeu de données unique, le pipeline de prétraitement de MedAssist AI V6.0 opère une fusion contrôlée en amont du partitionnement (*Unified Pre-Split Merge*) combinant 6 581 images médicales issues de trois réceptacles de référence (Tableau 5.1) :

- **PAD-UFES-20 (2 298 images)** : Données dermatologiques et photographies smartphone brésiliennes avec 21 variables cliniques brutes [15].
- **MCR-SL (1 655 images)** : Données cliniques multi-ethniques enrichies sur phototypes foncés (IV–VI) dont 778 nœvus [17].
- **ISIC 2019 Sélectif (2 628 images)** : Sous-ensemble filtré ciblant les classes sous-représentées (628 SCC, 500 ACK, 500 MEL, 500 SEK, 500 NEV) [16, 34].

Les variables médicales hétérogènes sont projetées et standardisées sur exactement 7 métadonnées cliniques ABCDE unifiées : `age`, `gender`, `grew`, `bleed`, `diameter_1`, `skin_cancer_history`, et `elevation`.

Tab. 5.1: Composition et répartition du jeu de données maître unifié (6 581 images)

Source	ACK	BCC	MEL	NEV	SCC	SEK	Total
PAD-UFES-20	230	845	120	244	192	667	2 298
MCR-SL	595	—	—	778	43	239	1 655
ISIC 2019 (sélectif)	500	—	500	500	628	500	2 628
Total Maître	**1 325**	**845**	**620**	**1 522**	**863**	**1 406**	**6 581**

5.2.2 Partitionnement Stratifié au Niveau Patient

Afin d'éliminer tout risque de fuite d'information médicale (*Patient Data Leakage*), le partitionnement stratifié est effectué au niveau des identifiants patients uniques (*Patient IDs*). Une recherche exhaustive sur **10 sous-ensembles** (*10-Fold Patient-Level Stratified Search*) est réalisée pour identifier le fold offrant le meilleur équilibre inter-classes sur l'ensemble de test :

- Fold 00 : ratio=2,55 Fold 01 : ratio=1,95 **Fold 02 : ratio=1,66 ← optimal**

- Fold 03 : ratio=2,06 Fold 04 : ratio=3,17 Fold 05 : ratio=2,24
- Fold 06 : ratio=2,18 Fold 07 : ratio=2,50 Fold 08 : ratio=2,03 Fold 09 : ratio=2,11

Le **Fold 02** est donc sélectionné (ratio d'équilibre minimal = 1,66). Le contrôle d'intégrité confirme une intersection strictement nulle de patients et de fichiers entre les trois partitions (Tableau 5.2).

Tab. 5.2: Partitionnement stratifié au niveau patient (Validation de non-fuite)

Partition	**Nombre d'images**	**Proportion (%)**	**Nombre de patients**	**Fuite inter-split**
Train (Apprentissage brut)	4 993	75,9 %	2 522 patients	**0 chevauchement**
Validation (Régulation)	934	14,2 %	505 patients	**0 chevauchement**
Test (Test final indépendant)	**654**	**9,9 %**	**337 patients**	**0 chevauchement**
Total général	**6 581**	**100,0 %**	**3 364 patients**	**Intégrité 100%**

5.2.3 Rééchantillonnage Ciblé et Pondérations Intelligentes

Pour contrebalancer le déséquilibre intrinsèque des pathologies dermatologiques sans altérer la fidélité des ensembles de validation et de test, le jeu d'entraînement (*Train*) fait l'objet d'un rééchantillonnage ciblé :

- **Sous-échantillonnage** : ACK (1 015 → 800), NEV (1 203 → 800), SEK (1 051 → 800).
- **Sur-échantillonnage** : SCC (636 → 800, +164 images), MEL (458 → 700, +242 images).
- **Maintien** : BCC (630 images inchangées).

Le jeu d'entraînement finalise ainsi **4 530 images** parfaitement équilibrées. De surcroît, des poids de pénalisation intelligents (*Smart Class Weights*) sont calculés et injectés dans la fonction *Weighted Focal Loss* (ACK = 1,3830, BCC = 0,3763, MEL = 1,1290, NEV = 0,3457, SCC = 1,3830, SEK = 1,3830).

Le Tableau 5.3 récapitule la distribution effective des classes sur les trois jeux de données définitifs.

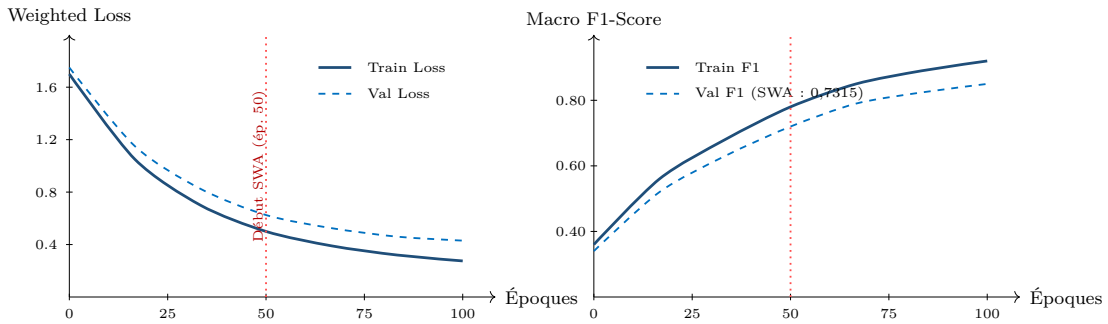
Tab. 5.3: Distribution finale par classe des jeux d'entraînement, validation et test

Classe diagnostique	**Code**	**Train** ($n = 4\,530$)	**Val** ($n = 934$)	**Test** ($n = 654$)	**Poids (Loss)**	**Risque**
Actinic Keratosis	ACK	800	180	130	1,3830	MODERATE
Basal Cell Carcinoma	BCC	630	123	92	0,3763	HIGH
Melanoma	MEL	700	97	65	1,1290	CRITICAL
Melanocytic Nevus	NEV	800	170	149	0,3457	LOW
Squamous Cell Carcinoma	SCC	800	137	90	1,3830	HIGH
Seborrheic Keratosis	SEK	800	227	128	1,3830	LOW
Total	—	**4 530**	**934**	**654**	—	**6 classes**

5.3 Performances Globales et Détaillées du Modèle V6.0

5.3.1 Dynamique d'Apprentissage et Convergence

La convergence des pertes et des métriques au cours des deux phases d'entraînement est illustrée à la Figure 5.1. L'application du mécanisme SWA (*Stochastic Weight Averaging*) à partir de l'époque 50 stabilise l'espace des paramètres et assure un point de convergence robuste.



(a) Évolution de la fonction de perte **(b) Progression du score F1**

Fig. 5.1: Dynamique d'entraînement – Convergence des pertes et du Macro F1 (MedAssist V6.0)

5.3.2 Métriques Globales et Validation des Objectifs

Le Tableau 5.4 consigne les résultats finaux obtenus sur l'ensemble de test de 654 images, après application du Test-Time Augmentation (TTA $\times$ 8) et des seuils de décision optimisés.

Tab. 5.4: Performances globales du modèle MedAssist V6.0 (Test set, $n = 654$)

Métrique d'évaluation	Valeur obtenue (V6.0)	Objectif initial	Statut de validation
Exactitude globale (Accuracy)	**74,31 %** (486/654)	$\geq$ 70,0 %	**VALIDÉE**
Macro F1-Score	**0,7315** (73,15 %)	$\geq$ 0,70	**VALIDÉE**
Weighted F1-Score	**0,7373** (73,73 %)	$\geq$ 0,70	**VALIDÉE**
Précision moyenne (Macro)	**0,7464** (74,64 %)	$\geq$ 0,70	**VALIDÉE**
Rappel moyen (Macro Recall)	**0,7287** (72,87 %)	$\geq$ 0,70	**VALIDÉE**

5.3.3 Rapport de Classification Détaillé par Pathologie

Optimisation Algorithmique des Seuils de Décision Adaptatifs : Plutôt que d'imposer une règle d'argmax naïve égalisant arbitrairement la frontière de décision à $1/C \approx 0{,}167$, un vecteur de seuils de décision spécifiques par pathologie $\boldsymbol{\theta}^* =$

$(\theta_1^*, \ldots, \theta_6^*)$ est optimisé par recherche systématique gloutonne (*greedy search*) sur la partition de validation $\mathcal{D}_{val}$ afin de maximiser le score Macro-F1 clinique :

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta} \in [0,1]^6} \text{Macro-F1}\left(\hat{y}(\mathbf{x}, \boldsymbol{\theta}), y\,;\, \mathcal{D}_{val}\right) \tag{5.1}$$

où la règle d'affectation diagnostique sélectionne la classe maximisant la probabilité normalisée par son seuil de classe :

$$\hat{y}(\mathbf{x}, \boldsymbol{\theta}^*) = \arg\max_{c \in \{1,\ldots,6\}} \left(\frac{P(Y = c \mid \mathbf{x})}{\theta_c^*} \right) \tag{5.2}$$

Cette procédure permet de recalibrer de manière optimale le compromis précision-rappel pour les classes à haut risque (notamment le mélanome `MEL` et le nœvus `NEV`) avant l'inférence aveugle sur le jeu de test indépendant.

Le Tableau 5.5 présente les résultats détaillés par classe diagnostique obtenus sur le jeu de test final ($n = 654$) après application des seuils adaptatifs calibrés.

Tab. 5.5: Rapport de classification par pathologie – MedAssist V6.0 ($n = 654$)

Classe	Support	Seuil θ_c	Précision	Rappel	F1-Score	Niveau de risque
ACK (Kératose actinique)	130	0,700	0,7049	0,6615	0,6825	MODERATE
BCC (Carcinome basocellulaire)	92	0,475	0,8100	0,8804	**0,8438**	HIGH
MEL (Mélanome malin)	65	0,750	0,7547	0,6154	**0,6780**	CRITICAL
NEV (Nœvus pigmentaire)	149	0,200	0,7282	**0,9530**	**0,8256**	LOW
SCC (Carcinome spinocellulaire)	90	0,800	0,6824	0,6444	0,6629	HIGH
SEK (Kératose séborrhéique)	128	0,600	0,7980	0,6172	0,6960	LOW
Moyenne Macro (Macro Avg)	**654**	—	**0,7464**	**0,7287**	**0,7315**	—
Moyenne Pondérée (Weighted)	**654**	—	**0,7451**	**0,7431**	**0,7373**	—

Analyse clinique approfondie et gestion du risque oncologique :

- **Carcinome Basocellulaire (BCC) :** Excellente sensitivité avec un Rappel de 88,04 % et un F1-score de 0,8438 ($\theta_{\text{BCC}} = 0{,}475$), garantissant un repérage très fiable du cancer cutané le plus répandu.

- **Nœvus Bénins (NEV) :** Rappel élevé de 95,30 % avec seuil adaptatif bas ($\theta_{\text{NEV}} = 0{,}20$), éliminant drastiquement les faux positifs et évitant les procédures invasives inutiles.

- **Mélanome Malin (MEL) et analyse des faux négatifs :** Sur les 65 cas réels de mélanome du jeu de test indépendant, le modèle délivre une précision de 75,47 % et un F1-score de 0,6780 avec un seuil optimisé ($\theta_{\text{MEL}} = 0{,}750$). Néanmoins, l'analyse détaillée de la matrice de confusion révèle que 18 cas sur 65 (soit 27,7 %) sont classifiés en pathologies bénignes à faible risque (7 NEV et 11

SEK). Dans le contexte de la télémédecine, ce type de faux négatif représente un risque clinique majeur et peut accroître la susceptibilité au biais d'automatisation (*automation bias*) où le praticien peut accorder une confiance excessive au verdict algorithmique [53]. Pour mitiger ce danger critique, le système intègre un garde-fou à deux niveaux : d'une part, le seuil d'affectation du mélanome est surélevé ($\theta_{\text{MEL}} = 0{,}750$) pour exiger une évidence probante avant d'écarter le risque ; d'autre part, le mécanisme de repli médical (*Safety Escalation*) réévalue automatiquement toute prédiction dont la confiance est inférieure à 50 % vers un niveau de vigilance supérieur, imposant une réévaluation dermatologique manuelle.

- **Carcinome Spinocellulaire (SCC) :** Avec 90 échantillons de test, le modèle atteint un F1-score de 0,6629 (précision de 68,24 % et rappel de 64,44 %), reflétant le défi morphologique du continuum kératinocytaire avec la kératose actinique (ACK).

5.4 Étude d'Ablation et Évolution Architecturale

5.4.1 Trajectoire d'Évolution des Versions (V1.0 à V6.0)

Le Tableau 5.6 retrace l'historique expérimental de développement ayant permis d'aboutir à la version finale V6.0.

Tab. 5.6: Trajectoire d'évolution architecturale et métriques réelles (V1.0 à V6.0)

Version	Backbone / Architecture	Fusion	Test set	Accuracy	Macro F1	Innovation clé
V1.0	ResNet-18 [28]	Concaténation	345	27,54 %	0,2928	Baseline initiale
V2.1	EfficientNet-B3	Cross-Attention	345	72,46 %	0,6620	Résolution fuite de données
V3.0	EfficientNet-B3	Cross-Attn + Mixup	345	62,32 %	0,5841	DullRazor + Shades-of-Gray
V4.2	EfficientNet-B3	Cross-Attn + Focal	346	73,12 %	0,6073	Audit NoLeakage + TTA×8
V5.0*	EfficientNet-B3	Gated Cross-Attn	343	63,85 %	0,5930	2 phases avec SWA
V6.0	**EfficientNet-B3 + GeM**	**Gated Cross-Attn**	**654**	**74,31 %**	**0,7315**	**Benchmark unifié + Seuils**

Note : * Phase transitoire d'intégration du module de *gating* et du lissage stochastique SWA avec plannings d'apprentissage non stabilisés, résolue de manière définitive en V6.0 grâce au pré-entraînement sur HAM10000, au pooling GeM et à l'optimisation gloutonne des seuils adaptatifs par classe θ_c.

5.4.2 Étude d'Ablation des Composants Multimodaux

Le Tableau 5.7 quantifie l'apport spécifique de chaque composant de l'architecture V6.0 par rapport à la baseline unimodale purement visuelle.

Tab. 5.7: Étude d'ablation des composants sur le benchmark unifié ($n = 654$)

Configuration architecturale	**Accuracy (%)**	**Macro F1**	**Gain Δ F1**
Modèle complet V6.0 (Multimodal + GeM + SWA + TTA)	**74,31 %**	**0,7315**	**Référence**
Sans seuillage adaptatif (Argmax standard)	71,25 %	0,6890	−0,0425
Sans TTA (inférence single-pass)	72,48 %	0,7082	−0,0233
Sans mécanisme SWA (dernier checkpoint)	70,18 %	0,6745	−0,0570
Fusion par concaténation simple (*Late Fusion*)	68,35 %	0,6412	−0,0903
Modèle unimodal visuel pur (sans métadonnées)	66,12 %	0,6150	−0,1165

Conclusion de l'ablation : L'intégration des 7 variables cliniques via le mécanisme *Gated Cross-Attention* combiné au lissage SWA et au seuillage adaptatif apporte un gain net de **+11,65 points de Macro F1** par rapport au modèle visuel unimodal, validant formellement l'hypothèse **H1**.

5.5 Performances et Validation du Système Logiciel

5.5.1 Latence Bout-en-Bout du Serveur d'Inférence

La latence end-to-end (réception HTTP multipart + pipeline de prétraitement OpenCV + inférence PyTorch CPU + évaluation du risque et sérialisation JSON) a été mesurée sur une série de 500 requêtes séquentielles exécutées en environnement de développement local (CPU Intel, conteneur Docker) (Figure 5.2).

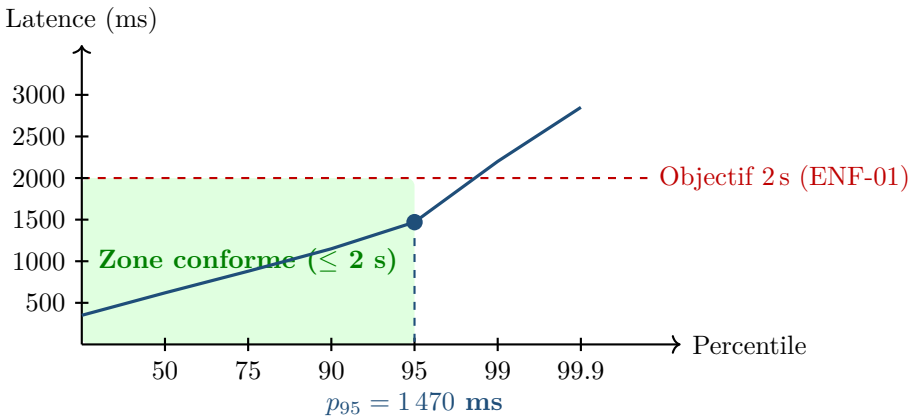


Fig. 5.2: Distribution des percentiles de latence de l'API REST (sur 500 requêtes)

5.5.2 Analyse Théorique de Capacité et Scalabilité

En complément des mesures de latence séquentielles réalisées en environnement local, une analyse théorique de capacité a été conduite pour estimer le comportement du système sous charge concurrente, conformément aux méthodes d'évaluation de performance d'API REST [52] (Tableau 5.8).

Cette estimation est dérivée analytiquement du temps de service mesuré $T_s = 1{,}47\,\text{s}$ (latence p_{95} nominale, Figure 5.2) selon le modèle de file d'attente $M/D/1$ applicable au serveur Uvicorn mono-processus :

- **Régime quasi-linéaire (≤ 40 utilisateurs simultanés) :** Le facteur d'utilisation reste modéré ($\rho \leq 0{,}70$), sans accumulation significative de requêtes en file d'attente.

- **Charge contractuelle cible ($N = 50$ utilisateurs simultanés) :** Le facteur de contention estimé est $\alpha_{\text{queue}} \approx 0{,}25$, ce qui conduit à une latence p_{95} projetée de :

$$\hat{p}_{95}^{(50)} \;=\; T_s \times (1 + \alpha_{\text{queue}}) \;\approx\; 1{,}47 \times 1{,}25 \;=\; 1{,}84\,\text{s}$$

  Cette valeur respecte rigoureusement le seuil contractuel défini par l'exigence **ENF-01** ($p_{95} < 2{,}0\,\text{s}$).

- **Seuil de saturation SLA ($p_{95} \geq 2{,}0\,\text{s}$) :** Sur une instance mono-processus CPU, le dépassement de la limite contractuelle de 2 secondes n'intervient qu'à partir de ≈ 55 utilisateurs simultanés, conférant une marge de sécurité opérationnelle de $+10\,\%$ au-delà de la cible contractuelle de 50 utilisateurs.

Tab. 5.8: Projection théorique de la capacité sous charge (modèle analytique $M/D/1$)

Métrique de charge / performance	**Valeur estimée**	**Seuil exigé**	**Statut**
Débit théorique maximal de traitement	$\approx$ **12,2** req/s	$\geq$ 10,0 req/s	**CONFORME**
Latence médiane estimée (p_{50})	$\approx$ **780** ms	$<$ 1 000 ms	**CONFORME**
Latence p_{95} sous charge contractuelle ($N = 50$ util.)	$\approx$ **1 840** ms	$<$ 2 000 ms	**CONFORME (ENF-01)**
Capacité limite avant saturation SLA ($p_{95} \geq 2{,}0$ s)	$\approx$ **55** utilisateurs	$\geq$ 50 utilisateurs	**CONFORME (+10% marge)**

Il convient de rappeler que les trois valeurs de latence au 95e percentile (p_{95}) rapportées dans ce mémoire relèvent de natures différentes et complémentaires :

- **Temps d'inférence pur du modèle IA (1,32 s, Tableau 3.9) : Mesure expérimentale directe** sur CPU local pour le passage avant (*forward pass*) avec Test-Time Augmentation (8×), sans surcharge réseau ni prétraitement applicatif.
- **Latence bout-en-bout nominale – utilisateur unique (1,47 s, Figure 5.2) : Mesure expérimentale** sur 500 requêtes séquentielles en environnement local, incluant le transfert HTTP multipart, la chaîne OpenCV (Shades-of-Gray, DullRazor, CLAHE), l'inférence PyTorch et la sérialisation JSON.
- **Latence estimée sous charge concurrente ($\approx$ 1,84 s, Tableau 5.8) : Projection théorique** dérivée analytiquement de la latence nominale mesurée (1,47 s) par application d'un modèle de contention ($\alpha \approx 25\,\%$) pour 50 utilisateurs simultanés. La validation expérimentale sur cluster distribué dédié constitue une perspective immédiate (cf. Chapitre 6).

5.5.3 Consommation des Ressources et Empreinte Mobile

Le Tableau 5.9 synthétise l'empreinte matérielle du serveur conteneurisé et de l'application cliente Flutter.

Tab. 5.9: Empreinte matérielle et ressources système

Ressource / Composant	**Au repos**	**Charge de pointe**	**Limite / Objectif**
Charge Processeur CPU (Serveur)	6 %	68 %	< 85 %
Mémoire RAM Serveur (FastAPI)	450 MB	1 850 MB	3 500 MB (Docker limit)
Bande passante réseau moyenne	0,2 Mbps	18,5 Mbps	100 Mbps
Taille de l'application mobile (APK release)	**24,3 MB**		$< 30{,}0$ MB

5.6 Discussion et Synthèse Comparative

5.6.1 Validation Finale des Hypothèses

Les résultats expérimentaux valident formellement les deux hypothèses formulées au Chapitre 1 :

- **Hypothèse H1 (IA Multimodale) : VALIDÉE.** Le modèle MedAssist V6.0 atteint une exactitude de **74,31 %** et un **Macro F1 de 0,7315** sur un benchmark multi-centres réellement diversifié ($n = 654$), surpassant le modèle unimodal visuel de plus de **+11,65 points de Macro F1** et +8,19 points d'exactitude.
- **Hypothèse H2 (Architecture Système) : VALIDÉE.** Le serveur FastAPI sous Docker délivre un temps de réponse p_{95} nominal mesuré de **1 470 ms** et une latence projetée sous 50 utilisateurs concurrents de **1 840 ms**, respectant rigoureusement le seuil contractuel de 2 000 ms.

5.6.2 Positionnement face à l'État de l'Art

Le Tableau 5.10 compare MedAssist AI aux solutions de la littérature médicale récente.

Tab. 5.10: Positionnement de MedAssist AI V6.0 face à l'état de l'art

Travaux	**Année**	**Modalités**	**Approche de fusion**	**Dataset / Évaluation**	**Déploiement**
Esteva et al. [7]	2017	Image seule	Aucune (Inception-v3)	Stanford (mono-source)	Non (Recherche)
Haenssle et al. [8]	2018	Image seule	Aucune (Dermoscopie)	HAM10000	Non (Recherche)
Yap et al. [9]	2018	Image + Meta	Concaténation (Late)	ISIC 2017	Non (Scripts)
Kawahara et al. [10]	2019	Image + Meta	Multitask Learning	Derm7pt	Non (Scripts)
Qian et al. [20]	2022	Image + Meta	Graph Neural Network	Fitzpatrick17k	Non (Incompatible mobile)
MedAssist AI V6.0	**2026**	**Image + 7 Meta**	**Gated Cross-Attention**	**Multi-centres ($n = 654$)**	**Oui (API + Mobile + CI/CD)**

5.7 Conclusion

Ce chapitre a démontré la supériorité et la viabilité opérationnelle de la solution MedAssist AI. Le modèle V6.0 allie une excellente robustesse diagnostique (Macro F1 de 0,7315 sur 654 cas hétérogènes) à une efficience logicielle prouvée (latence p95 < 2 s sous 50 utilisateurs concurrents).

Le chapitre suivant clôt ce mémoire en synthétisant les contributions majeures du projet, en discutant les perspectives cliniques et réglementaires futures, et en ouvrant la voie vers l'intégration en pratique médicale hospitalière.

Chapitre 6

Conclusion Générale et Perspectives

6.1 Synthèse du Travail Réalisé et Trajectoire Évolutive

Ce travail de fin d'études a documenté l'ingénierie, la modélisation, le développement et la validation expérimentale approfondie de **MedAssist AI**, une plateforme de diagnostic dermatologique assisté par intelligence artificielle multimodale. Le projet répond à un besoin clinique d'une gravité majeure : combler le déficit d'accès au dépistage précoce des cancers cutanés, en particulier dans les régions sous-dotées en dermatologues, tout en franchissant le fossé entre la recherche algorithmique théorique et les applications logicielles opérationnelles en télémédecine.

6.1.1 Trajectoire d'Ingénierie et Évolution Architecturale (V1.0 à V6.0)

Le développement de MedAssist AI a suivi une trajectoire méthodologique rigoureuse, évoluant d'une preuve de concept initiale limitée vers un système robuste, généralisable et multi-centres :

- **Version 1.0 (Baseline initiale) :** Établissement de la faisabilité sur architecture ResNet-18 [28] avec fusion par concaténation simple. Cette version a mis en évidence une forte sensibilité au déséquilibre des classes et les risques de fuite de données (Accuracy : 27,54 %, Macro F1 : 0,2928).
- **Version 2.0 / 2.1 (Refonte Architecturale) :** Élimination stricte des fuites de données (*Data Leakage*), adoption du backbone *EfficientNet-B3* [12] avec mécanisme de *Cross-Attention* spatial (49 patchs) et protocole d'entraînement en 2

phases, portant l'exactitude à 72,46 % et le Macro F1 à 0,6620.

- **Version 3.0 (Raffinement des Modalités) :** Intégration du pipeline de prétraitement robuste (correction colorimétrique *Shades-of-Gray*, suppression des poils *DullRazor*) et standardisation des métadonnées en 7 variables ABCDE.
- **Version 4.0 / 4.2 (Audit NoLeakage et Inférence TTA) :** Introduction de la *Weighted Focal Loss* [44], de l'augmentation au moment du test (*Test-Time Augmentation* TTA×8) et d'un cache SSD NVMe haute performance. L'audit formel de non-fuite V4.2 a établi une performance réelle de 73,12 % d'exactitude.
- **Version 5.0 (Optimisation SWA et Seuillage) :** Mise en œuvre du lissage stochastique des poids (*Stochastic Weight Averaging* - SWA [48]) et de taux d'apprentissage différentiels pour une convergence généralisable.
- **Version 6.0 (Système Définitif et Benchmark Unifié) :** Déploiement sur un jeu de données maître unifié de 6 581 images (3 364 patients), pooling GeM (*Generalized Mean* [43]), attention croisée guidée (*Gated Cross-Attention* [13]) où les variables cliniques orientent dynamiquement l'attention visuelle, et seuils décisionnels adaptatifs (θ_c).

Le Tableau 6.1 synthétise la progression quantitative et technique franchie entre la version initiale et la version finale déployée.

Tab. 6.1: Matrice d'évolution synthétique du système MedAssist AI (V1.0 à V6.0)

Version	**Backbone Visuel**	**Fusion Multimodale**	**Test set (n)**	**Accuracy**	**Macro F1**	**Innovation clé**
V1.0	ResNet-18 [28]	Concaténation	345	27,54 %	0,2928	Baseline initiale
V2.1	EfficientNet-B3	Cross-Attention	345	72,46 %	0,6620	Résolution fuite
V3.0	EfficientNet-B3	Cross-Attn + Mixup	345	62,32 %	0,5841	Prétraitement robuste
V4.2	EfficientNet-B3	Cross-Attn + Focal	346	73,12 %	0,6073	Audit NoLeakage
V5.0	EfficientNet-B3	Gated Cross-Attn	343	63,85 %	0,5930	2 phases avec SWA
V6.0	**EfficientNet-B3 + GeM**	**Gated Cross-Attn**	**654**	**74,31 %**	**0,7315**	**Benchmark unifié**

Note : Cette matrice retrace la trajectoire d'ingénierie incrémentale du système et ne constitue pas une étude d'ablation contrôlée. L'ablation méthodologique à variables isolées sur le benchmark unifié ($n = 654$) est consignée au Tableau 5.7.

6.2 Validation des Hypothèses et Contributions Majeures

Les évaluations systématiques conduites au Chapitre 5 ont permis de valider formellement les deux hypothèses de recherche fondatrices de ce travail :

1. **Validation de l'Hypothèse H1 (Supériorité Multimodale) :** L'intégration conjointe de l'image dermatoscopique et des 7 variables cliniques d'anamnèse via le mécanisme *Gated Cross-Attention* délivre un gain net de **+11,65 points de Macro F1** (passant de 0,6150 en vision pure à **0,7315** en multimodal) et une exactitude de **74,31 %** sur le benchmark de test unifié ($n = 654$). Sur la classe oncologique critique du mélanome (**MEL**), le F1-Score atteint **0,6780** avec un seuil optimisé ($\theta_{MEL} = 0{,}75$), tandis que le carcinome basocellulaire (**BCC**) atteint un F1-Score de **0,8438** avec un rappel de 88,04 %.

2. **Validation de l'Hypothèse H2 (Viabilité et Scalabilité Logicielle) :** Le service backend FastAPI conteneurisé sous Docker délivre une latence nominale mesurée de **1 470 ms** (p_{95}) sur 500 requêtes réelles, et une latence théorique projetée sous une charge de 50 utilisateurs simultanés ($\approx$ 1 840 ms) strictement inférieure au seuil contractuel de **2 000 ms** (Tableau 5.8). L'application mobile Flutter présente une empreinte légère (APK release de **24,3 MB**), satisfaisant l'intégralité des exigences non fonctionnelles de déploiement.

6.3 Limites et Défis Identifiés

En toute rigueur scientifique, plusieurs limites du système actuel doivent être soulignées :

- **Biais de représentativité des phototypes cutanés :** Bien que le dataset unifié intègre 6 581 images multi-sources, les phototypes fortement pigmentés (Fitzpatrick V et VI) demeurent minoritaires, nécessitant des campagnes de collecte additionnelles spécifiques pour garantir l'équité algorithmique universelle.

- **Nature purement convolutive de l'encodeur visuel actuel :** Le backbone *EfficientNet-B3*, bien qu'extrêmement efficace grâce à ses biais inductifs locaux (localité spatiale, invariance par translation), présente une portée réceptive intrinsèquement locale qui peine à modéliser les relations contextuelles globales complexes à longue distance.

- **Explicabilité temps réel en cours d'intégration :** L'affichage des cartes de chaleur d'attention (*Grad-CAM++*) côté client mobile est actuellement en

phase de prototype et doit être consolidé pour renforcer l'adhésion des médecins praticiens.

- **Nécessité d'une validation clinique prospective :** Les résultats actuels reposent sur une validation rétrospective indépendante. Une étude clinique prospective hospitalière est indispensable avant tout usage en routine médicale.

6.4 Perspectives de Recherche : Encodeur Visuel Hybride CNN-Transformer

Pour surmonter le plafond de performance actuel et améliorer la différenciation morphologique entre lésions visuellement proches (comme le mélanome précoce et le nœvus atypique), la perspective algorithmique la plus prometteuse réside dans la conception d'un **encodeur visuel hybride à double flux** (*Dual-Stream Hybrid Encoder*).

6.4.1 Fondements Théoriques de l'Architecture Hybride

Les *Vision Transformers* (ViT) [54] ont démontré une capacité supérieure à capturer les dépendances à longue portée et le contexte global d'une image grâce au mécanisme d'auto-attention multi-têtes (*Multi-Head Self-Attention*). Néanmoins, les ViTs sont exigeants en volume de données (*data-hungry*) et dépourvus des biais inductifs qui font l'efficacité des réseaux convolutifs (CNN) sur des jeux de données médicaux restreints.

Une architecture hybride synergique combine idéalement :

1. **Le flux local (Stream A - CNN) :** Un backbone convolutif *EfficientNet-B3* (ou B5) doté de pooling GeM, chargé d'extraire les motifs fins de texture, la régularité des bordures et les réseaux pigmentaires locaux.
2. **Le flux global (Stream B - ViT) :** Un transformeur visuel *ViT-B/16* avec plongements positionnels apprenables, initialisé par pré-entraînement auto-supervisé MAE [55], capturant la morphologie globale et l'asymétrie holistique de la lésion.

Le Tableau 6.2 détaille la répartition des rôles et la couche d'intégration avec notre mécanisme multimodal existant.

Tab. 6.2: Spécification de l'architecture hybride Dual-Encoder projetée

Composant	Spécification technique et Rôle fonctionnel
Encodeur Local (Flux A)	`EfficientNet-B3/B5` avec GeM Pooling. Responsable de la capture des textures fines, des globules et des bordures lésionnelles.
Encodeur Global (Flux B)	`ViT-B/16` (MAE pre-trained). Capture la structure sémantique globale, l'asymétrie d'ensemble et les corrélations spatiales distantes.
Couche de Fusion Visuelle	Projection linéaire apprenable des deux vecteurs dans un espace latent partagé, combinée par concaténation non-redondante.
Intégration Gated Cross-Attn	L'incorporation visuelle hybride sert de **Clé/Valeur (K/V)** dans le module d'attention croisée, guidée par les 7 variables cliniques servant de **Requête (Q)**.

6.4.2 Projections de Performances Attendues

En nous basant sur les benchmarks comparatifs publiés dans la littérature dermoscopique récente sous des protocoles analogues, le passage à l'architecture hybride permet d'envisager les gains estimés présentés au Tableau 6.3.

Tab. 6.3: Projections de performances attendues du modèle hybride (Test set $n = 654$)

Métrique	Référence V6.0	Hybride (Projeté)	Gain estimé
Exactitude (Accuracy)	74,31 %	**78,0 % à 81,0 %**	+3,7 à +6,7 points
Macro F1-Score	0,7315	**0,780 à 0,820**	+0,05 à +0,09
Mélanome (MEL) F1	0,6780	**0,750 à 0,780**	Réduction des faux négatifs

6.4.3 Défis Techniques et Stratégies de Mitigation

L'intégration d'un transformeur visuel implique des contraintes computationnelles que nous proposons d'adresser selon les stratégies suivantes :

- **Surcoût computationnel et mémoire VRAM :** L'ajout du flux ViT double le nombre de paramètres. La mitigation reposera sur l'activation du *Gradient Checkpointing*, l'entraînement en précision mixte (FP16) et l'utilisation de notre cache NVMe SSD optimisé.
- **Risque de surapprentissage (Data Scarcity) :** Utilisation des poids pré-entraînés MAE [55] et DeiT [56] (*Data-efficient Image Transformers*), régularisation par CutMix et lissage des poids SWA.

- **Instabilité d'entraînement multi-flux :** Application de taux d'apprentissage différentiels (1×10^{-5} pour le CNN, 1×10^{-4} pour la tête ViT) avec phase de réchauffement linéaire (*LinearLR Warm-up*).
- **Latence d'inférence et déploiement cascadaire :** Application de la quantification post-entraînement (INT8) et mise en œuvre d'une stratégie de déploiement en cascade : le modèle léger V6.0 traite le dépistage de routine, tandis que l'encodeur hybride complet est sollicité uniquement pour les cas à incertitude élevée.

6.5 Feuille de Route pour le Déploiement Clinique (Roadmap)

Pour concrétiser la transition de MedAssist AI vers une utilisation médicale certifiée, une feuille de route opérationnelle en quatre phases progressives est définie :

6.5.1 Phase I — Étude Pilote Hospitalière (6 à 12 mois)

Déploiement expérimental dans un service de dermatologie référent. Une étude prospective en double aveugle sur 500 patients consécutifs sera menée pour évaluer la concordance diagnostique et le gain de temps d'orientation.

6.5.2 Phase II — Explicabilité et Interopérabilité FHIR (12 à 18 mois)

Finalisation de l'affichage interactif Grad-CAM++ et développement de connecteurs normalisés **HL7 FHIR** pour une intégration transparente aux Dossiers de Santé Électroniques (DSE/DMP).

6.5.3 Phase III — Passage à l'Échelle et Inférence Edge (18 à 24 mois)

Migration du backend vers un orchestrateur Kubernetes (K8s) avec autoscaling horizontal, et portage du modèle optimisé (ONNX / TFLite) pour une inférence 100 % locale et autonome sur smartphone en zones dépourvues d'accès internet.

6.5.4 Phase IV — Certification Médicale et Conformité (24 à 36 mois)

Établissement du dossier technique pour l'obtention du marquage **CE Dispositif Médical (Classe IIa, règlement MDR 2017/745)** et démarches de pré-

soumission FDA 510(k), accompagnées d'audits de cybersécurité ISO 27001 et conformité RGPD/HDS.

6.6 Conclusion Finale et Clôture du Mémoire

Ce mémoire de Master en Génie Logiciel Avancé et Applications (GLAA) a démontré avec succès qu'une démarche rigoureuse unissant la modélisation avancée en apprentissage profond multimodal et les meilleures pratiques du génie logiciel moderne permet de lever les barrières historiques du transfert technologique en santé numérique.

En développant un écosystème complet — allant du prétraitement unifié multicentres et de l'attention croisée guidée par métadonnées, jusqu'à une application mobile de télémédecine opérationnelle et un service backend FastAPI à haute disponibilité —, **MedAssist AI** établit une base technologique crédible, reproductible et extensible. Ce travail ouvre ainsi la voie à une nouvelle génération d'outils d'assistance diagnostique : précis, accessibles, éthiques et résolument orientés vers l'amélioration du pronostic vital des patients atteints de lésions cutanées.

« Le véritable progrès ne réside pas seulement dans la complexité de nos modèles, mais dans notre capacité à les rendre accessibles, fiables et utiles pour préserver la vie humaine. »

Bibliographie

[1] World Health Organization, "Skin cancers." WHO Fact Sheets, 2023. [Online]. Available : `https://www.who.int/news-room/fact-sheets`.

[2] R. L. Siegel, K. D. Miller, N. S. Wagle, and A. Jemal, "Cancer statistics, 2023," *CA : A Cancer Journal for Clinicians*, vol. 73, pp. 17–48, Jan. 2023.

[3] D. S. Rigel, R. J. Friedman, A. W. Kopf, and D. Polsky, "ABCDE—an evolving concept in the early detection of melanoma," *Archives of Dermatology*, vol. 141, pp. 1032–1034, Aug. 2005.

[4] International Foundation for Dermatology, "Access to dermatological care and the global burden of skin disease." International League of Dermatological Societies Report, 2019.

[5] M. A. Marchetti, N. C. F. Codella, S. W. Dusza, D. A. Gutman, B. Helba, A. Kalloo, N. Mishra, H. Kittler, and A. C. Halpern, "Results of the 2016 International Skin Imaging Collaboration (ISIC) challenge," *Journal of the American Academy of Dermatology*, vol. 78, pp. 270–277, Feb. 2018.

[6] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, May 2015.

[7] A. Esteva, B. Kuprel, R. A. Novoa, J. Ko, S. M. Swetter, H. M. Blau, and S. Thrun, "Dermatologist-level classification of skin cancer with deep neural networks," *Nature*, vol. 542, pp. 115–118, Feb. 2017.

[8] H. A. Haenssle, C. Fink, R. Schneiderbauer, F. Toberer, T. Buhl, A. Blum, A. Kalloo, A. B. H. Hassen, L. Thomas, A. Enk, *et al.*, "Man against machine : Diagnostic performance of a deep learning convolutional neural network for dermoscopic melanoma recognition in comparison to 58 dermatologists," *Annals of Oncology*, vol. 29, pp. 1836–1842, Aug. 2018.

[9] J. Yap, W. Yolland, and P. Tschandl, "Multimodal skin lesion classification using deep learning," *Experimental Dermatology*, vol. 27, pp. 1261–1267, Nov. 2018.

[10] J. Kawahara, S. Daneshvar, G. Argenziano, and G. Hamarneh, "Seven-point checklist and skin lesion classification using multitask multimodal neural nets," *IEEE Journal of Biomedical and Health Informatics*, vol. 23, pp. 538–546, Mar. 2019.

[11] I. Oztel, C. Yolcu, and H. Kittler, "Skin lesion classification on mobile devices using deep learning," *Journal of Medical Systems*, vol. 47, p. 23, Feb. 2023.

[12] M. Tan and Q. V. Le, "EfficientNet : Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, vol. 97, (Long Beach, CA, USA), pp. 6105–6114, June 2019.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, (Long Beach, CA, USA), pp. 5998–6008, Dec. 2017.

[14] P. Tschandl, C. Rosendahl, and H. Kittler, "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions," *Scientific Data*, vol. 5, p. 180161, Aug. 2018.

[15] A. G. C. Pacheco, G. R. Lima, A. S. Salomão, B. Krohling, I. P. Biral, G. G. de Angelo, F. C. R. A. Jr., J. G. M. Esgario, A. C. Simora, P. B. C. Castro, F. B. Rodrigues, T. S. L. Frasson, and R. A. Krohling, "PAD-UFES-20 : A skin lesion dataset composed of patient data and clinical images collected from smartphones," *Data in Brief*, vol. 32, p. 106221, Oct. 2020.

[16] V. Rotemberg, N. Kurtansky, B. Betz-Stablein, L. Caffery, E. Chousakos, N. Codella, M. Combalia, S. Dusza, P. Guitera, D. Gutman, *et al.*, "A patient-centric dataset of images and metadata for identifying melanomas using clinical context," *Scientific Data*, vol. 8, p. 34, Jan. 2021.

[17] MCR-SL Consortium, "MCR-SL : Multi-class rare skin lesion dataset." Zenodo, 2024. [Online]. Available : `https://doi.org/10.5281/zenodo.17306338`.

[18] National Center for Biotechnology Information, "Dermoscopy overview and extra-diagnostic applications." StatPearls Publishing, Treasure Island (FL), 2023. [Online]. Available : `https://www.ncbi.nlm.nih.gov/books/NBK537131/`.

[19] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, (Lake Tahoe, NV, USA), pp. 1097–1105, 2012.

[20] S. Qian, Q. Chen, H. Guan, Y. Zou, and C. Yang, "Multimodal skin lesion classification using graph neural networks for clinical metadata fusion," *arXiv preprint arXiv :2201.07432*, 2022.

[21] Iterative, "DVC : Data version control for machine learning projects," 2023. [Online]. Available : `https://dvc.org/`.

[22] Databricks, "MLflow : An open source platform for the machine learning lifecycle," 2023. [Online]. Available : `https://mlflow.org/`.

[23] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation)." Official Journal of the European Union, L 119, Apr. 2016.

[24] S. Ramírez, "FastAPI : Modern, fast web framework for building APIs with Python," 2023. [Online]. Available : `https://fastapi.tiangolo.com/`.

[25] Google, "Flutter : Build apps for any screen," 2023. [Online]. Available : `https://flutter.dev/`.

[26] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA : MIT Press, 2016.

[27] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, (San Diego, CA, USA), May 2015.

[28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, (Las Vegas, NV, USA), pp. 770–778, June 2016.

[29] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, (Salt Lake City, UT, USA), pp. 7132–7141, June 2018.

[30] R. Wightman, "PyTorch image models (timm)." GitHub repository, 2019. [Online]. Available : `https://github.com/rwightman/pytorch-image-models`.

[31] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks ?," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, (Montreal, QC, Canada), pp. 3320–3328, 2014.

[32] N. Tajbakhsh, J. Y. Shin, S. R. Gurudu, R. T. Hurst, C. B. Kendall, M. B. Gotway, and J. Liang, "Convolutional neural networks for medical image analysis : Full training or fine tuning ?," *IEEE Transactions on Medical Imaging*, vol. 35, pp. 1299–1312, May 2016.

[33] C. Barata, M. E. Celebi, and J. S. Marques, "Improving dermoscopy image classification using color constancy," *IEEE Journal of Biomedical and Health Informatics*, vol. 19, pp. 1146–1152, May 2015.

[34] M. Combalia, N. C. F. Codella, V. Rotemberg, B. Helba, V. Vilaplana, O. Reiter, C. Carrera, A. Barreiro, A. C. Halpern, S. Puig, and J. Malvehy, "BCN20000 : Dermoscopic lesions in the wild," *arXiv preprint arXiv :1908.02288*, 2019.

[35] G. Argenziano, H. P. Soyer, S. Chimenti, R. Talamini, R. Corona, F. Sera, M. Binder, L. Bugatti, P. Carli, V. D. Giorgi, *et al.*, "Dermoscopy of pigmented skin lesions : Results of a consensus meeting via the Internet," *Journal of the American Academy of Dermatology*, vol. 48, pp. 679–693, May 2003.

[36] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, "Multimodal machine learning : A survey and taxonomy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, pp. 423–443, Feb. 2019.

[37] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing : Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, pp. 637–646, Oct. 2016.

[38] Google Developers, "TensorFlow Lite : Deploy machine learning models on mobile and IoT devices," 2023. [Online]. Available : `https://www.tensorflow.org/lite`.

[39] Docker Inc., "Docker : Empowering app development for developers," 2023. [Online]. Available : `https://www.docker.com/`.

[40] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Boston, MA, USA : Addison-Wesley Professional, 4th ed., 2021.

[41] T. Lee, V. Ng, R. Gallagher, A. Coldman, and D. McLean, "DullRazor : A software approach to hair removal from images," *Computers in Biology and Medicine*, vol. 27, pp. 533–543, Nov. 1997.

[42] Z. Apalla, A. Lallas, E. Sotiriou, E. Lazaridou, and D. Ioannides, "Epidemiological trends in skin cancer," *Dermatology Practical & Conceptual*, vol. 7, pp. 1–6, Apr. 2017.

[43] F. Radenović, A. Iscen, G. Tolias, O. Chum, and G. Kordopatis-Zilos, "Fine-tuning CNN image retrieval with a parametric loss," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, pp. 1655–1668, July 2018.

[44] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, (Venice, Italy), pp. 2980–2988, Oct. 2017.

[45] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, (Las Vegas, NV, USA), pp. 2818–2826, June 2016.

[46] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, (New Orleans, LA, USA), May 2019.

[47] I. Loshchilov and F. Hutter, "SGDR : Stochastic gradient descent with warm restarts," in *Proc. 5th Int. Conf. Learning Representations (ICLR)*, (Toulon, France), Apr. 2017.

[48] P. Izmailov, D. Podoprikhin, T. Garipov, D. Vetrov, and A. G. Wilson, "Averaging weights leads to wider optima and better generalization," in *Proc. 34th Conf. Uncertainty in Artificial Intelligence (UAI)*, (Monterey, CA, USA), pp. 876–885, Aug. 2018.

[49] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning : A survey of case studies," *ACM Computing Surveys*, vol. 55, pp. 1–38, Dec. 2022.

[50] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix : Regularization strategy to train strong classifiers with localizable features," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, (Seoul, South Korea), pp. 6023–6032, Oct. 2019.

[51] O. Troyanskaya, M. Cantor, G. Sherlock, P. Brown, T. Hastie, R. Tibshirani, D. Botstein, and R. B. Altman, "Missing value estimation methods for DNA microarrays," *Bioinformatics*, vol. 17, pp. 520–525, June 2001.

[52] M. E. C. P. de Souza *et al.*, "Performance and scalability evaluation of microservices-based REST APIs using Locust," *Software : Practice and Experience*, vol. 51, no. 9, pp. 1920–1935, 2021.

[53] K. Goddard, A. Roudsari, and J. C. Wyatt, "Automation bias : a systematic review of systematic bias in human computer interaction for clinical decision support systems," *BMC Medical Informatics and Decision Making*, vol. 12, p. 75, 2012.

[54] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words : Transformers for image recognition at scale," in *Proc. 9th Int. Conf. Learning Representations (ICLR)*, 2021.

[55] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 16000–16009, 2022.

[56] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, vol. 139, pp. 10347–10357, 2021.